UNIT I – INTRODUCTION TO DBMS – 3 HRS

fppt.com

DBMS Overview

- A **database management system (DBMS)** is a software package designed to define, manipulate, retrieve and manage data in a database.
- A DBMS generally manipulates the data itself, the data format, field names, record structure and file structure. It also defines rules to validate and manipulate this data.
- A DBMS relieves users of framing programs for data maintenance. Fourth generation query languages, such as SQL, are used along with the DBMS package to interact with a database.
- Some other DBMS examples include: MySQL, SQL Server, Oracle, dBASE, FoxPro, MS-Access etc.

Database

- A database is a collection of information that is organized so that it can be easily accessed, managed and updated. Data is organized into rows, columns and tables, and it is indexed to make it easier to find relevant information.

fppt.com

Characteristics of DBMS

- Provides security and removes redundancy (duplication)
- Insulation between programs and data abstraction
- Support of multiple views of the data
- Sharing of data and multiuser transaction processing
- DBMS allows entities and relations among them to form tables.
- It follows the ACID concept (Atomicity, Consistency, Isolation, and Durability).
- DBMS supports multi-user environment that allows users to access and manipulate data in parallel.

fppt.com

DBMS vs. Flat File

DBMS	Flat File Management System
Multi-user access	It does not support multi-user access
Design to fulfill the need for small and large businesses	It is only limited to smaller DBMS system.
Remove redundancy and Integrity	Redundancy and Integrity issues
Expensive. But in the long-term Total Cost of Ownership is cheap	It's cheaper
Easy to implement complicated transactions	No support for complicated transactions

fppt.com

Objective of DBMS

1. Eliminate redundant data.
2. Make access to the data easy for the user.
3. Provide for mass storage of relevant data.
4. Protect the data from physical harm and un-authorized systems.
5. Allow for growth in the data base system.
6. Make the latest modifications to the data base available immediately.
7. Allow for multiple users to be active at one time.
8. Provide prompt response to user requests for data.

fppt.com

Importance of DBMS

- A database management system is important because it manages data efficiently and allows users to perform multiple tasks with ease.
- A database management system stores, organizes and manages a large amount of information within a single software application. Use of this system increases efficiency of business operations and reduces overall costs.
- Database management systems are important to businesses and organizations because they provide a highly efficient method for handling multiple types of data.
- Some of the data that are easily managed with this type of system include: employee records, student information, payroll, accounting, project management, inventory and library books. These systems are built to be extremely versatile.
- Without database management, tasks have to be done manually and take more time. Data can be categorized and structured to suit the needs of the company or organization.
- Data is entered into the system and accessed on a routine basis by assigned users. Each user may have an assigned password to gain access to their part of the system. Multiple users can use the system at the same time in different ways.

fppt.com

Advantages of DBMS

- DBMS offers a variety of techniques to store & retrieve data
- DBMS serves as an efficient handler to balance the needs of multiple applications using the same data
- Application programmers never exposed to details of data representation and storage.
- A DBMS uses various powerful functions to store and retrieve data efficiently.
- Offers Data Integrity and Security
- The DBMS implies integrity constraints to get a high level of protection against prohibited access to data.
- A DBMS schedules concurrent access to the data in such a manner that only one user can access the same data at a time
- Reduced Application Development Time

fppt.com

Disadvantages of DBMS

- Cost of Hardware and Software of a DBMS is quite high which increases the budget of the organization.
- Most database management systems are often complex systems, so the training for users to use the DBMS is required.
- In some organizations, all data is integrated into a single database which can be damaged because of electric failure or database is corrupted on the storage media
- Use of the same program at a time by many users sometimes lead to the loss of some data.
- DBMS can't perform sophisticated calculations.

fppt.com

Application of DBMS

Sector	Use of DBMS
Banking	For customer information, account activities, payments, deposits, loans, etc.
Airlines	For reservations and schedule information.
Universities	For student information, course registrations, colleges and grades.
Telecommunication	It helps to keep call records, monthly bills, maintaining balances, etc.
Finance	For storing information about stock, sales, and purchases of financial instruments like stocks and bonds.
Sales	Use for storing customer, product & sales information.
Manufacturing	It is used for the management of supply chain and for tracking production of items. Inventories status in warehouses.
HR Management	For information about employees, salaries, payroll, deduction, generation of paychecks, etc.

fppt.com

UNIT - 2

Database Design, Architecture and Model

Table of Contents

- Overview of The Database Designing Process and View of Data
- Structure of Database Management System
- Level Database Architecture and Data Independence
- Database Languages: DDL, DML, QBE
- Data Models: Hierarchical, Network, Relational, E-R Model, Object Base Data Model
- E-R Diagram: Concepts, Relationship, Entity Relationship Diagram, Weak Entity Sets, Strong Entity Set, Aggregation, Generalization, Converting ER Diagrams to Tables.

Overview of Database Designing Process

Definition:

Database design refers to the process of defining the structure, storage, and organization of data in a database to efficiently support queries and operations.

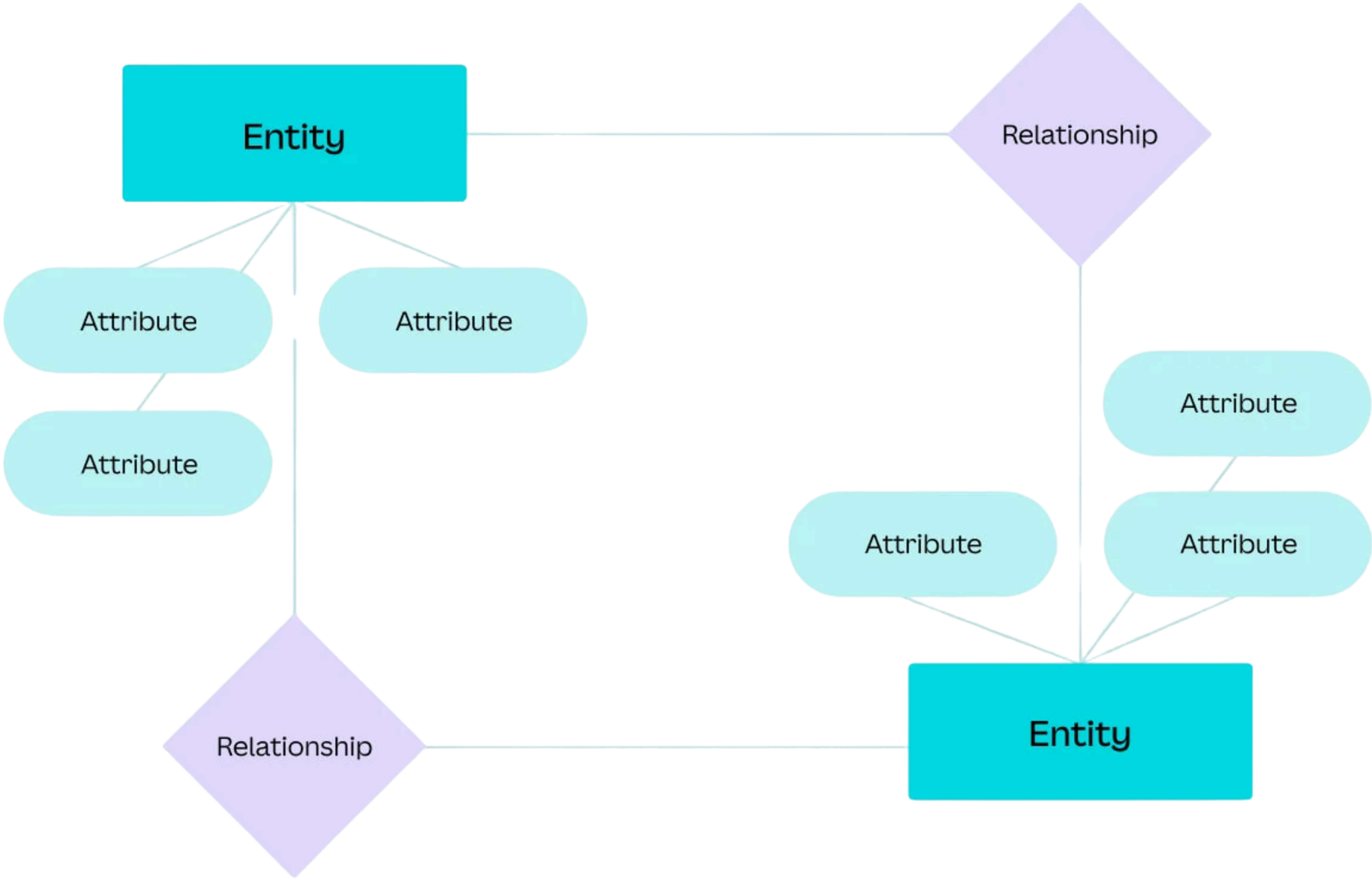
Steps in the Database Design Process:

- **Requirement Analysis:**
  - Understand the purpose of the database.
  - Identify what data will be stored and how it will be used.
  - **Example**: An e-commerce website needs a database for storing customer details, product listings, and orders.

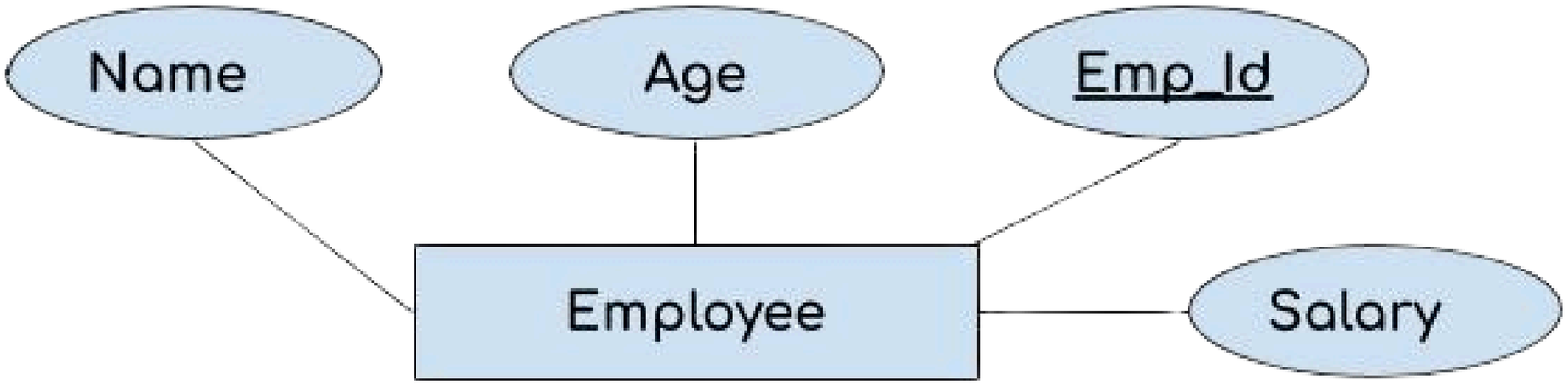
Overview of Database Designing Process

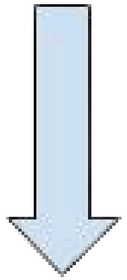
- **Conceptual Design:**
  - Design a visual representation of data using Entity-Relationship (ER) Diagrams.
  - Entities are objects like Customer and Product, and Relationships show how they interact, such as "Customer places Order."
  - Example: Draw an ER diagram with Customer, Order, and Product connected with lines to show relationships.

ER Diagram


Entity
Relationship
Attribute
Attribute
Attribute
Attribute
Attribute
Attribute
Attribute
Relationship
Entity

ER - Diagram of Employee



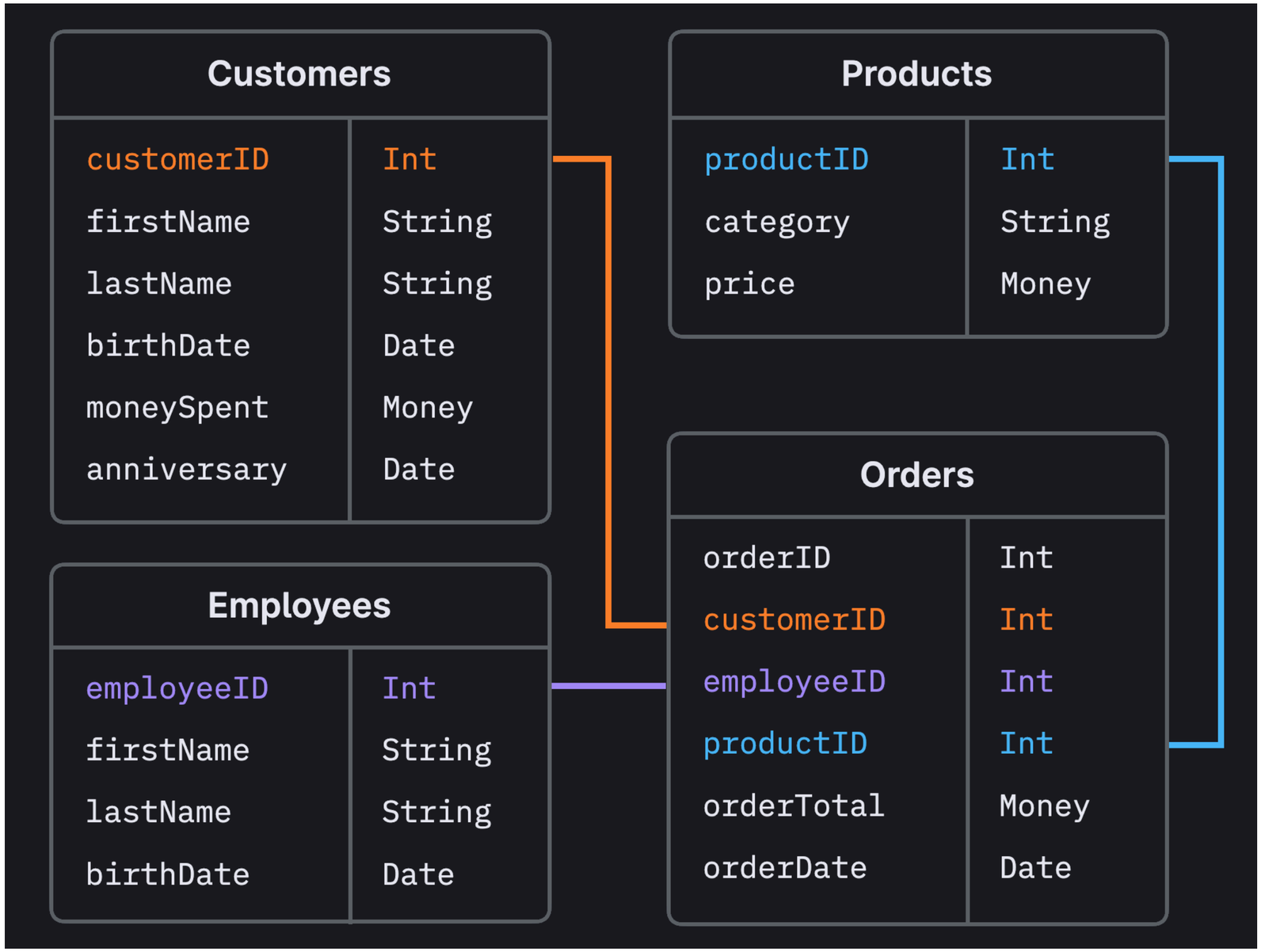


Emp_Id	Name	Age	Salary

Overview of Database Designing Process

- **Logical Design:**
  - Convert ER diagrams into a tabular form called the Relational Model.
  - Define tables, columns, and relationships using Primary Keys (unique identifiers) and Foreign Keys (connections between tables).
  - Example: Create a Customer table with columns like CustomerID, Name, and Email.
  -

Logical Design Example



Overview of Database Designing Process

- **Physical Design:**
  - Implement the database in a specific software like MySQL, Oracle, or PostgreSQL.
  - Define how data will be stored physically on disks.
  - Example: Configure indexes for fast searching or partitioning large tables for better performance.

Importance of Database Design

- **Prevents Redundancy**: Reduces unnecessary duplication of data, saving storage space and ensuring data integrity.
- **Improves Performance**: Ensures that queries are executed quickly and efficiently.
- **Supports Maintenance**: Makes the database easier to update and expand as business requirements change.
- **Ensures Data Integrity and Security**: Ensures that data remains accurate, consistent, and secure against unauthorized access.

View of Data (Data abstraction levels)

The view of data in a database refers to the different ways the database presents data to users or systems, depending on their role and needs. It's based on the idea that not everyone should see or deal with data at the same level of detail.

- **Physical View** – Describes how and where the data is actually stored in the system (files, blocks, indexes). This is handled by the database internals and usually hidden from users.

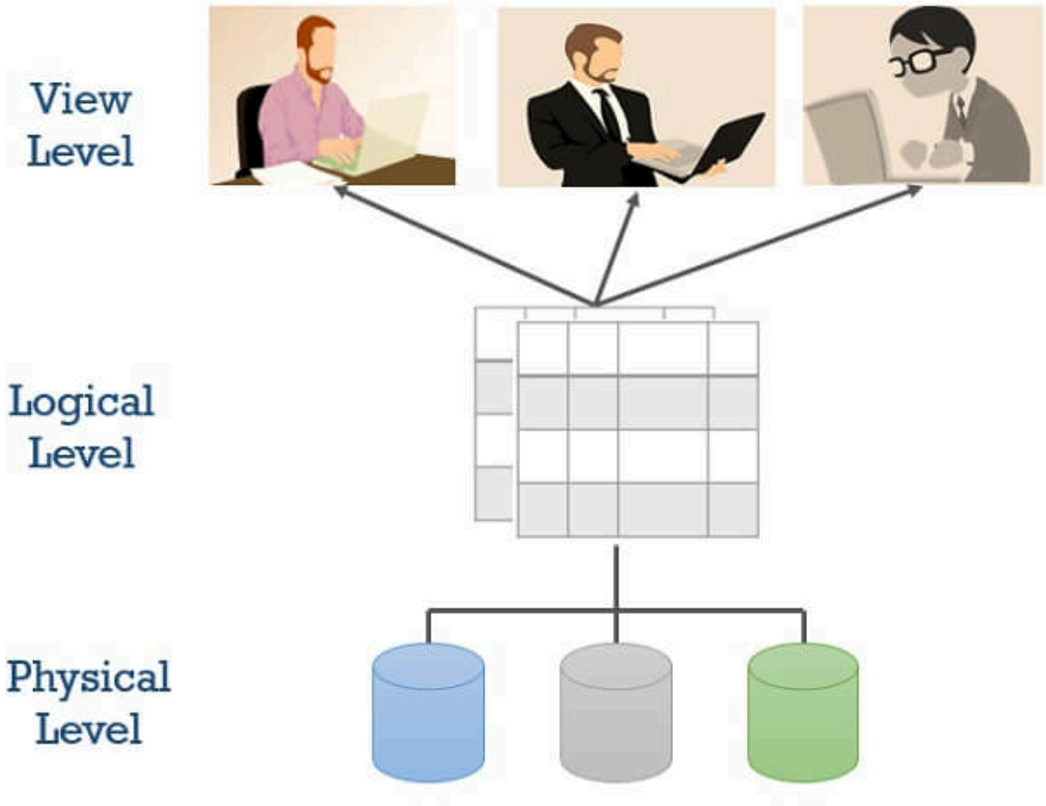
- **Logical View** – Describes what data is stored and how it is logically organized (tables, columns, relationships, constraints). It's the overall design of the database from a developer's perspective.

- **User/External View** – Describes the specific part of the data a particular user or application can access. It can hide certain fields or records for security or simplicity.

Analogy

Think of a restaurant:

- **Physical view** → The kitchen's ingredients and storage.
- **Logical view** → The recipes (how dishes are prepared from ingredients).
- **User view** → The menu (what customers see and order).



Structure of Database Management System

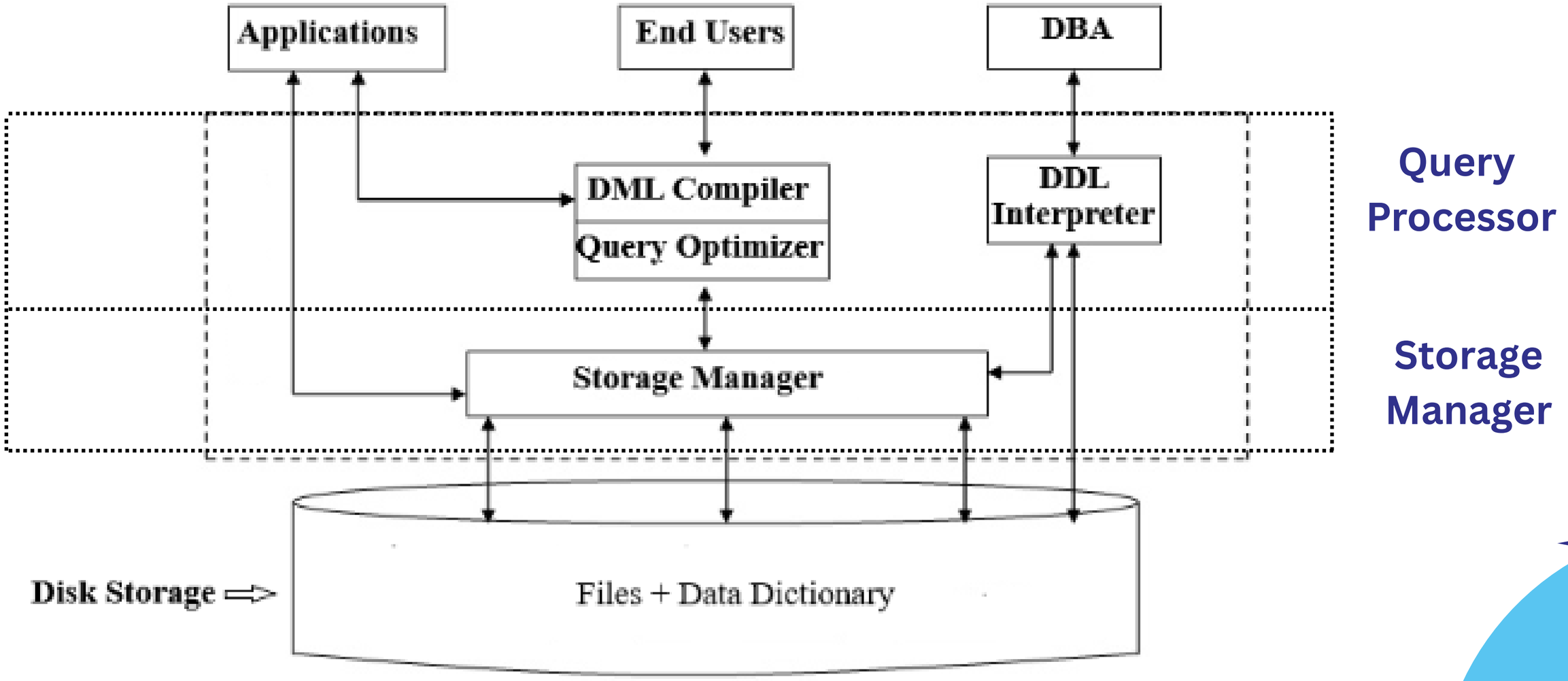


Fig: Structure of Database Management System

Structure of Database Management System

Query Processor

The Query Processor receives the queries (requests) from the user and interprets them in the form of instructions. It also executes the instructions received from the DML Compiler. It has the following four components:

1. **DML Compiler:** It converts the DML (Data Manipulation Language) Instructions into Machine Language (low-level language).
2. **DDL Interpreter:** It interprets the DDL (Data Definition Language) Instructions and stores the record in a data dictionary (in a table containing meta-data)
3. **Query Optimizer:** It executes the DML Instructions and picks the lowest cost evaluation plan out of all the alternatives present.

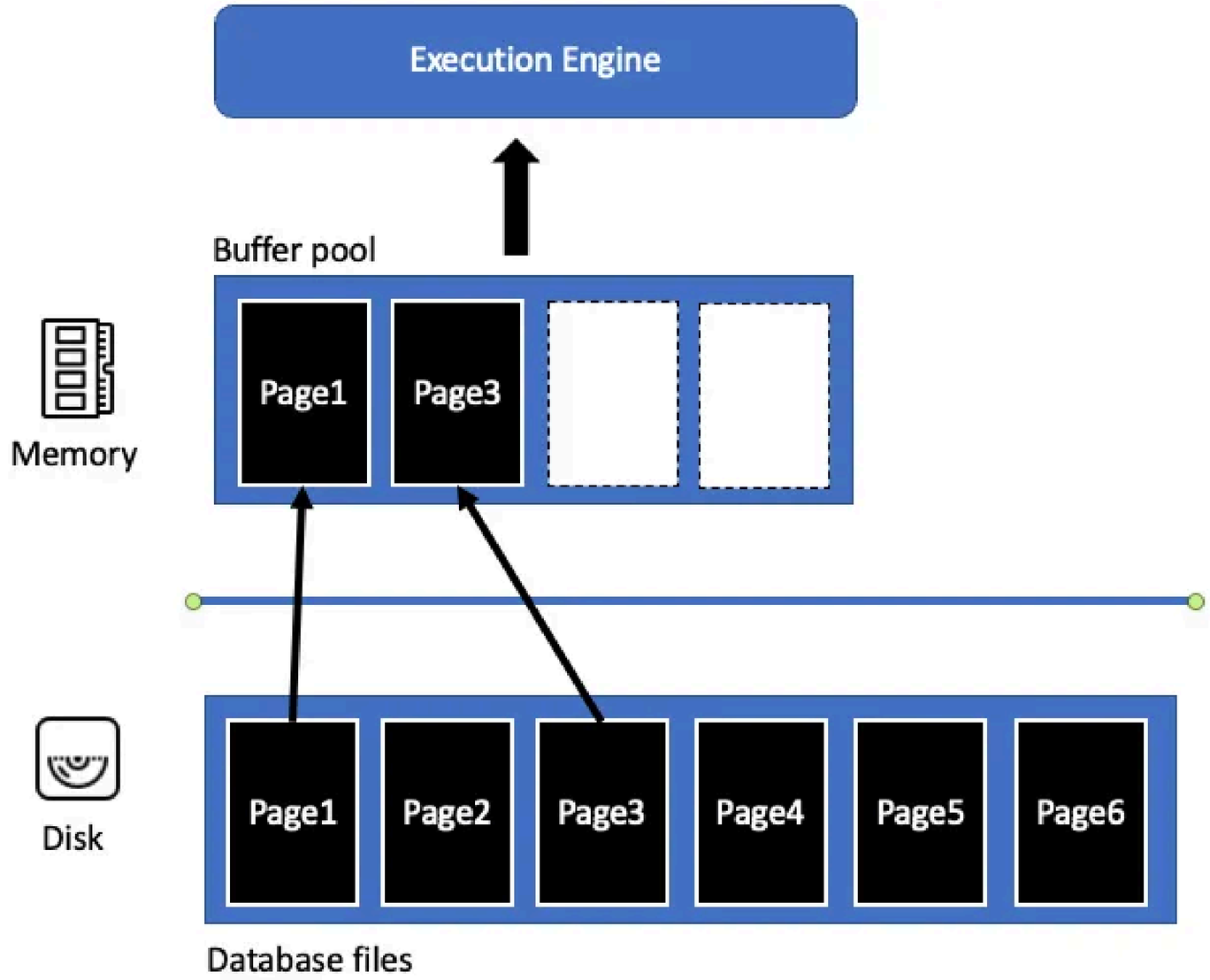
Structure of Database Management System

Storage Manager

Storage manager acts as the interface between the data stored and the queries received from the end-user. This component in the Structure of DBMS is responsible for the constraints applied to the data so that it **remains consistent**. It encapsulates the following modules:

1. **Authorization and Integrity Manager**: It checks the authority of various users who access data and the Integrity Constraints of the database.
2. **Transaction Manager**: Its job is to assure the system remains in a proper state during the transaction process. It also ensures that concurrent transactions are executed without any conflict.
3. **File Manager**: It manages the space allocation of files in disk and data structures which stores information in the database.
4. **Buffer Manager**: It manages the transfer of data between the secondary and main memory. It also decides what data should be cached in the memory.

Buffer Manager



Structure of Database Management System

Disk Storage

The Disk Storage in the Structure of DBMS represents the space where data is stored. It has the following components:

1. **Files**: These are responsible for storing the data.
2. **Data Dictionary**: It is the repository that maintains the information of the database object and maintains the metadata.
3. **Indices**: These are the keys that are used for faster retrieval of data.

Data Dictionary

DATA

emlployee_id	first_name	last_name	nin	dept_id
44	Simon	Martinez	HH 45 09 73 D	1
45	Thomas	Goldstein	SA 75 35 42 B	2
46	Eugene	Comelsen	NE 22 63 82	2
47	Andrew	Petculescu	XY 29 87 61 A	1
48	Ruth	Stadick	MA 12 89 36 A	15
49	Barry	Scardelis	AT 20 73 18	2
50	Sidney	Hunter	HW 12 94 21 C	6
51	Jeffrey	Evans	LX 13 26 39 B	6
52	Doris	Berndt	YA 49 88 11 A	3
53	Diane	Eaton	BE 08 74 68 A	1

DATA DICTIONARY (METADATA)

Column	Data Type	Description
emlployee_id	int	Primary key of a table
first_name	nvarchar(50)	Employee first name
last_name	nvarchar(50)	Employee last name
nin	nvarchar(15)	National Identification Number
position	nvarchar(50)	Current postion title, e.g. Secretary
dept_id	int	Employee deparmtnet. Ref: Departmetns
gender	char(1)	M = Male, F = Female, Null = unknown
employment_start_date	date	Start date of employment in organization.
employment_end_date	date	Employment end date.

Data Independence

Definition: It refers to the ability to modify the database system at one level without affecting other levels. This ensures flexibility and protects applications or users from disruptions when changes are made to the database.

Types of data independence:

Logical Data Independence

Changes to the logical schema (structure of the database) won't affect the external views or the applications using the database.

Logical Schema: Describes what data is stored and relationships between the data (e.g., tables, columns, constraints).

External View: The specific part of the database seen or used by users or programs.

Data Independence

Example:

Let’s say we add a new table for storing order history in an e-commerce database.

- **Logical Schema Update**: Add a new `OrderHistory` table.
- **External View**: Reports and dashboards for customers are still showing their orders as usual. Users or programs querying the database don’t need to know the new table was added.

Physical Data Independence

- Changes to the physical storage of data won’t affect the logical schema.
  - **Physical Schema**: Deals with how data is stored in files, formats, or indexes.

Data Independence

Example:

Suppose you move the database from HDD (Hard Disk Drive) to SSD (Solid-State Drive) for faster performance.

- **Physical Storage Update**: Change the storage device.
- **Logical Schema**: Tables like Customers or Orders remain the same, so queries like SELECT * FROM Customers still work as expected.

Levels of Database Architecture

The database architecture provides a structured way to organize and interact with data, ensuring flexibility, efficiency, and scalability. There are three main levels of database architecture: Single-tier, Two-tier, and Three-tier. These tiers define how data is accessed and processed.

Single-Tier Architecture

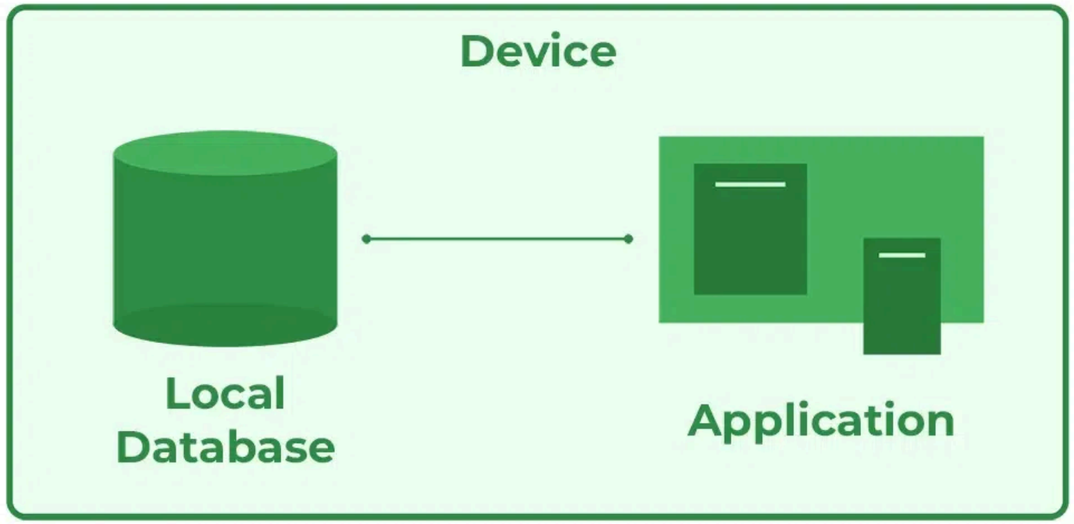
Definition: In this architecture, the database and application reside on the same system. It is a simple design suitable for small-scale, standalone applications.

Characteristics:

- Direct interaction with the database.
- No network is involved as everything runs on the same machine.
- Limited scalability and security.

Single-Tier Architecture

- **Example:**
  A personal finance tracker created in MS Access
  - A single application installed on your PC stores and retrieves your expense data from the local database.
  - No other users or systems can interact with it.
- **Use Case:**
  Small businesses or individual users, where only one user needs access to the database.



Levels of Database Architecture

- **Two-Tier Architecture**

Definition: This architecture separates the client (front-end) from the database server (back-end). The client sends queries to the database, and the database processes these queries and sends the results back to the client.

Characteristics:

- The user interface and database are separate.
- A direct connection exists between the client and the database server.
- More secure and scalable than single-tier architecture.

Two-Tier Architecture

Example:

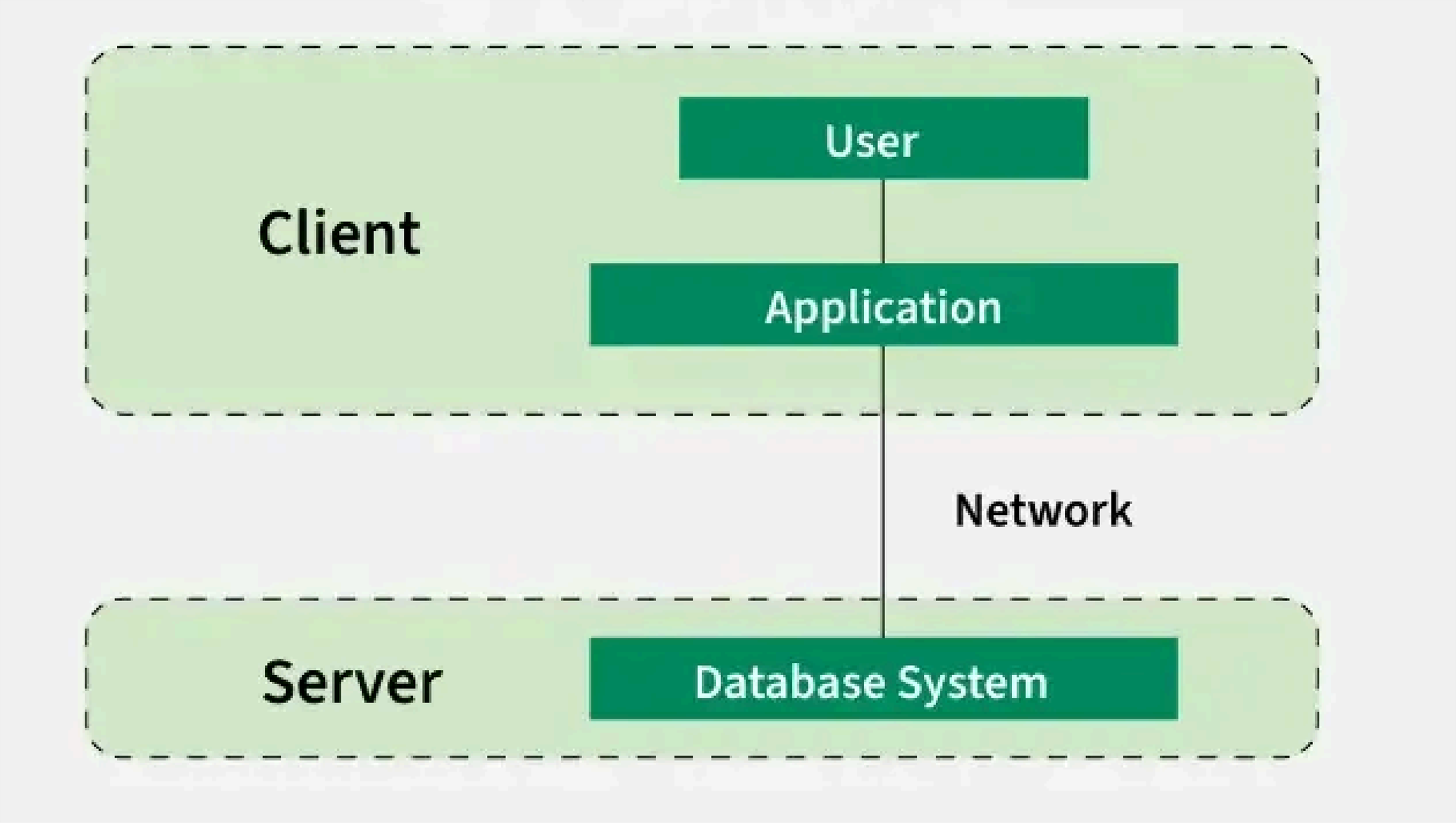
A payroll system for a small business:

- **Client**: The admin interface where the HR department enters employee details and views salary reports.
- **Database Server**: A MySQL database stores employee data, salary calculations, and reports.

Use Case:

Small to medium-sized applications where a few users need access.

Two-Tier Architecture



Levels of Database Architecture

Three-Tier Architecture

- **Definition**: This architecture introduces an additional application layer between the client and the database. It is the most common and scalable architecture, used in modern enterprise systems.
- **Characteristics:**
  - **Three Layers:**
    i. **Presentation Layer (Client):** The user interface where users interact with the system.
    ii. **Application Layer (Middle Layer)**: The logic that processes user requests and interacts with the database.
    iii. **Database Layer**: The backend where data is stored and retrieved.
  - The client never interacts directly with the database.
  - Provides enhanced security and scalability.

Three-Tier Architecture

Example:

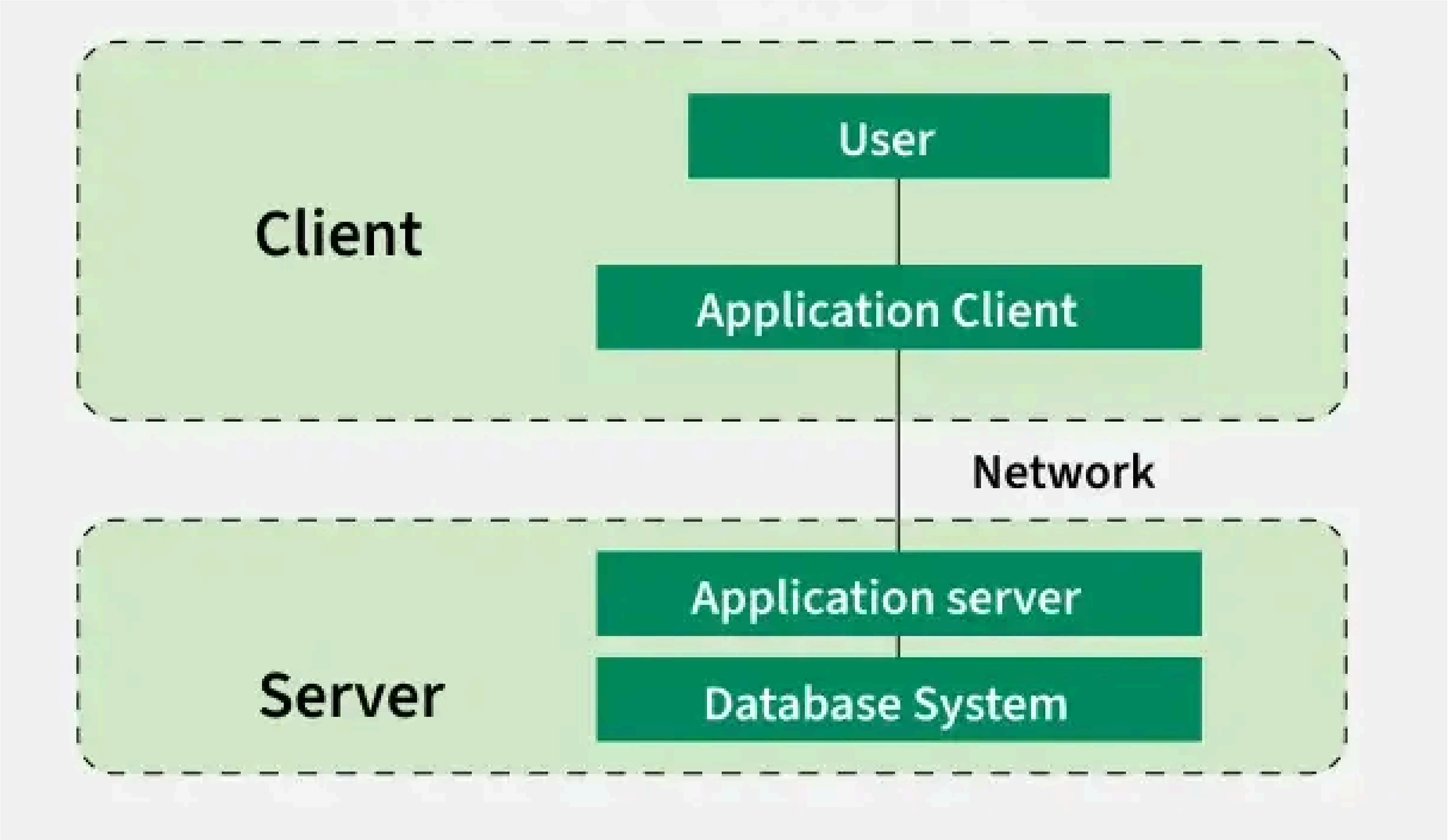
An online shopping application:

- **Presentation Layer**: The web interface or mobile app that displays products to customers.
- **Application Layer**: Handles business logic, like managing orders, calculating discounts, or validating payments.
- **Database Layer**: Stores data like product inventory, customer details, and order history.

Use Case:

Large-scale systems like e-commerce platforms, banking systems, or university management systems, where multiple users access the database simultaneously.

Three-Tier Architecture



Database Languages

Definition: These are specialized languages used to interact with and manage a database system.

Why Important?

- Databases store critical data for businesses, governments, and personal applications.
- Effective data handling ensures accuracy, security, and efficiency.

Classification of Database Languages:

- **DDL (Data Definition Language):** For defining the structure.
- **DML (Data Manipulation Language):** For interacting with data.
- **QBE (Query By Example):** For querying using graphical interfaces.

Data Definition Language (DDL)

Definition: Used to create, modify, and delete database objects such as tables, indexes, and views.

Key Characteristics:

- Operates at the schema level.
- Changes are permanent once changed.
- Ensures database structure aligns with business needs.

Key Commands:

- **CREATE** – To create tables, indexes, and other database objects.
- **ALTER** – To modify existing database objects.
- **DROP** – To delete database objects.
- **TRUNCATE** – To quickly remove all data from a table.

Data Definition Language (DDL)

Example:

- CREATE TABLE Students (
  ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
  );
- ALTER TABLE Students ADD COLUMN Gender CHAR(1);
- DROP TABLE Students;

Data Manipulation Language (DML)

Definition: Enables users to manipulate the data stored in a database.

Features of DML

- **Focus on Data**: Primarily works on rows and columns in a database.
- **Transaction Support**: Allows rolling back changes if required using ROLLBACK.
- **CRUD Operations**: Ensures full lifecycle management of data (Create, Read, Update, Delete).

Key Commands:

- **SELECT** – To retrieve data.
- **INSERT** – To add new data.
- **UPDATE** – To modify existing data.
- **DELETE** – To remove data.

Data Manipulation Language (DML)

Example:

- INSERT INTO Students (ID, Name, Age) VALUES (1, 'Alice', 20);
- UPDATE Students SET Age = 21 WHERE ID = 1;
- DELETE FROM Students WHERE ID = 1;
- SELECT * FROM Products WHERE Price > 100;

Query By Example (QBE)

Definition: A visual method for querying databases using a graphical interface.

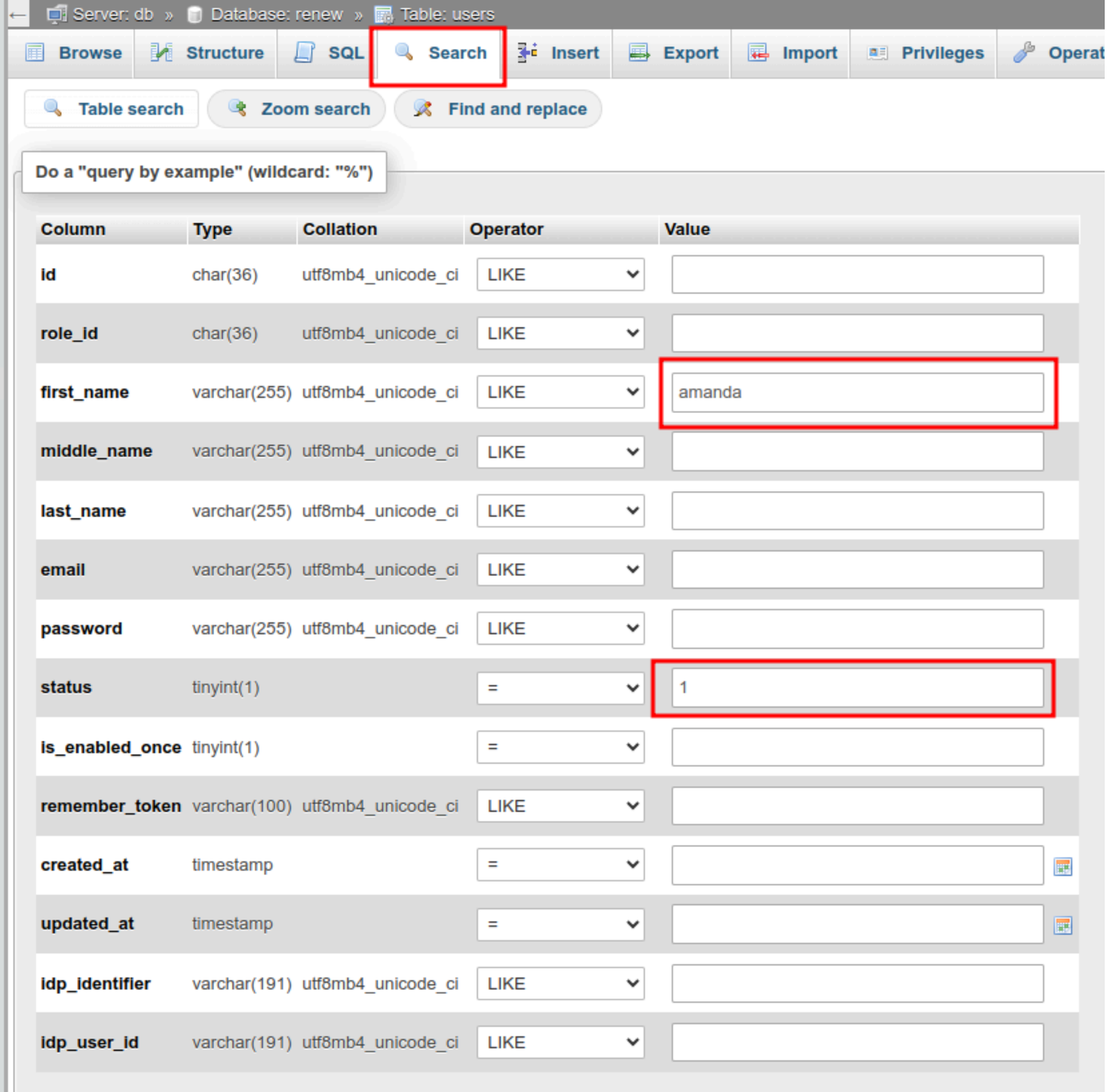
Key Characteristics:

- No need for SQL knowledge.
- Ideal for non-programmers.
- Converts graphical input into SQL commands automatically.

How it Works:

- User enters desired criteria into fields in a GUI.
- System generates an equivalent SQL query.

Query By Example (QBE)



Query By Example (QBE)

Server: db » Database: renew » Table: users

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Table search | Zoom search | Find and replace

Show search criteria

Showing rows 0 - 0 (1 total, Query took 0.0023 seconds.)

```
SELECT * FROM `users` WHERE `first_name` LIKE 'amanda' AND `status` = 1
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

	id	role_id	first_name	middle_name	last_name	email	password	status	is_enabled_once
Edit Copy Delete	99d4d589-d8ff-4fb5-a94f-820a3d21acd7	98261985-5590-4fba-83ed-c884b74ec4e4	Amanda	*NULL*	Långberg	amanda@hbgdesignlab.se	$2y$10$/zS/HS4M48dxFFi3Z3clRe548QdP5ja9lu8dBC98p2G...	1	1

Check all | *With selected:* Edit | Copy | Delete | Export

Introduction to Data Models

Definition: A data model is a simple plan or blueprint that shows how data is organized, stored, and connected in a database, and how it can be used.

Importance of Data Models:

- Helps organize and structure data clearly
- Defines relationships between different data items
- Makes it easier to design and build databases
- Ensures data consistency and accuracy
- Simplifies communication between developers, users, and database designers
- Guides how data will be stored, retrieved, and updated
- Helps in reducing redundancy and improving efficiency

Introduction to Data Models

Categories of Data Models:

- Hierarchical Model
- Network Model
- Relational Model
- Entity-Relationship (E-R) Model

Hierarchical Model

Definition: Represents data in a tree-like structure where each child node has a single parent, but a parent can have multiple children.

Real-World Example: *File System*

- **Parent**: Main folder (e.g., Documents).
- **Child**: Subfolders (e.g., Work, Personal).
- **Sub-Child**: Files (e.g., Resume.doc, Budget.xlsx).

```
Documents
├── Work
│   ├── Resume.doc
│   └── Reports.xlsx
└── Personal
    ├── Photos
    └── Budget.xlsx
```

Hierarchical Model

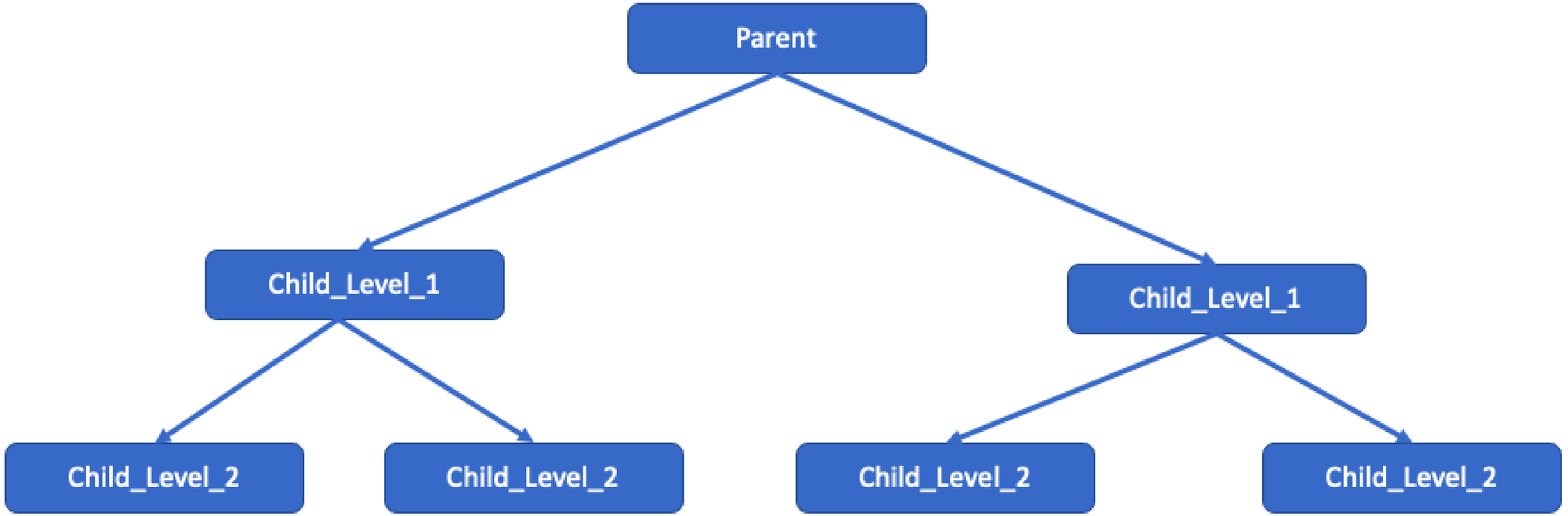


Fig: Hierarachical Database Model

Hierarchical Model

Advantages:

- Simple and easy to understand
- High security, as a child record can only be accessed or modified through its parent
- Faster searching when the parent record is known
- Well-suited for handling one-to-many relationships
- Follows a clear top-down structure for data organization

Hierarchical Model

Disadvantages:

- Old-fashioned and outdated compared to modern models
- Non-flexible, making modification or addition of child records complex
- Cannot handle many-to-many relationships
- Higher data redundancy due to duplicate storage in different locations
- Deleting a parent node automatically deletes all its associated child nodes

Network Model

Definition: The network database model is an extension of the hierarchical model where each child can have multiple parents, storing records in interconnected sets for greater flexibility, but with more complex operation, maintenance, and lower user-friendliness and security.

Real-World Example: *Airline Route Management*

- Nodes: Cities (e.g., New York, London, Tokyo).
- Edges: Flights between cities.

Network Model

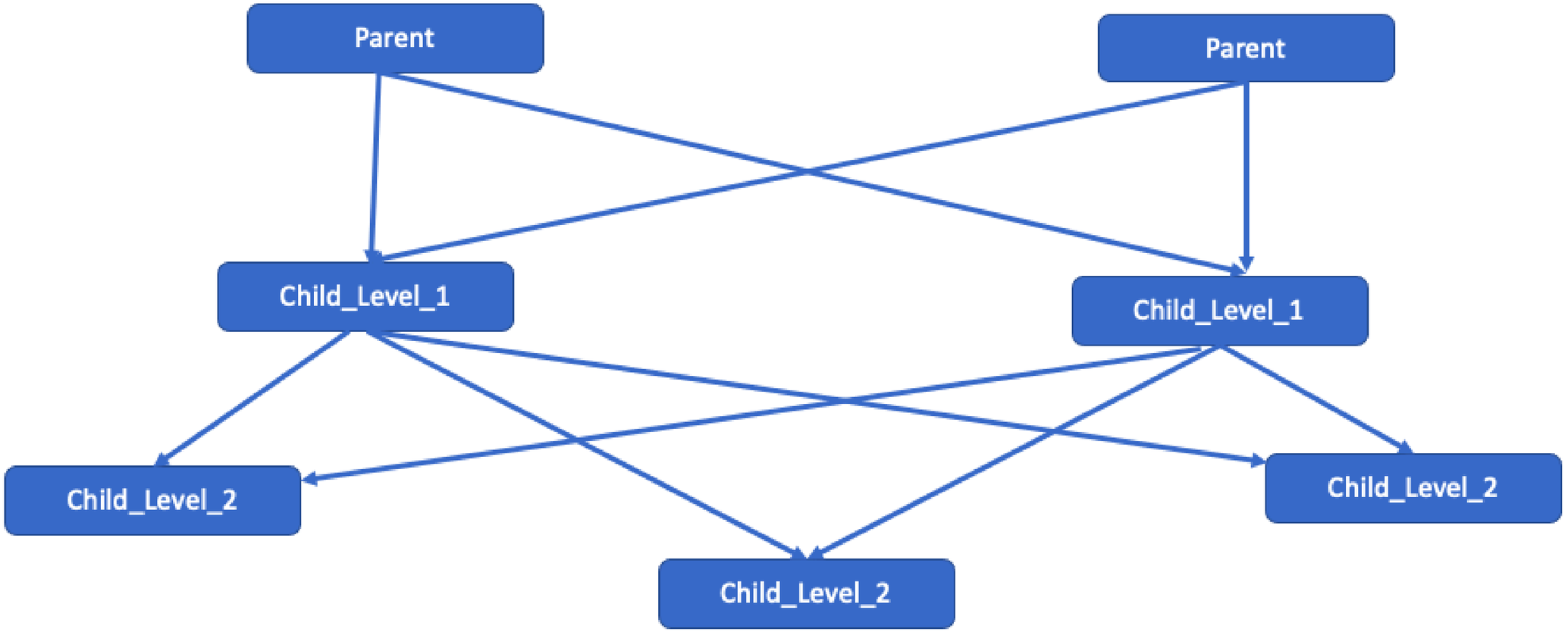


Fig: Network Database Model

Network Model

Advantages:

- 'many-to-many' relationship is possible which makes it more flexible than hierarchical
- It reduces data redundancy i.e. repetition of data.
- Searching is fast because of multi-directional pointers.

Use Case: University course registration system

- Example: Students can enroll in multiple courses, and each course can have many students. Direct links (pointers) connect students and courses for fast access.

Network Model

Disadvantages:

- It is one of the complex database models as it can also follow bottom up approach.
- As nodes increases, table and relationship increases which needs larger program to handle.
- Pointers needed in this model increases over head of storage.
- It is less secured compared to hierarchical model, as its nodes are open to all.

Relational Model

Definition: The relational database model stores data in tables of rows and columns, where each table represents a relation and each row a record. It is easy to understand, flexible, and works efficiently even on limited systems, but has lower processing efficiency and requires defined data relationships for processing.

Real-World Example: *Online Shopping System*

- Products Table: Contains product details.
- Orders Table: Stores order information.

Relational Model

1. **Products:**

ProductID	Name	Price
1	Smartphone	800
2	Laptop	1200

2. **Orders:**

OrderID	ProductID	Quantity
1001	1	2
1002	2	1

Relational Model

How They Work Together:

Using SQL, you can query both tables to find out what a customer ordered:

```
SELECT Products.Name, Orders.Quantity
FROM Products
JOIN Orders ON Products.ProductID = Orders.ProductID;
```

Advantages:

- Easy implementation of common fields and rules across related tables
- Minimal data redundancy
- Supports normalization of complex tables into simpler ones
- Enables rapid data processing
- Allows adding new tables without redesigning the database

Relational Model

Disadvantages:

- More complex than other database models
- Too many rules make it less user-friendly
- Performance may degrade with very large datasets and complex queries
- Requires significant design effort to establish proper relationships and normalization

Entity-Relationship (E-R) Model

Definition: The ER data model is a conceptual framework used to represent and describe the structure of a database. It shows entities (objects), attributes (properties of entities), and relationships (associations between entities) in a clear diagrammatic form called an ER diagram.

Use Case:

- Used during database design to plan how data will be organized before creating the actual database.
- Helps communicate the database structure between developers, analysts, and users.

Entity-Relationship (E-R) Model

Example:

- A Library Management System:
  - Entities: Book, Member, Librarian
  - Attributes:
    - Book → Book_ID, Title, Author
    - Member → Member_ID, Name, Contact
  - Relationships:
    - Member borrows Book
    - Librarian issues Book

An ER diagram can visually show these entities and how they are related.

Entity-Relationship (E-R) Model

Pros (Advantages):

- Provides a clear and visual representation of database structure
- Helps in accurate database design and reduces errors
- Easy to understand for both technical and non-technical users
- Supports complex relationships between entities
- Can be easily converted into relational database tables

Cons (Disadvantages):

- Only a conceptual model, does not store actual data
- Can become complex for very large databases
- Requires expertise to design correctly
- Does not define physical storage details or performance optimizations

Comparison of Data Models

Feature	Hierarchical	Network	Relational	E-R Model
Structure	Tree-like	Graph-like	Tables (rows & columns)	Diagrammatic (conceptual)
Navigation	Top-down	Pointer-based	SQL-based	Visual-only
Best for	Simple hierarchies	Complex relationships	General-purpose use	Conceptual design
Real-World Example	File systems	Airline route maps	Online shopping systems	University database

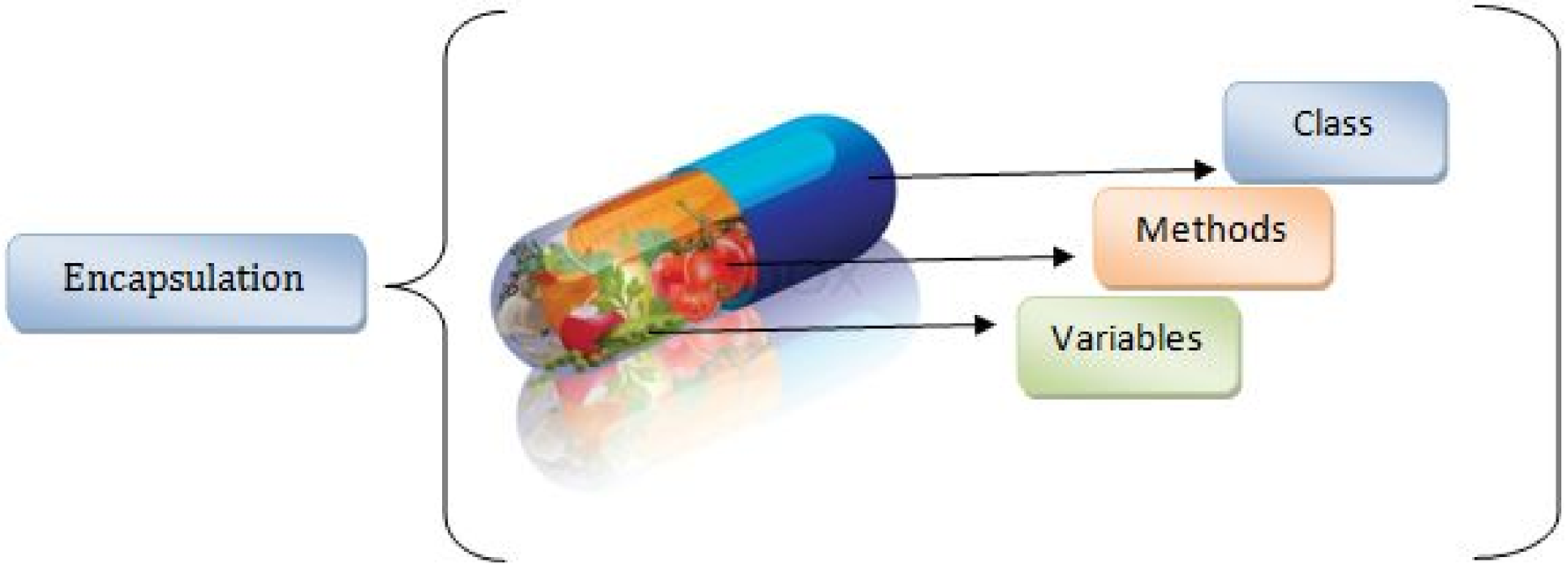
Object-Based Data Model

Definition: The Object-Based Data Model (OBDM) is a data representation framework inspired by Object-Oriented Programming (OOP) principles. It extends traditional database models by incorporating objects, their attributes (data), and behaviors (methods). This model is especially useful for representing complex and real-world entities where relationships and operations need to be part of the data structure.

Key Features of Object-Based Data Model:

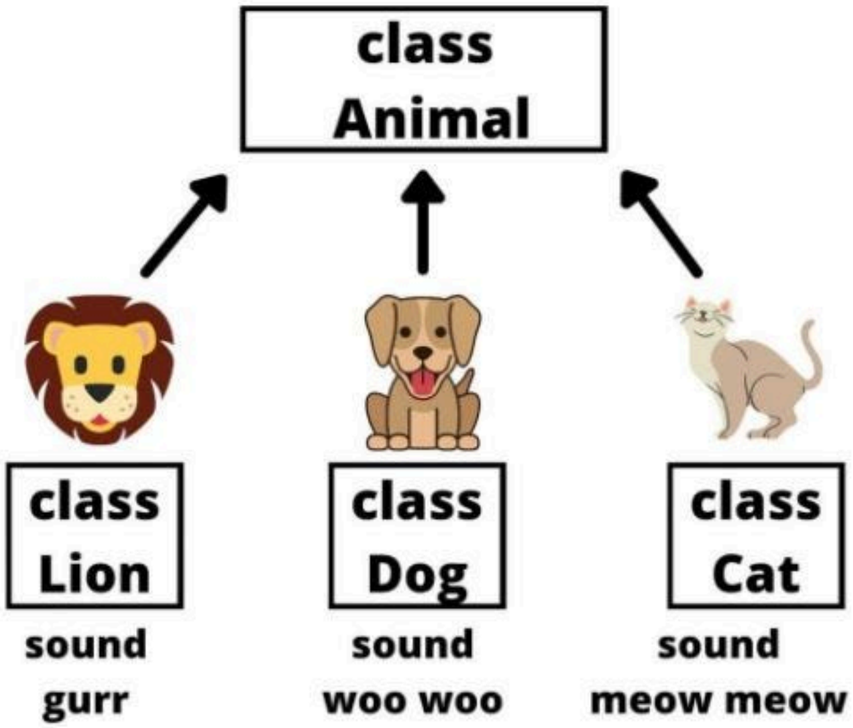
- **Encapsulation:**
  - Data (attributes) and behavior (methods) are encapsulated together in objects.
  - **Example**: A Car object has attributes like brand, model, year, and methods like start() or accelerate().

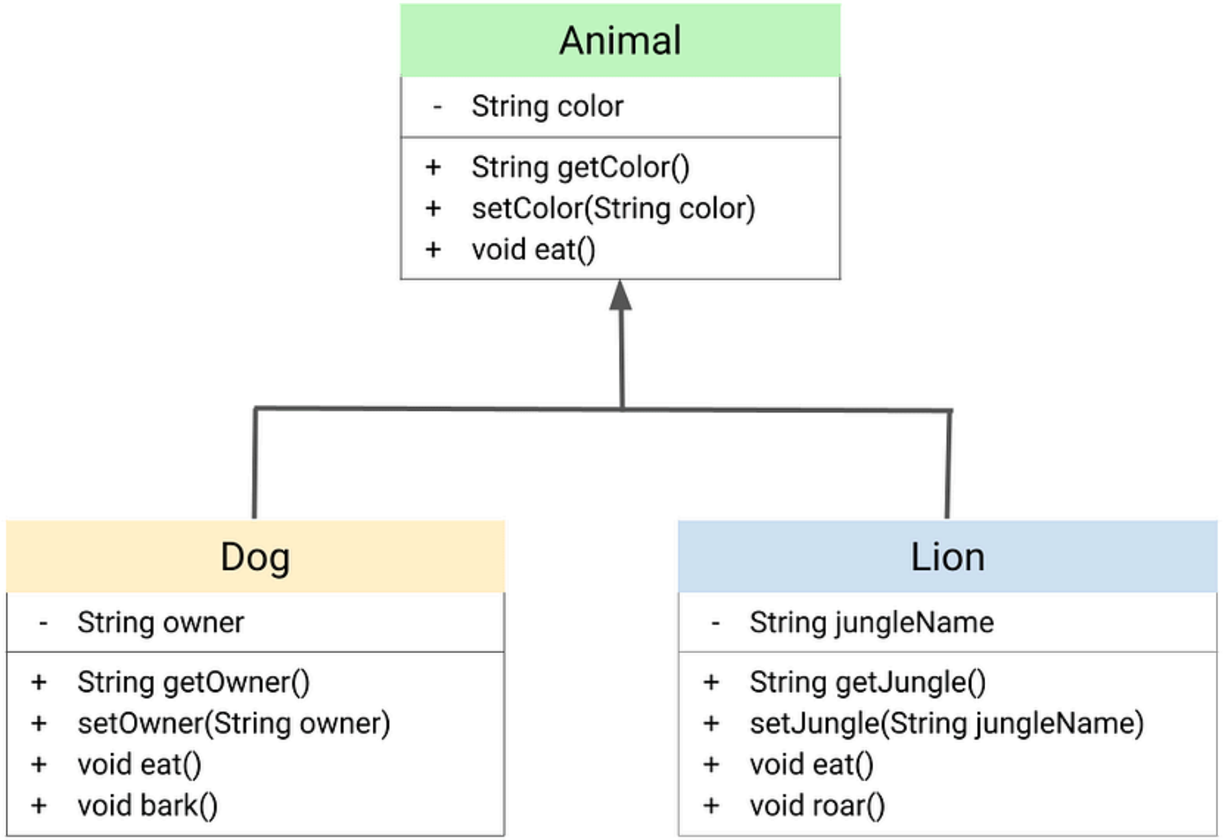
Object-Based Data Model



Object-Based Data Model

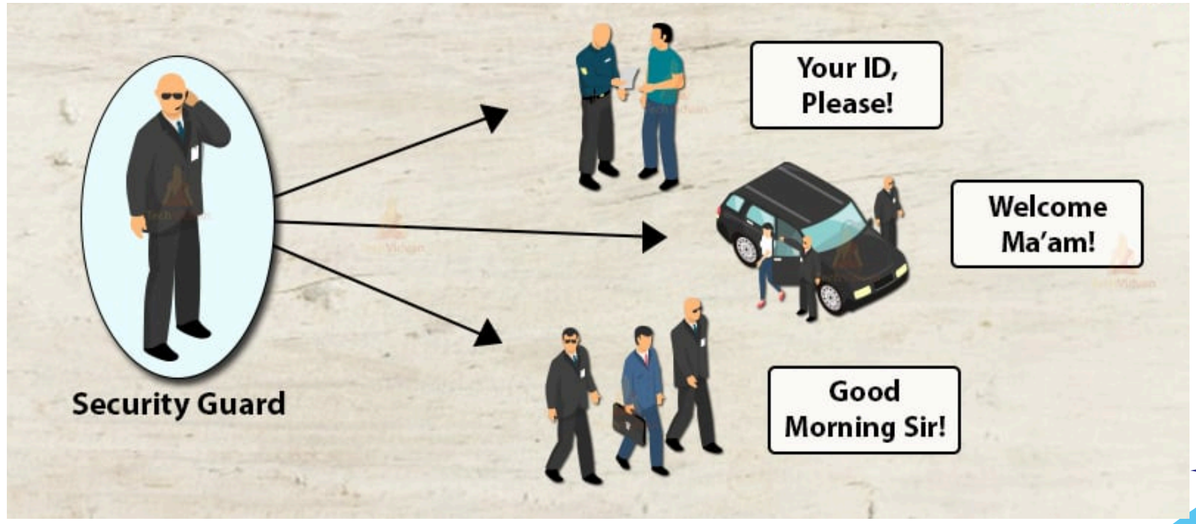
- **Inheritance:**
  - Objects can inherit attributes and behaviors from other objects (classes).
  - Example: A SportsCar class inherits from a Car class but may add unique attributes like topSpeed.

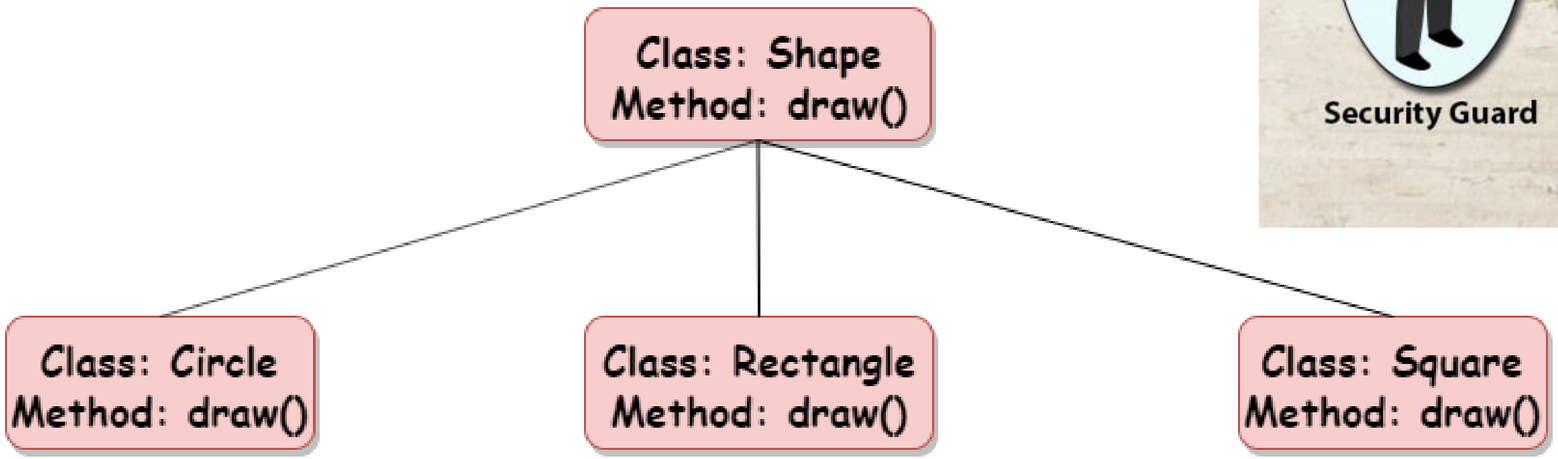




Object-Based Data Model

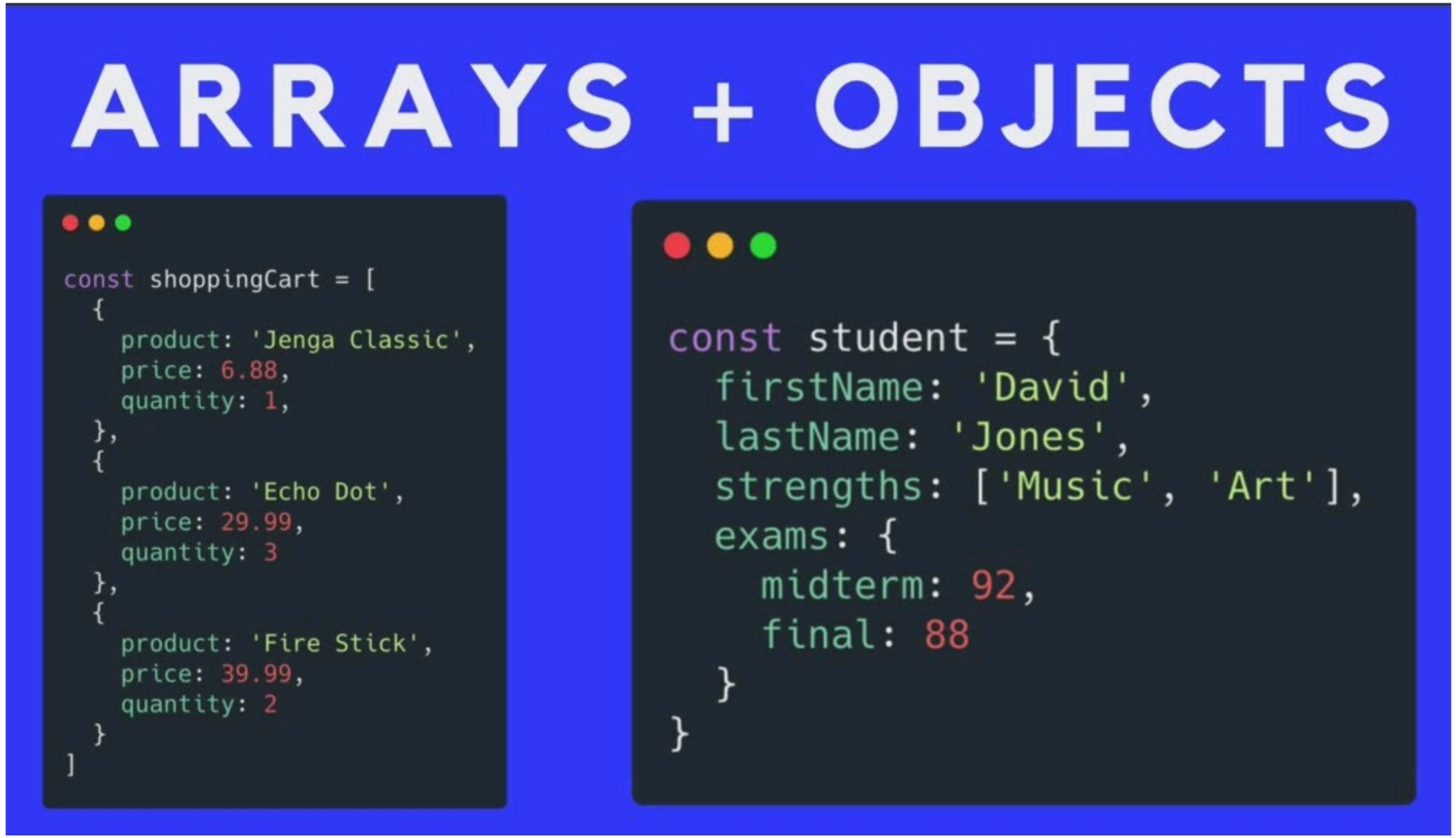
- **Polymorphism:**
  - The same operation can behave differently for different objects.
  - Example: A draw() method may work differently for Circle and Rectangle objects.





Object-Based Data Model

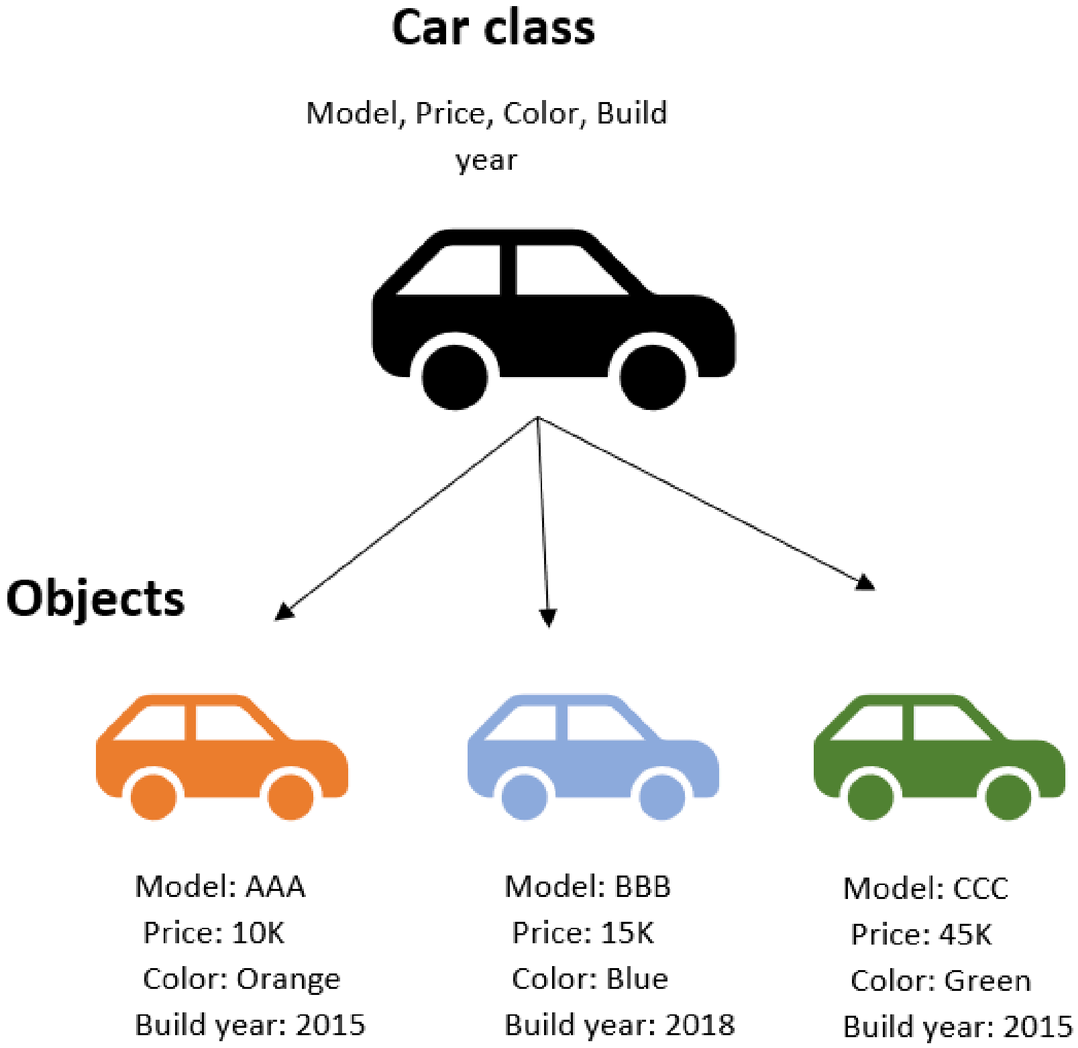
- **Complex Data Types:**
  - Supports arrays, lists, and nested objects, enabling detailed representations.
  - Example:



Components of Object-Based Data Model

- **Objects:**
  - Basic building blocks representing real-world entities.
  - Example: A Person object with attributes like Name, Age, and methods like walk().
- **Classes:**
  - Blueprints for objects, defining their structure and behavior.
  - Example: A Student class with attributes like RollNo and Subjects.
- **Attributes:**
  - Characteristics or properties of an object.
  - Example: A Car object with attributes: Color, Model, Year.
- **Methods:**
  - Functions defining the behavior of an object.
  - Example: A Deposit() method for a BankAccount object.

Components of Object-Based Data Model



Components of Object-Based Data Model

Attributes

>>> Door
>>> Seats
>>> Licence plate
>>> Wheel
>>> Side mirror
>>> Handle
>>> Make
>>> Color



Attributes and Methods of a Car

Methods

>>>headLightOn
>>>Start
>>>Brake
>>>Horn
>>>turnOnAC
>>>startWiper
>>>muffler
>>>exhaustFumes

Components of Object-Based Data Model

CLASS
Human



OBJECTS
Name

PROPERTIES
Email
Address

METHODS
Verify
Send mail

Real-World Example: Library System

Objects:

1. **Library**: Represents the library itself.
    - Attributes: Name, Location
    - Methods: addBook(), issueBook()
2. **Book**: Represents a book in the library.
    - Attributes: Title, Author, ISBN, Status
    - Methods: borrow(), return()
3. **Member**: Represents a library member.
    - Attributes: Name, MembershipID, BooksIssued
    - Methods: register(), borrowBook()

E-R Diagram Concepts

Definition:

A graphical representation of entities, their attributes, and the relationships between them.

Widely used during the database design phase.

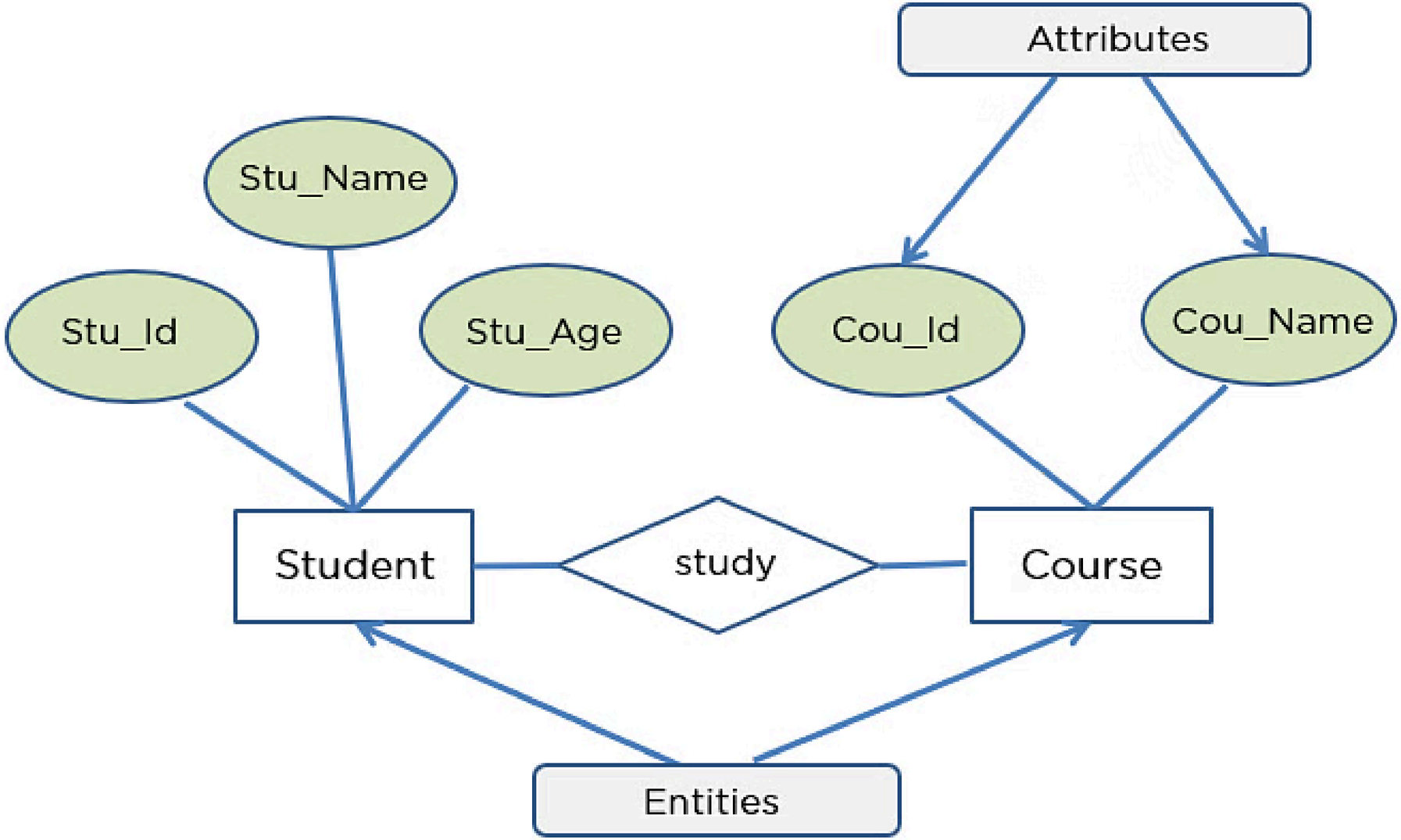
Components of an E-R Diagram:

- **Entities**: Represent objects (e.g., Student, Course).
- **Attributes**: Properties of entities (e.g., Name, RollNo).
- **Relationships**: Links between entities (e.g., Enrolled In).

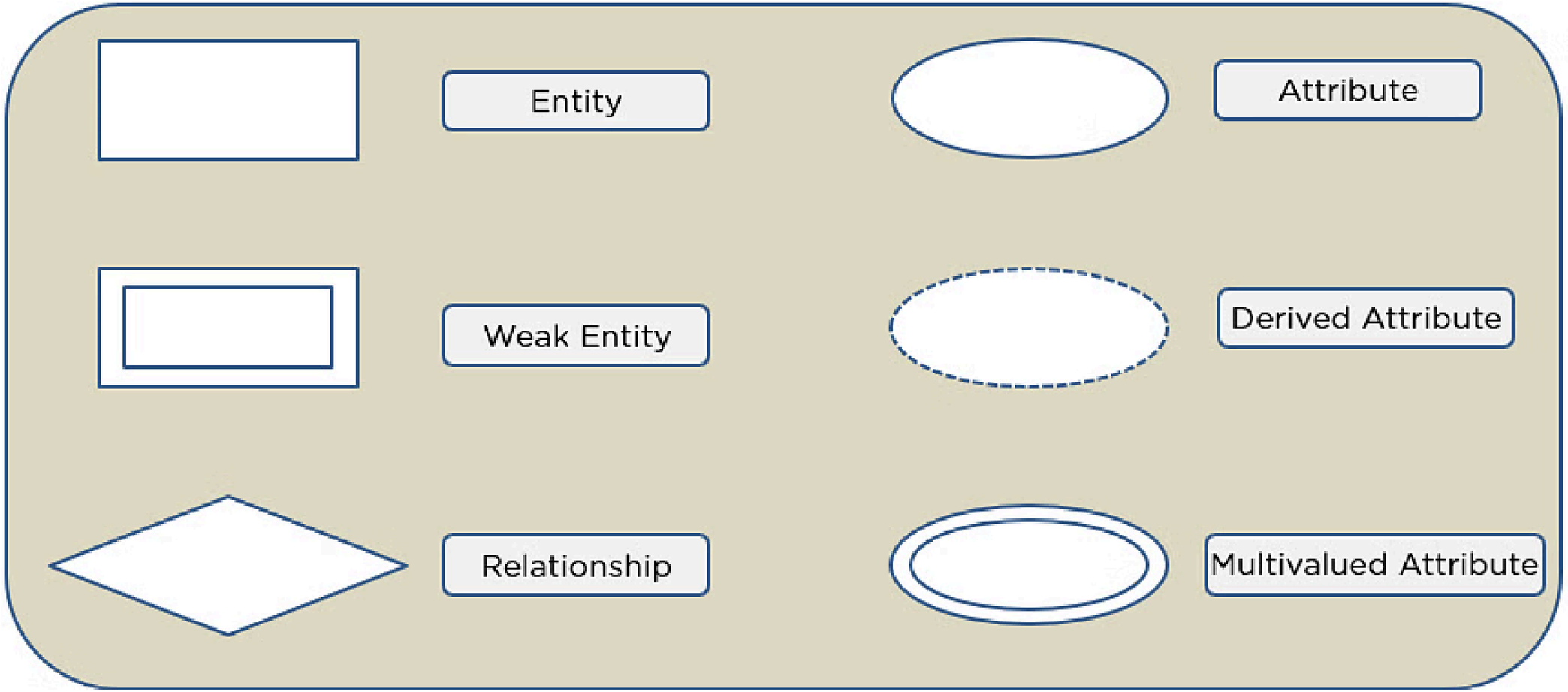
Example:

- A Student entity is related to a Course entity via an Enrollment relationship.

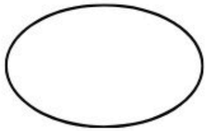
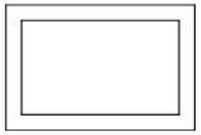
E-R Diagram Concepts



Symbols Used in ER Diagrams



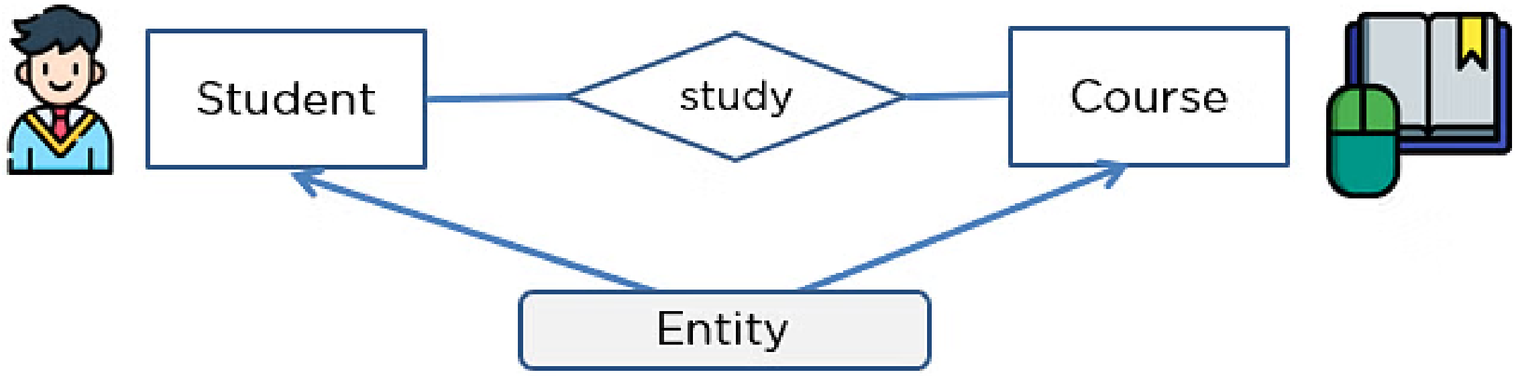
Symbols Used in ER Diagrams

Figures	*Symbol*	*Representation*
Rectangle		Entities in the ER Model
Ellipse		Attributes in the ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-valued Attributes
Double Rectangle		Weak Entity

Entities and Attributes

Entities

An entity represents a distinct "thing" or "object" in the real world. These objects could be anything you want to track or store data about in a database. Entities are typically the subjects of the database and can represent people, places, events, or concepts.

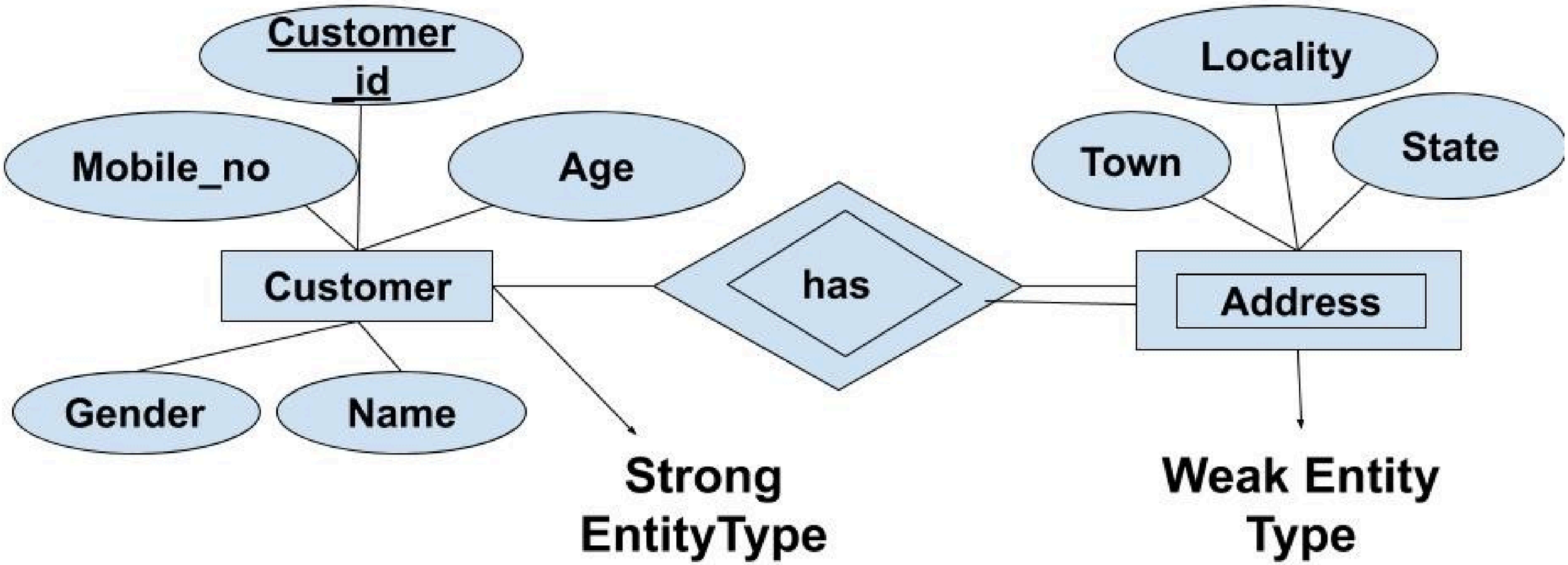


Entities and Attributes

Types of Entities:

1. **Strong Entities:**
   - These entities have their primary key to uniquely identify each instance.
   - They exist independently (doesn't rely on other entities).
   - Example: In a university database, a Student entity is strong because it has a unique StudentID.
2. **Weak Entities:**
   - These entities cannot exist independently and rely on a related strong entity for identification.
   - They use a foreign key in conjunction with their attributes to form a unique identifier.
   - Example: In an order processing system, an OrderItem is weak because it depends on the Order entity.

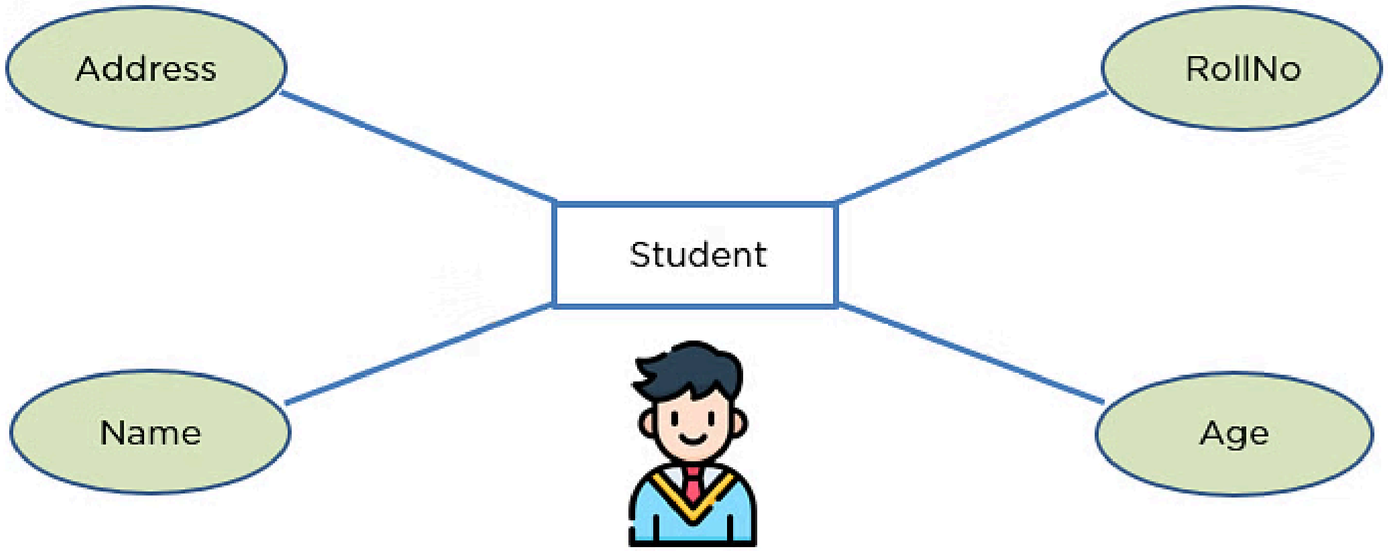
Entities and Attributes



Entities and Attributes

Attributes

An attribute represents a characteristic or property of an entity. Attributes define the properties that describe an entity in detail. Each attribute corresponds to a column in a relational database.

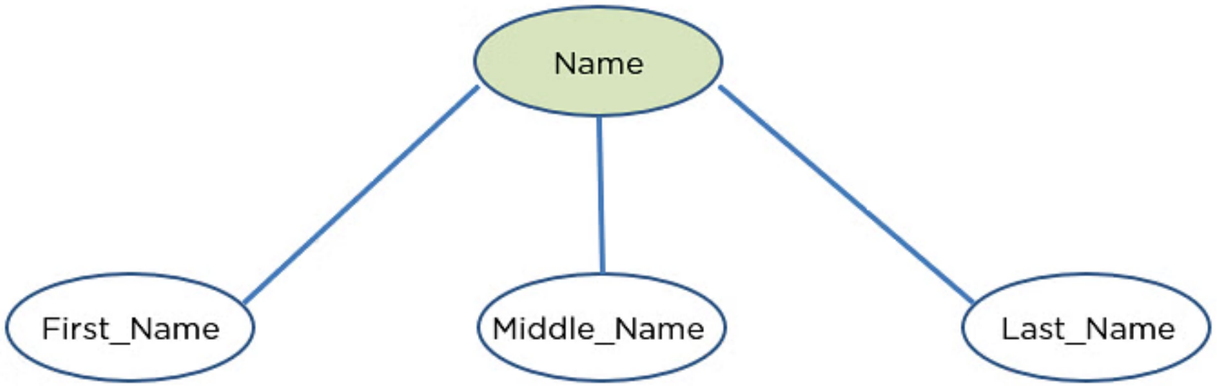


Entities and Attributes

Types of Attributes:

- **Simple Attributes:**
  - Cannot be broken down into smaller parts.
  - Example: Name, Age, Gender.

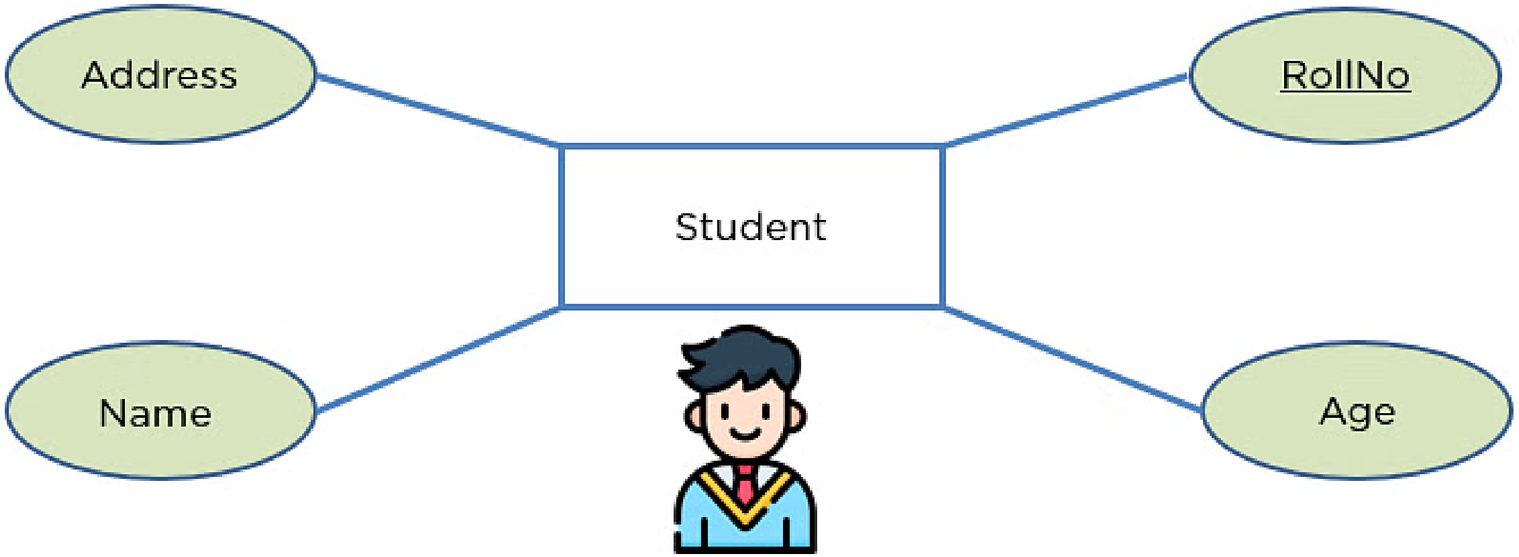
- **Composite Attributes:**
  - Can be subdivided into smaller parts.
  - Example: FullName can be broken into FirstName and LastName.



Entities and Attributes

Key Attributes:

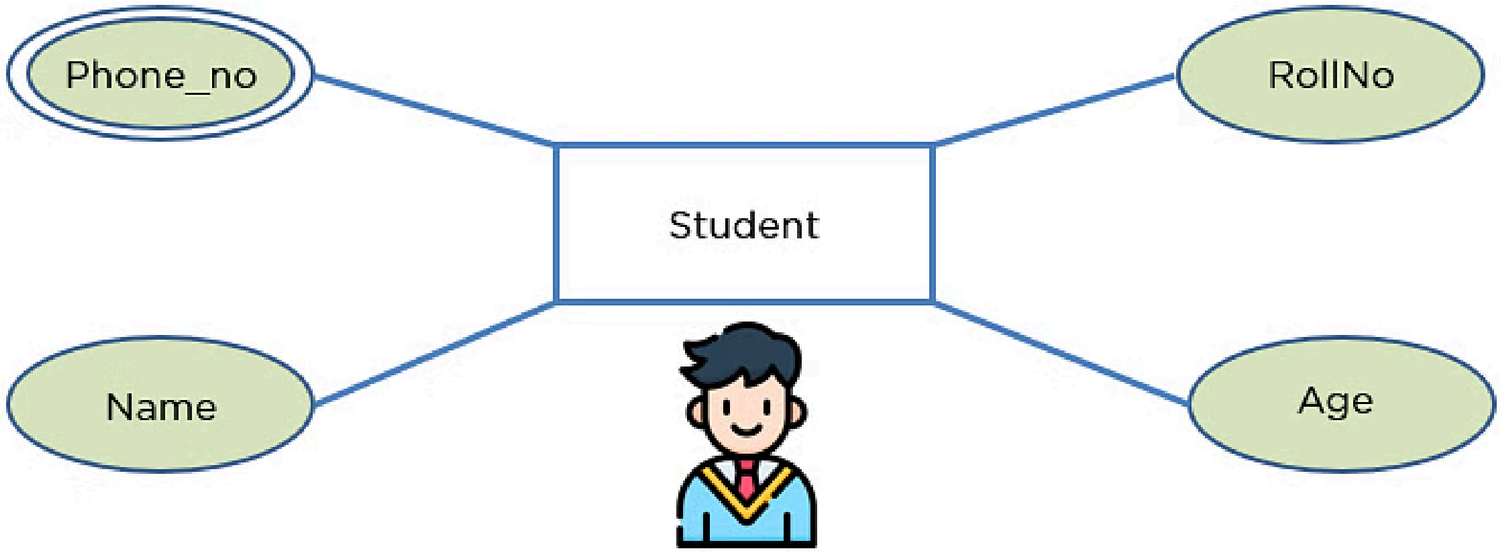
- A unique identifier for an entity instance (like a primary key).
- Example: RollNo for a Student entity.



Entities and Attributes

Multivalued Attributes:

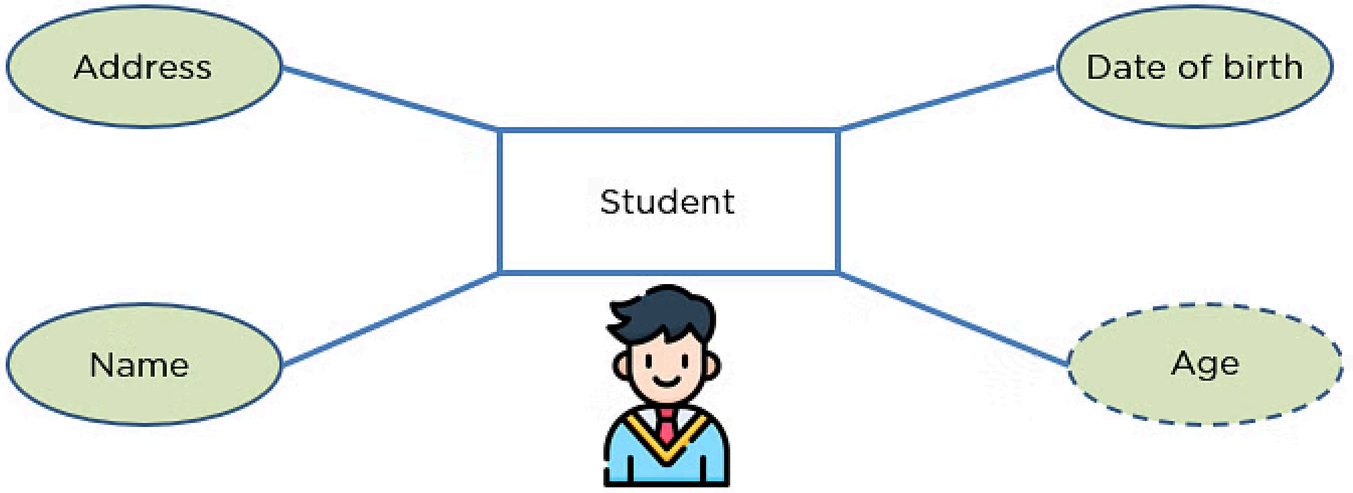
- Can have multiple values for a single entity.
- Example: A Student might have multiple PhoneNumbers.



Entities and Attributes

Derived Attributes:

- Attributes that are not stored directly but are derived from other attributes.
- Example: Age can be derived from DateOfBirth.



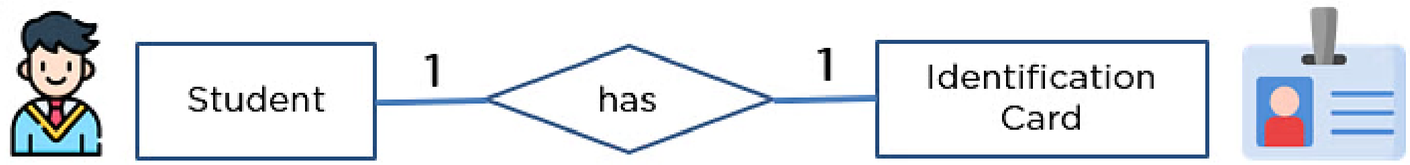
Types of Relationships

One-to-One (1:1) Relationship

A one-to-one relationship occurs when a single instance of an entity is associated with exactly one instance of another entity.

Example:

- **Entities:**
  - Person (with attributes: PersonID, Name)
  - Passport (with attributes: PassportID, IssueDate)
- **Relationship:**
  - A person can have only one passport, and each passport belongs to one person.



Types of Relationships

One-to-Many (1:N) Relationship

A one-to-many relationship occurs when one instance of an entity is associated with multiple instances of another entity.

Example:

- **Entities:**
  - Customer (with attributes: CustomerID, CustomerName)
  - Order (with attributes: OrderID, CustomerID)
- **Relationship:**
  - One customer can place many order and many order may belong to one customer.



Types of Relationships

Many-to-Many (M:M) Relationship

A many-to-many relationship occurs when multiple instances of one entity are associated with many instances of another entity.

Example:

- **Entities**:
  - Employee (with attributes: empId, Name)
  - Project (with attributes: pid, Name)
- **Relationship:**
  - you can assign an employee to many projects and a project can have many employees.



Aggregation

Definition

Aggregation is a higher-level abstraction used in the ER model when we need to model a relationship between a relationship and an entity. It treats a relationship as an entity and allows it to participate in another relationship.

When is Aggregation Used?

- When a relationship itself has attributes.
- When a relationship needs to participate in another relationship.

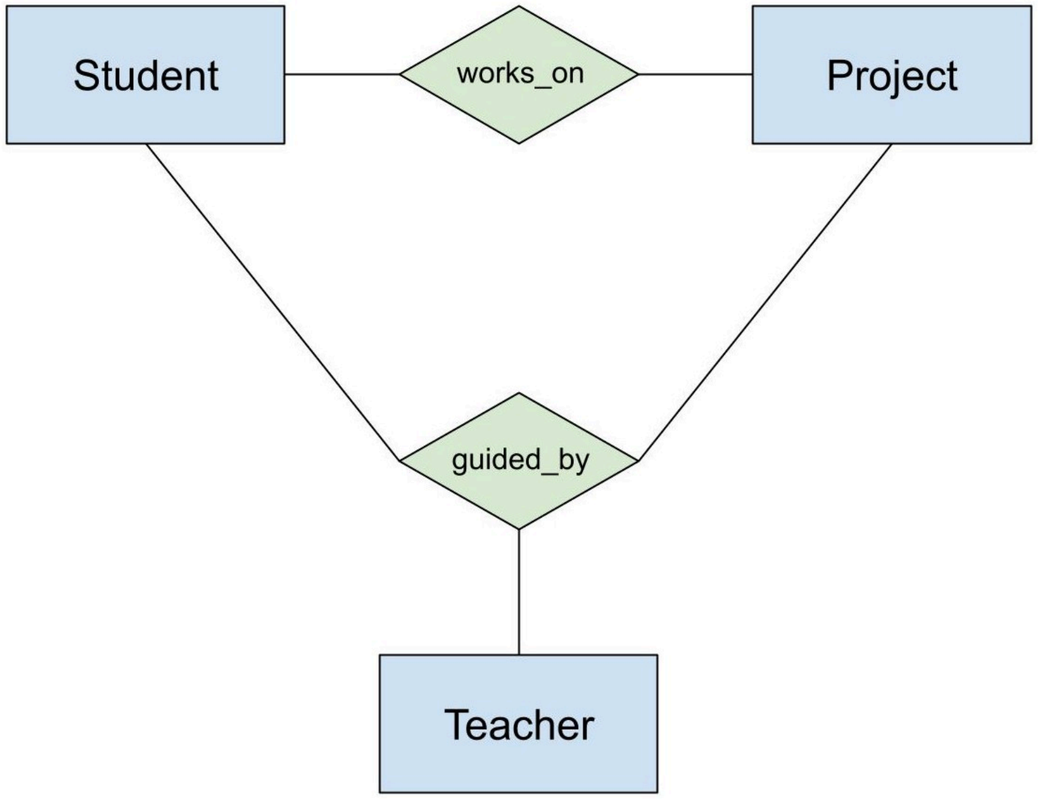
Aggregation

Example:
Consider a university database:

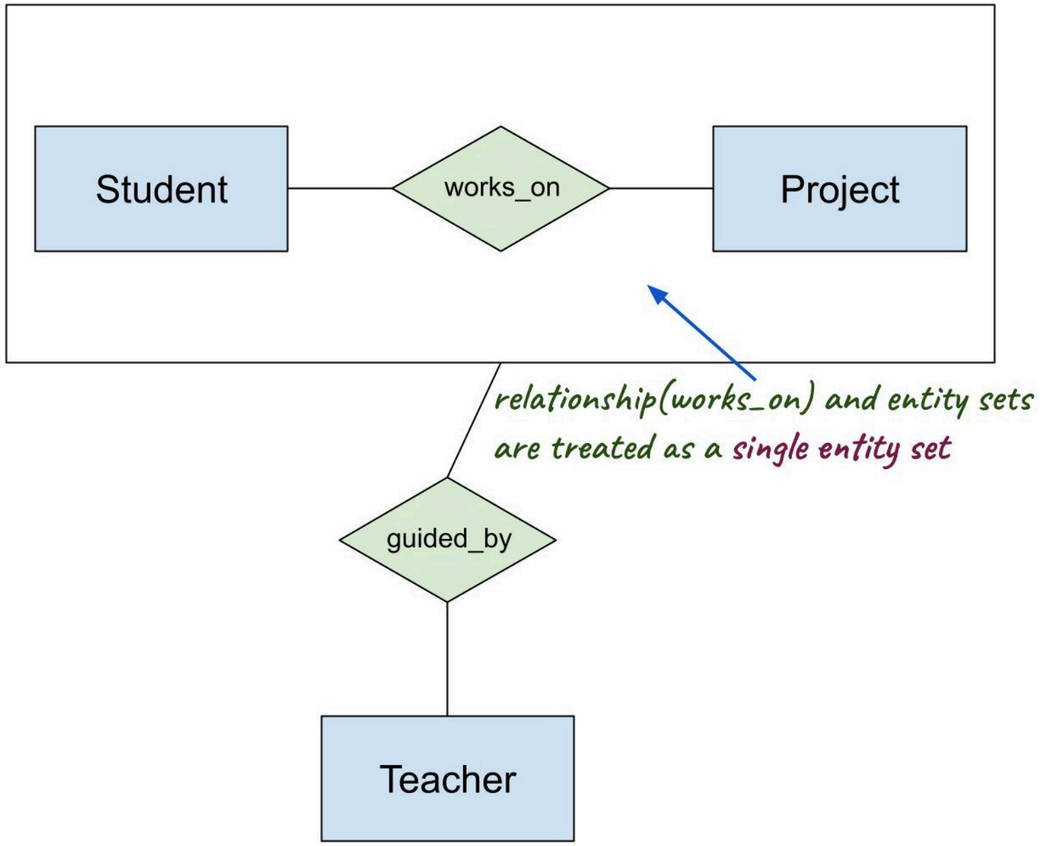
- **Entities:**
  - Student (attributes: StudentID, Name)
  - Course (attributes: CourseID, Title)
  - Professor (attributes: ProfessorID, Name)
- **Relationships:**
  - Teaches between Professor and Course.
  - Enrolled between Student and Course.

Now, let’s say a Project requires both the Student and the Course relationship. Aggregation allows us to treat Enrolled as an entity that relates to Project.

Aggregation



without aggregation



with aggregation

Generalization

Definition

Generalization is a bottom-up approach in the ER model. It combines two or more lower-level entities into a single higher-level entity by identifying shared characteristics.

Key Characteristics of Generalization

- **Focus**: Reduce redundancy by generalizing common features.
- **Representation**: The higher-level entity is connected to lower-level entities with a triangle.

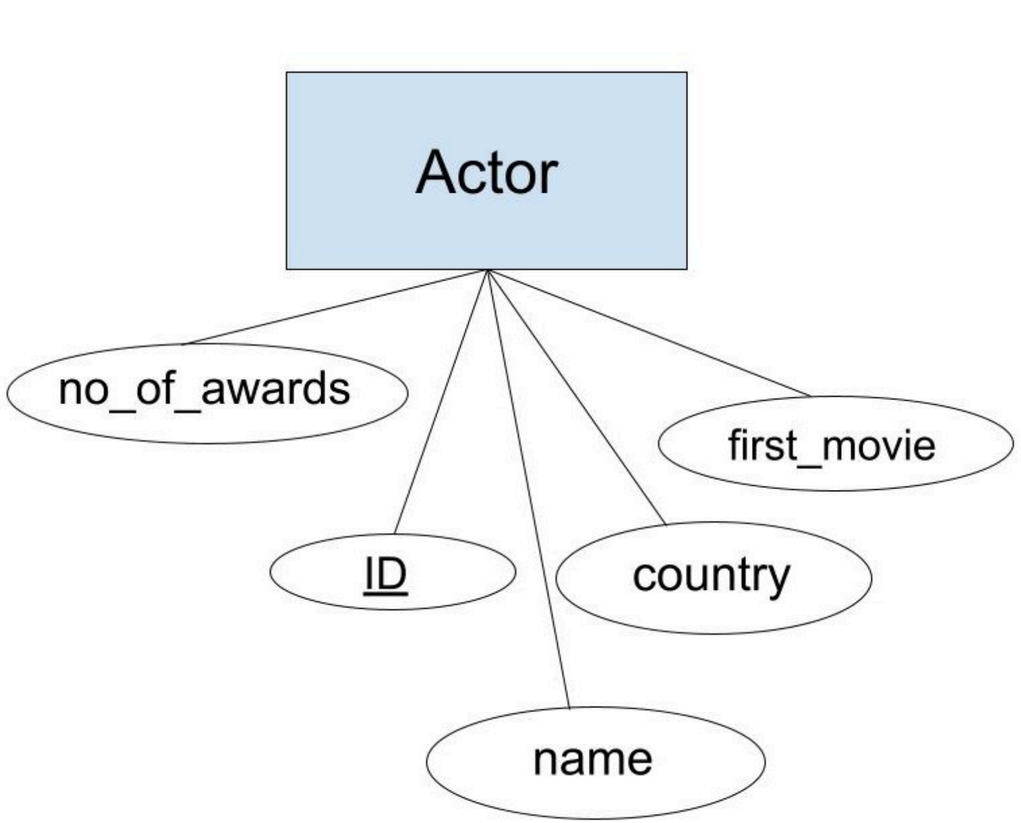
Generalization

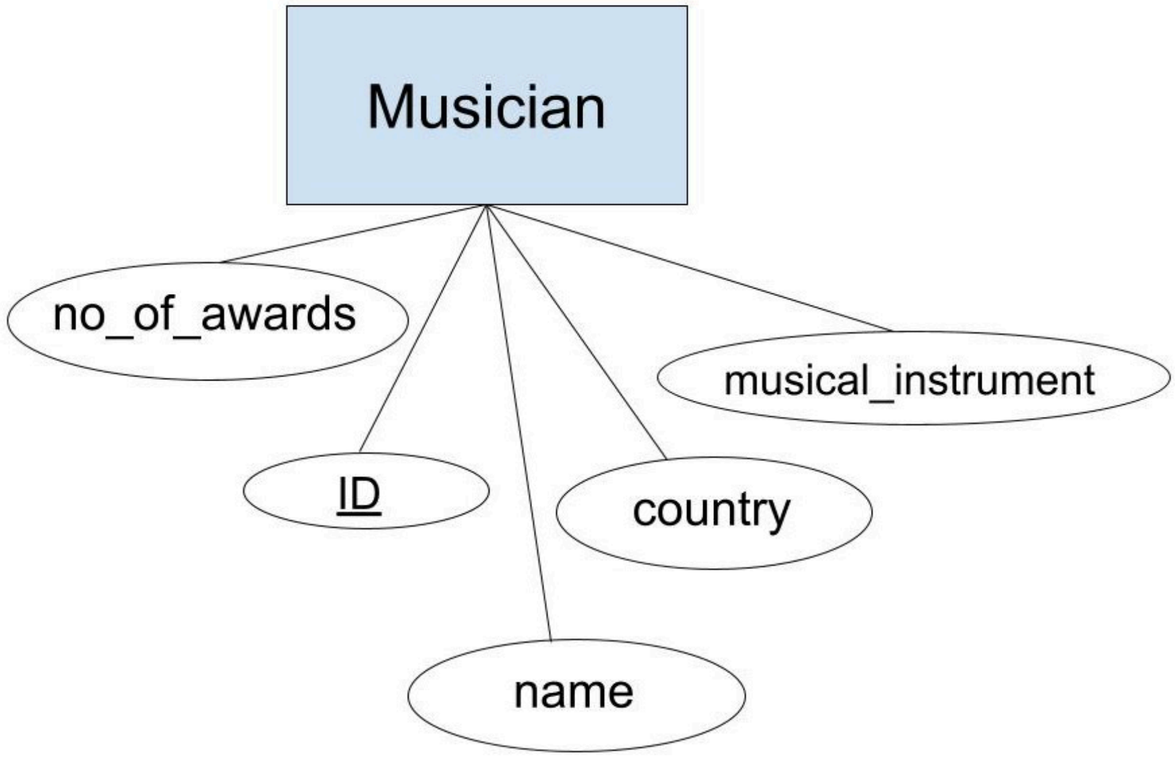
Example:
In an employee database:

- **Entities:**
  - Actor (attributes: ActorID, Role, no_of_movies)
  - Musician (attributes: MusicianID, Role, no_of_albums)
- **Generalized Entity:**
  - Person (common attributes: Name, Country, Phone)

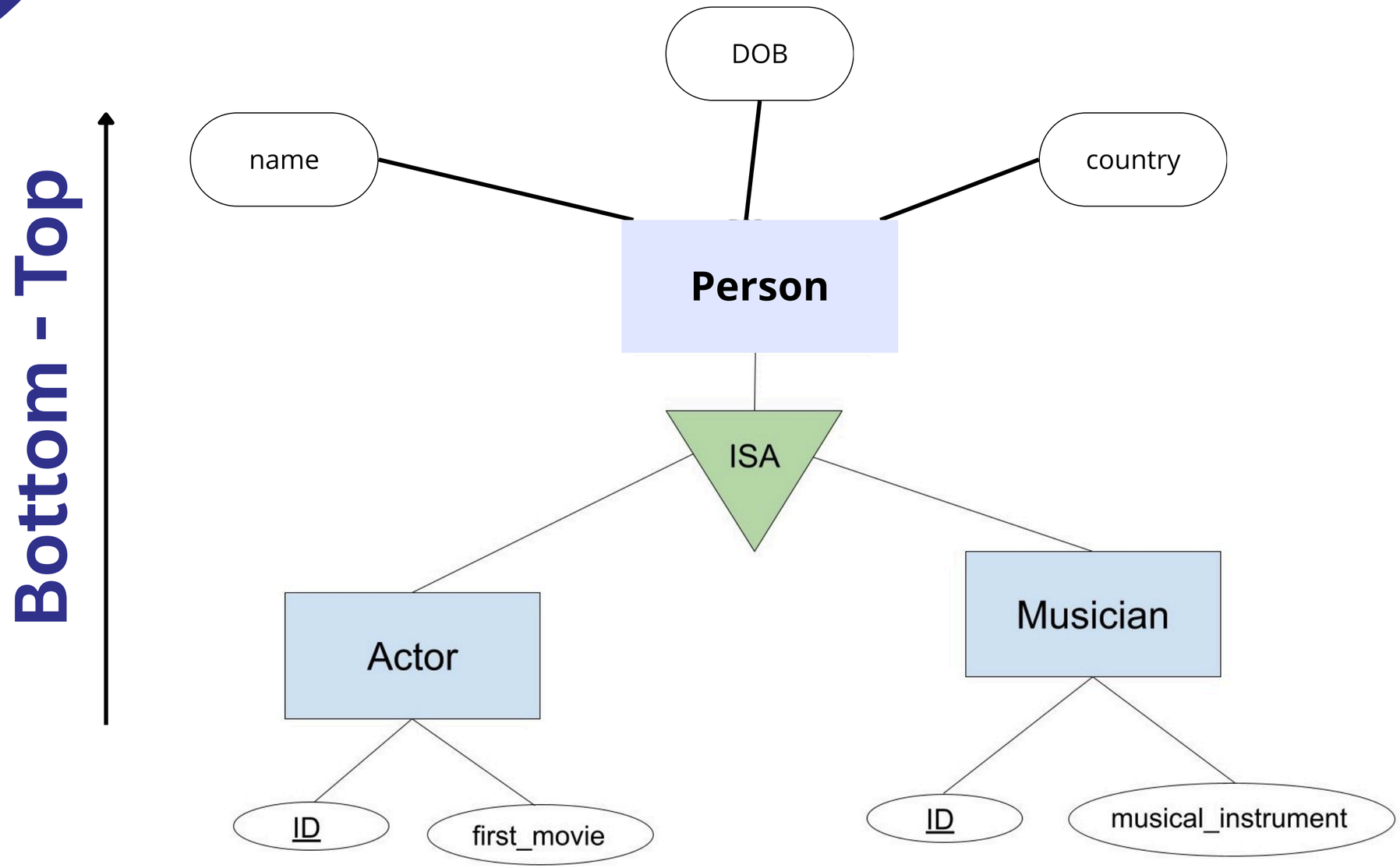
Both Actor and Musician are subclasses of the Person superclass. Shared attributes like Name and Country are generalized in Person.

Without Generalization





With Generalization



Converting ER Diagrams to Tables

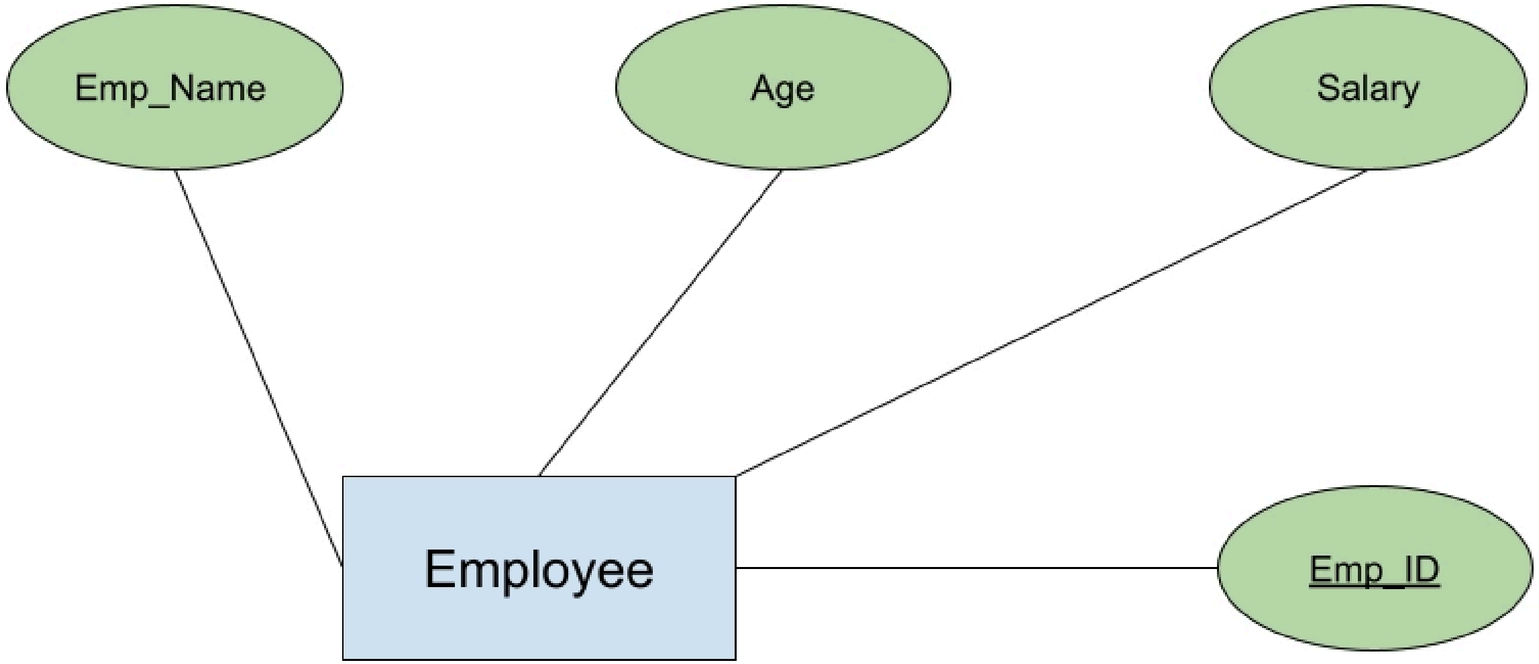
Steps:

1. **Entities → Tables**
    - Create a table for each entity.
    - Attributes become columns.
2. **Relationships → Foreign Keys**
    - One-to-Many: Add the primary key of one table as a foreign key in the related table.
    - Many-to-Many: Create a junction table.
    -

Example:

- **Entities:**
    - Student(Name, RollNo, Address)
    - Course(CourseID, Title)
- **Relationship:**
    - EnrolledIn(StudentID, CourseID)

Converting ER Diagrams to Tables



Emp_ID	Emp_Name	Age	Salary

Thank You

UNIT - 3

Relational Database Model

Table of Contents

- Structure of RDBMS and Terminology
- Database Schema and Schema Diagram
- Keys: Super, Candidates, Primary, Foreign, Composite etc. and Relationships
- Introduction to Relational Algebra
- Relational Algebra Operations: Select, Project, Cartesian Product, Union, Set Difference, Natural Join, Outer Join.

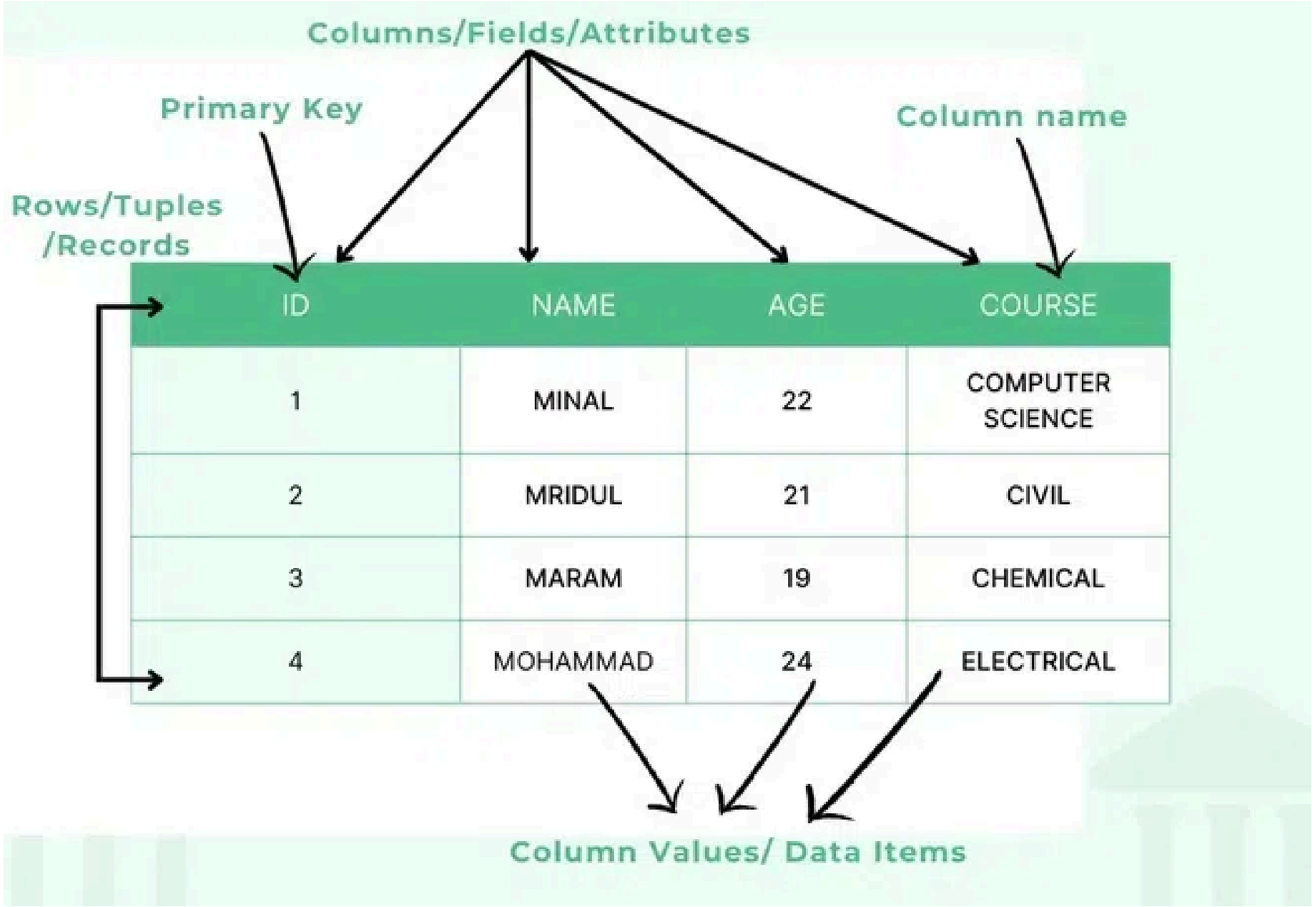
What is RDBMS?

Definition:

A Relational Database Management System (RDBMS) stores data in tables (rows + columns) and keeps relationships between those tables using keys.

It gives you a reliable way to store, query, update, and protect data while enforcing rules (constraints), fast lookups (indexes), and consistent multi-step changes (transactions).

What is RDBMS?



Structure of RDBMS

Main components:

- **SQL Interface (Parser/Translator):** Accepts SQL, checks syntax/semantics, turns it into an internal plan.
- **Query Optimizer:** Chooses the fastest way to run your query (e.g., use index vs. full scan).
- **Execution Engine**: Runs the chosen plan—joins tables, filters rows, computes aggregates.
- **Storage Engine:** Reads/writes pages on disk/SSD; manages tables, indexes, and files.
- **Buffer/Cache Manager:** Keeps hot pages in memory to avoid slow disk reads.

Structure of RDBMS

Main components:

- **Transaction Manager**: Groups operations into transactions with ACID guarantees.
  - **Atomicity**: all-or-nothing
  - **Consistency**: rules/constraints never broken
  - **Isolation**: concurrent work doesn’t clash (isolation levels)
  - **Durability**: once committed, changes survive crashes
- **Concurrency Control**: Locks to let many users work safely at once.

Structure of RDBMS

Main components:

- **Recovery Manager**: Uses write-ahead logs, checkpoints to recover after failures.
- **Catalog/Metadata Manager**: Stores definitions of tables, columns, indexes, constraints—your database's "map."
- **Security/Authorization**: Users, roles, privileges (who can see/change what).
- **Utilities**: Backup/restore, import/export, statistics collection, maintenance.

Key Terminologies

Term	Definition (Technical but simple)	Example (University DB)
Relation (Table)	Collection of tuples (rows) with attributes (columns).	`Students` table.
Tuple (Row/Record)	A single entry (record) in a relation.	`(1001, Dipesh, Tamang, dipesh.t@example.com)`
Attribute (Column/Field)	A property/characteristic of entity.	`Email` in `Students`.
Constraints	Rules applied to ensure data validity.	`NOT NULL`, `UNIQUE`, `CHECK(Age>15)`.

Key Terminologies

Null	Special marker for “unknown / not applicable”.	`Grade` is NULL until exam complete.
Index	Data structure to speed lookups.	Index on `Email` for faster search.
View	Virtual table from a saved query.	`ActiveStudents` view.
Normalization	Splitting tables to reduce redundancy & anomalies.	Store `DeptName` in `Departments`, not in every `Student`.
Schema	Blueprint of DB (tables, attributes, relationships).	University DB schema.
Instance (Database State)	Actual data in DB at a specific time.	`Students` table has 1200 rows today.
Entity Integrity	PK must be unique & not NULL.	No two students with same `StudentID`.

Database Schema

Definition:

A Database Schema is the blueprint or structure of a database.

It defines how the data is organized, including:

- Tables (relations)
- Columns (attributes) and their data types
- Keys (Primary Key, Foreign Key, Candidate Key, etc.)
- Relationships between tables
- Constraints (NOT NULL, UNIQUE, CHECK, etc.)

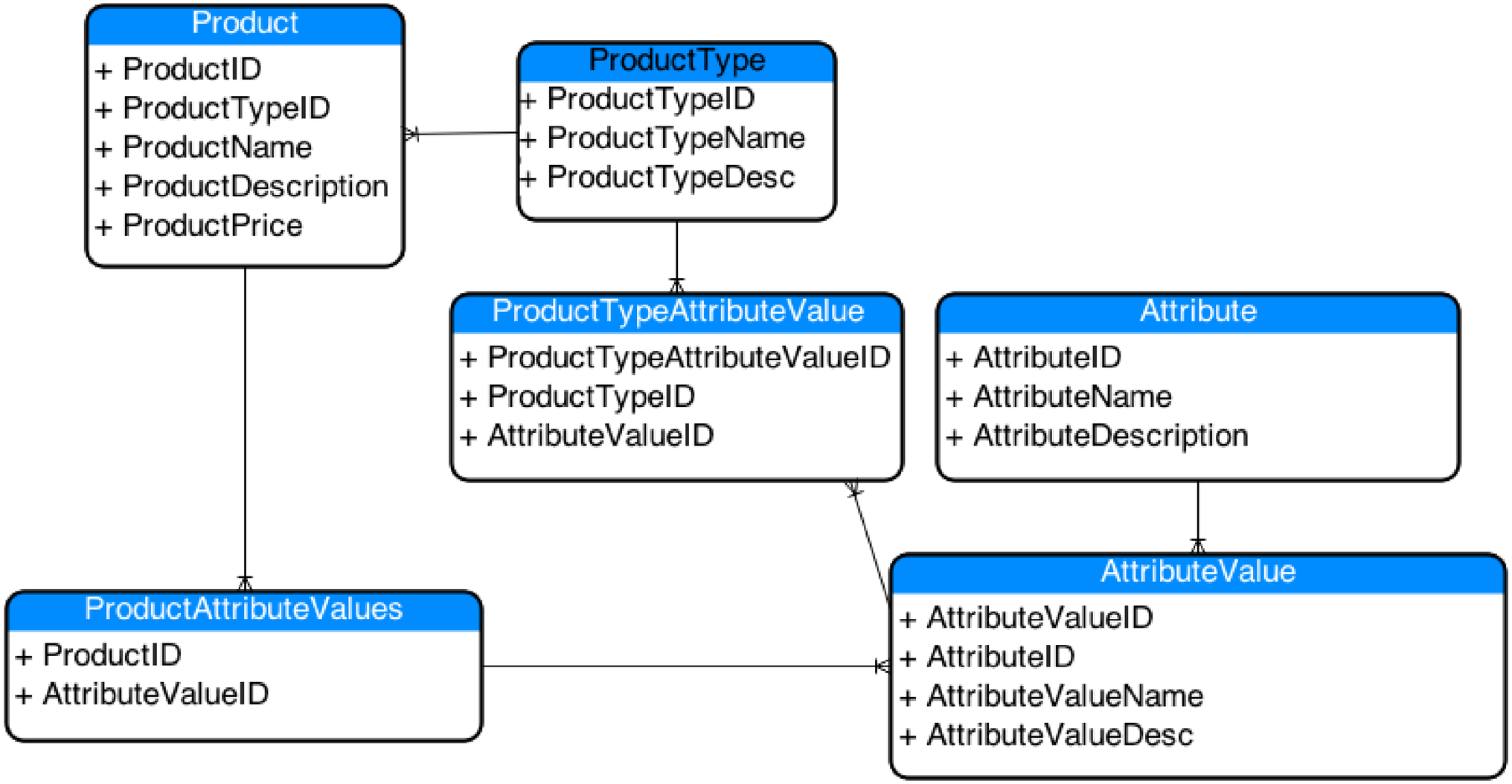
Types of Database Schemas

- **Physical Schema** – Describes how and where the data is actually stored in the system (files, blocks, indexes). This is handled by the database internals and usually hidden from users.

- **Logical Schema** – Describes what data is stored and how it is logically organized (tables, columns, relationships, constraints). It’s the overall design of the database from a developer’s perspective.

- **View/External Schema** – Describes the specific part of the data a particular user or application can access. It can hide certain fields or records for security or simplicity.

Example of Database Schemas



Keys in RDBMS

Definition of Key:

- A key is a column (or combination of columns) in a database table that helps to uniquely identify each row and/or enforce relationships between tables.

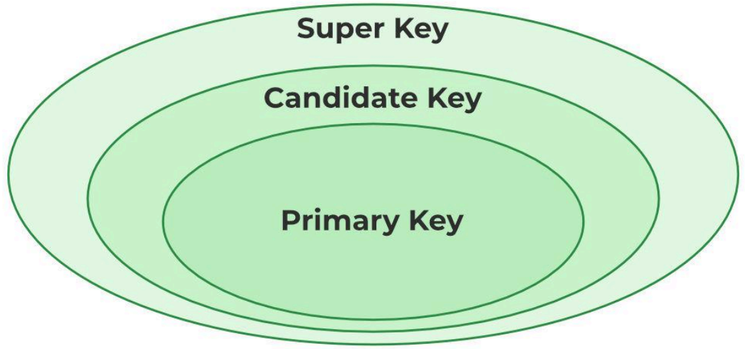
Main Points:

- Keys ensure uniqueness of records.
- Keys help link tables through relationships (foreign keys).
- Keys are essential for data integrity and normalization.

Types of Keys in DBMS:

- **Super Key** – Any set of attributes that uniquely identifies a row.
- **Candidate Key** – Minimal super key (no extra attributes).
- **Primary Key (PK)** – Candidate key chosen to uniquely identify rows.
- **Alternate Key** – Candidate key not chosen as PK.
- **Composite Key** – Key made of multiple columns to uniquely identify a row.
- **Foreign Key (FK)** – Attribute that references another table's PK.
- **Surrogate Key** – System-generated key, usually numeric or auto-increment.

Keys in DBMS



Candidate Key

Foreign Key

Table 1

EmpID	Emp Name	EmpLicence	EmpPassport	DId
001	Tom	EL101	PA123	2
005	John	EL102	PA125	3
008	Alice	EL103	PA129	5

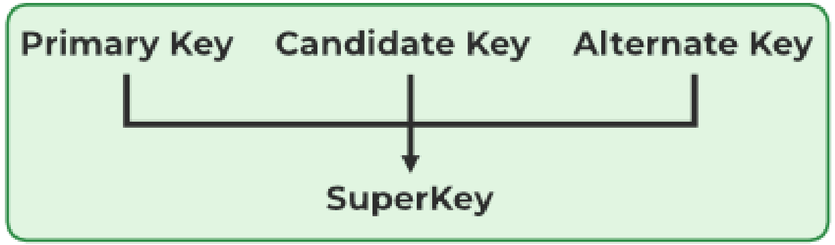
Primary Key

Alternate Key

Unique Key

Table 2

DId	Designation
2	BPO
3	Account
5	IT



Super Key

Definition:

- A super key is any set of attributes that can uniquely identify a row in a table.
- It may contain extra attributes not necessary for uniqueness.
- **Example**:
  - In Students(StudentID, Email, FirstName, LastName), both:
    - {StudentID} → unique
    - {StudentID, Email} → also unique (extra column)
- **Analogy**: Like using passport + phone number together to identify someone, even though passport alone is enough.

Candidate Key

Definition:

- A candidate key is a minimal super key (no extra attributes) that can uniquely identify a row.
- There can be more than one candidate key in a table.
- **Example**:
  - In Students, both {StudentID} and {Email} can uniquely identify a student.
- **Analogy**: Like choosing either passport number or national ID as a unique identifier.

Primary Key (PK)

Definition:

- The primary key is the candidate key chosen to uniquely identify rows in a table.
  - Must be unique and NOT NULL.
- **Example**:
  - StudentID is chosen as PK in Students.
- **Analogy**: Like a roll number in a school — one unique ID assigned to each student.

Alternate Key

Definition:

- A candidate key not chosen as the primary key.
- **Example**:
  - Email in Students → alternate key because StudentID is PK.
- **Analogy**: Backup ID — like a national ID if passport is the primary identifier.

Composite Key

Definition:

- A key made of multiple columns to uniquely identify a row.
- **Example**:
  - In Enrollments(StudentID, CourseID, Semester), the combination (StudentID, CourseID, Semester) uniquely identifies each enrollment.
- **Analogy**: Like first name + last name + birthdate together to identify someone uniquely in a large population.

Foreign Key (FK)

Definition:

- A foreign key is an attribute (or set of attributes) in one table that references the primary key of another table.
  - Ensures referential integrity.
- **Example**:
  - DeptID in Students → references Departments(DeptID).
- **Analogy**: Like a department ID card that points to which department a student belongs.

Surrogate Key

Definition:

- A system-generated key used as a primary key, usually numeric or auto-increment.
- **Example**:
  - EnrollmentID = E0001, E0002, ... in Enrollments.
- **Analogy**: Like a ticket number given automatically at a counter.

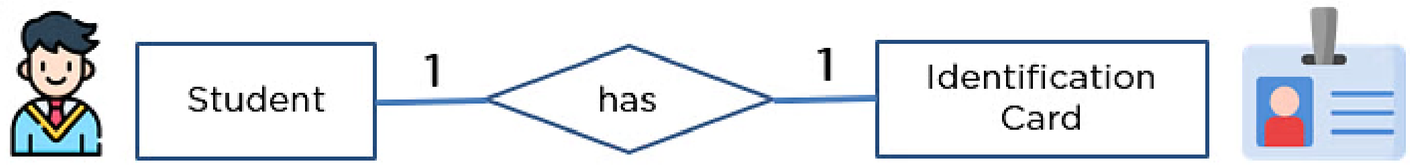
Types of Relationships

One-to-One (1:1) Relationship

A one-to-one relationship occurs when a single instance of an entity is associated with exactly one instance of another entity.

Example:

- **Entities:**
  - Person (with attributes: PersonID, Name)
  - Passport (with attributes: PassportID, IssueDate)
- **Relationship:**
  - A person can have only one passport, and each passport belongs to one person.



Types of Relationships

One-to-Many (1:N) Relationship

A one-to-many relationship occurs when one instance of an entity is associated with multiple instances of another entity.

Example:

- **Entities:**
  - Customer (with attributes: CustomerID, CustomerName)
  - Order (with attributes: OrderID, CustomerID)
- **Relationship:**
  - One customer can place many order and many order may belong to one customer.



Types of Relationships

Many-to-Many (M:M) Relationship

A many-to-many relationship occurs when multiple instances of one entity are associated with many instances of another entity.

Example:

- **Entities**:
  - Employee (with attributes: empId, Name)
  - Project (with attributes: pid, Name)
- **Relationship:**
  - you can assign an employee to many projects and a project can have many employees.



Introduction to Relational Algebra

Definition

Relational Algebra is the theoretical foundation of relational databases.
It’s a set of operations that take one or more relations (tables) as input and produce a new relation as output.

Purpose:

- Provides a foundation for SQL queries.
- Ensures data retrieval and manipulation are structured and predictable.
- Helps in optimizing queries for better performance.

Analogy:

Relational Algebra is like a toolkit for working with tables. You can filter, combine, or manipulate tables to get the exact data you want. It doesn’t change the original table it only produces a new table as a result.

Relational Algebra

- Relational algebra is a **procedural** query language that works on relational model.
- It tells what data to be retrieved and how to be retrieved.

- **Basic/Fundamental Operations:**
    1. Select (σ)
    2. Project (∏)
    3. Union (∪)
    4. Set Difference (-)
    5. Cartesian product (X)
    6. Rename (ρ)
- **Derived Operations**
    1. Natural Join (⋈)
    2. Left, Right, Full outer join (⟕, ⟖, ⟗)
    3. Intersection (∩)
    4. Division (÷)

SELECT Operator

- Select Operator is denoted by sigma (σ) and it is used to find the tuples (or rows) in a relation (or table) which satisfy the given condition.

student

Sid	Name	City	mobile
1	RAM	Kathmandu	9801169156
2	HARI	Patan	9801169155

- $\sigma_{\text{city="Patan"}}(\text{student})$ output

Sid	Name	City	mobile
2	HARI	Patan	9801169155

Project Operator

- Project operator is denoted by ∏ symbol and it is used to select desired columns (or attributes) from a table (or relation).

STUDENT

Sid	Name	City	mobile
1	RAM	Kathmandu	9801169156
2	HARI	Patan	9801169155

- ∏ name,city (STUDENT)

Name	City
RAM	Kathmandu
HARI	Patan

- ∏ name (STUDENT)

Name
RAM
HARI

Union Operator

- Union operator is denoted by ∪ symbol and it is used to select all the rows (tuples) from two tables (relations).
- Let's say we have two relations R1 and R2 both have same columns and we want to select all the tuples(rows) from these relations then we can apply the union operator on these relations
- The rows (tuples) that are present in both the tables will only appear once in the union set. In short you can say that there are no duplicate
- **Syntax of Union Operator (∪):** table_name1 ∪ table_name2

CUSTOMER

cid	Name	mobile
1	Ram	980****
2	Hari	9841***
3	Sita	980****

BORROWER

bid	cid	Amount
1	1	10000
2	2	20000

DEPOSITOR

did	cid	Amount
1	1	20000
2	3	25000

Write a query to select all customer id that are borrower and depositor

- ∏ cid (BORROWER) ∪ ∏ cid (DEPOSITOR)

cid
1
2
3

Set Difference

- Set Difference is denoted by – symbol.
- Let's say we have two relations R1 and R2 and we want to select all those tuples(rows) that are present in Relation R1 but **not** present in Relation R2, this can be done using Set difference R1 – R2.
- Syntax: table_name1 - table_name2

CUSTOMER

cid	Name	mobile
1	Ram	980****
2	Hari	9841***
3	Sita	980****

BORROWER

bid	cid	Amount
1	1	10000
2	2	20000

DEPOSITOR

did	cid	Amount
1	1	20000
2	3	25000

Write a Query to select those customers present in borrower but not in depositor.

- $\prod$ cid (BORROWER) − $\prod$ cid (DEPOSITOR) →

cid
2

Cartesian Product

- **Cartesian Product in DBMS** is an operation used to merge columns from two relations into one.
- Generally, a cartesian product is never a meaningful operation when it performs alone. However, it becomes meaningful when it is followed by other operations. It is also called Cross Product or Cross Join.
- **Syntax of Cartesian product (X) = college X student**

college

cid	College
1	Divya Gyan College
2	Shankerdev College
3	Trichandra College

student

Sid	Name	cid
1	Ram	1
2	Hari	2

cid	College	Sid	Name	cid
1	Divya Gyan College	1	Ram	1
1	Divya Gyan College	2	Hari	2
2	Shankerdev College	1	Ram	1
2	Shankerdev College	2	Hari	2
3	Trichandra College	1	Ram	1
3	Trichandra College	2	Hari	2

Rename

- Rename (ρ) operation can be used to rename a relation or an attribute of a relation.
- Rename (ρ) Syntax:

  ρ(new_relation_name, old_relation_name)

CUSTOMER

cid	Name	mobile
1	Ram	980****
2	Hari	9841***
3	Sita	980****

Write a Query to fetch customer name from customer table and rename to cust_name.

$\rho_{(cust_name,\ \prod_{(Name)}(CUSTOMER))}$

cust_name
Ram
Hari
Sita

Intersection Operator

- Intersection operator is denoted by ∩ symbol and it is used to select common rows (tuples) from two tables (relations).
- Two relations R1 and R2 both have same columns and we want to select all those tuples(rows) that are present in both the relations, then in that case we can apply intersection operation on these two relations R1 ∩ R2.
- Syntax: table_name1 ∩ table_name2

CUSTOMER

cid	Name	mobile
1	Ram	980****
2	Hari	9841***
3	Sita	980****

BORROWER

bid	cid	Amount
1	1	10000
2	2	20000

DEPOSITOR

did	cid	Amount
1	1	20000
2	3	25000

- Only those rows that are present in both the tables will appear in the result set

- ∏ name (BORROWER) ∩ ∏ name (DEPOSITOR) →

cid
1

Natural Join

A natural join combines two tables (relations) based on their common attributes, matching rows with the same values in those attributes. Duplicate columns (from the common attributes) are removed in the result.

Characteristics:

Automatically identifies and matches columns with the same name and data type.

Removes redundancy by excluding duplicate columns in the output.

Natural Join

Employees:

EmployeeID	Name	DepartmentID
1	Alice	10
2	Bob	20
3	Charlie	30

Departments:

DepartmentID	DepartmentName
10	HR
20	IT
40	Marketing

Query:

`Employees ⋈ Departments`

Result:

EmployeeID	Name	DepartmentID	DepartmentName
1	Alice	10	HR
2	Bob	20	IT

Outer Join

An **outer join** is used to combine rows from two tables, including unmatched rows by filling them with `NULL`.

Types of Outer Joins:

1. Left Outer Join
2. Right Outer Join
3. Full Outer Join

Left Outer Join

Includes all rows from the left table and matched rows from the right table. Unmatched rows from the right table are filled with NULL.

Example:

Query: Employees ⟕ Departments

Result:

EmployeeID	Name	DepartmentID	DepartmentName
1	Alice	10	HR
2	Bob	20	IT
3	Charlie	30	NULL

Right Outer Join

Includes all rows from the right table and matched rows from the left table. Unmatched rows from the left table are filled with NULL.

Example:

Query: Employees ⟖ Departments

Result:

EmployeeID	Name	DepartmentID	DepartmentName
1	Alice	10	HR
2	Bob	20	IT
NULL	NULL	40	Marketing

Full Outer Join

Combines both left and right outer joins, including all rows from both tables. Unmatched rows are filled with NULL.

Example:

Query: Employees ⟗ Departments

Result:

EmployeeID	Name	DepartmentID	DepartmentName
1	Alice	10	HR
2	Bob	20	IT
3	Charlie	30	NULL
NULL	NULL	40	Marketing

UNIT - 4

Database Normalization

Introduction to Database Normalization

Definition: Normalization is the systematic process of organizing data in a database to reduce redundancy and improve data integrity.

Primary Objectives:

- Minimize duplication of data.
- Ensure data dependencies are logical and efficient.

Benefits:

- Streamlined database structure.
- Reduced data redundancy.
- Improved data consistency and integrity.
- Simplified data maintenance.

Why Normalize a Database?

Key Benefits:

- **Improved Data Integrity**: Reduces inconsistencies by ensuring accurate relationships.
- **Optimized Storage**: Saves space by avoiding duplicate entries.
- **Efficient Queries**: Enhances the speed of data retrieval.
- **Scalability:** Facilitates easier database expansion.

Functional Dependency (FD)

Definition of Functional Dependency (FD):

A functional dependency exists when one attribute in a table uniquely determines another attribute. It represents a relationship between two attributes in a relational database.

- **Notation**: A→B
  - This means that if two rows have the same value for attribute A, they must have the same value for attribute B.
    - A is called the determinant.
    - B is called the dependent.

Key Characteristics of Functional Dependency

Uniqueness of Determination

The value of the determinant uniquely determines the value of the dependent.

Example:

In a "Students" table, if StudentID → StudentName, then knowing StudentID allows us to uniquely determine StudentName.

One-Way Relationship

A functional dependency does not imply a two-way relationship.

A→B does not mean B→A.

Example of Functional Dependency

Consider a Table of Students:

StudentID	StudentName	Course	Grade
S1	Alice	Math	A
S2	Bob	Physics	B
S3	Alice	Physics	A

Functional Dependencies in This Table:

StudentID→StudentName:

- If we know StudentID, we can uniquely determine StudentName.
- For example: S1→Alice.

Example of Functional Dependency

StudentID→Course:

- This is not valid in this table because the same StudentID (e.g., S1) can appear for multiple courses (Math and Physics).

StudentID,Course→Grade:

- The combination of StudentID and Course uniquely determines Grade.
- For example: (S1,Math)→A

Types of Functional Dependencie

Trivial Functional Dependency

- A functional dependency is trivial if the dependent is a subset of the determinant.
- Example: StudentID,StudentName→StudentName.

Non-Trivial Functional Dependency

- A functional dependency is non-trivial if the dependent is not a subset of the determinant.
- Example: StudentID→StudentName

Types of Functional Dependencie

Partial Functional Dependency

- A dependency is partial if a non-prime attribute is dependent on part of a composite primary key.
- Example: StudentID,Course→StudentName (where StudentID is part of composite primary key).

Transitive Functional Dependency

- A dependency is transitive if A→B, B→C, then A→C.
- Example: StudentID→Department, Department→Dean, so StudentID→Dean.

Example of a Functional Dependency Issue

Consider This Table:

EmployeeID	EmployeeName	Department	Manager
E1	Alice	IT	John
E2	Bob	IT	John
E3	Alice	HR	Lisa

Functional Dependencies:

1. $EmployeeID \rightarrow EmployeeName$ (Valid).
2. $Department \rightarrow Manager$ (Valid).
3. $EmployeeName \rightarrow Department$ (Invalid).

Problem:

- An employee (e.g., `Alice`) may belong to multiple departments (`IT` and `HR`), so `EmployeeName` cannot uniquely determine the `Department`.
- This leads to redundancy and potential anomalies.

First Normal Form (1NF)

Definition:

A table is in 1NF if all attributes contain atomic (individual) values, and there are no repeating groups or arrays.

Rules:

- Data must be stored in a tabular format.
- Each cell should have a single value.
- Each record (row) must be unique.

First Normal Form (1NF)

Before Normalization:

```diff
 StudentID | Name    | Courses
-----------|---------|----------------
 1         | Alice   | Math, Science
 2         | Bob     | History
```

After Normalization:

```diff
 StudentID | Name    | Course
-----------|---------|---------
 1         | Alice   | Math
 1         | Alice   | Science
 2         | Bob     | History
```

Issues:

- Data retrieval becomes complex (e.g., finding students studying Math requires searching within strings).

Second Normal Form (2NF)

Definition:

A table is in 2NF if it is in 1NF and all non-key attributes are fully dependent on the primary key.

Rules:

- Eliminate partial dependencies, where non-key attributes depend on only part of a composite primary key.

Key Terminologies in 2NF

Composite Primary Key:

- A primary key that is made up of two or more columns to uniquely identify each row in the table.
- Example: In a table for Orders, the combination of OrderID and ProductID might be the composite primary key.

Non-Primary Key Attribute:

- Any column that is not part of the primary key. These attributes store additional information about the data.
- Example: ProductName in the Orders table is a non-primary key attribute.

Key Terminologies in 2NF

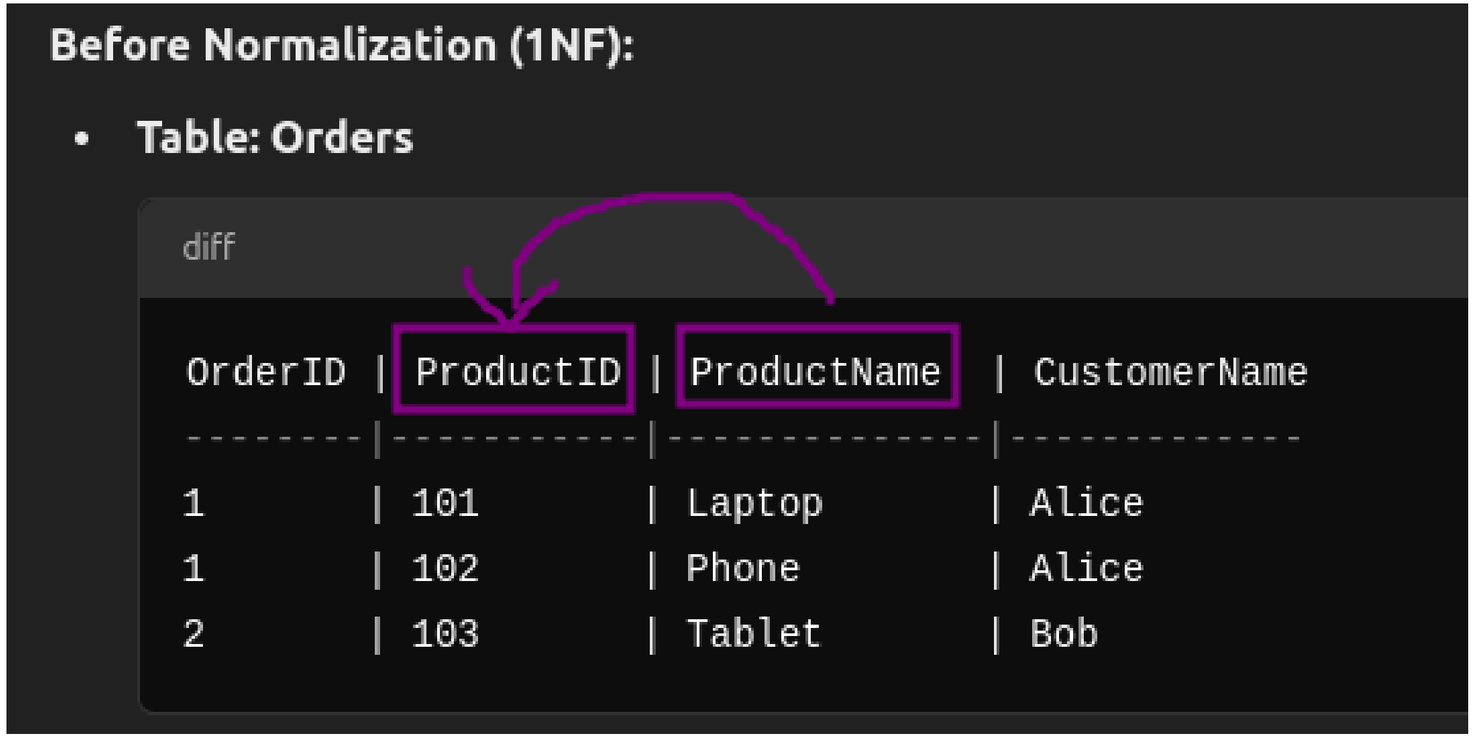
Partial Dependency:

- This occurs when a non-primary key attribute depends on part of a composite primary key, not the whole key.
- Example: If ProductName depends only on ProductID, but the primary key is a combination of OrderID and ProductID, this is a partial dependency.

Second Normal Form (2NF) - Example

Before Normalization (1NF):

- **Table: Orders**



```diff
OrderID | ProductID | ProductName   | CustomerName
--------|-----------|---------------|-------------
1       | 101       | Laptop        | Alice
1       | 102       | Phone         | Alice
2       | 103       | Tablet        | Bob
```

Issues:

- ProductName depends only on ProductID, not on the full primary key (OrderID + ProductID).
- This creates a partial dependency, violating 2NF.

Second Normal Form (2NF) - Example

Example Table with Partial Dependency:

```diff
 OrderID | ProductID | ProductName   | CustomerName
---------|-----------|---------------|-------------
1        | 101       | Laptop        | Alice
2        | 101       | Laptop        | Bob
```

Issues:

- ProductName = Laptop is stored twice, leading to wasted storage and potential inconsistencies.

Converting to 2NF

Table 1: Orders (Customer-specific data):

```diff
 OrderID | CustomerName
---------|--------------
1        | Alice
2        | Bob
```

Table 2: Products (Product-specific data):

```diff
 ProductID | ProductName
-----------|-------------
101        | Laptop
102        | Phone
103        | Tablet
```

Third Normal Form (3NF)

Definition of 3NF:

A table is in Third Normal Form (3NF) if:

- It is already in Second Normal Form (2NF).
- It has no transitive dependencies (non-prime attributes should depend only on the primary key, not on other non-prime attributes).

Key Terminologies in 3NF

Primary Key

- A primary key is the unique identifier for each record in a table.
  - Example: In a "Students" table, StudentID could be the primary key because it uniquely identifies each student.

Non-Prime Attribute

- A non-prime attribute is any column in the table that is not part of the primary key.
  - Example: In a table with primary key StudentID, attributes like StudentName and CourseName are non-prime.

Key Terminologies in 3NF

Transitive Dependency

- A transitive dependency occurs when:
  - A non-prime attribute depends on another non-prime attribute, instead of depending directly on the primary key.

StudentID → CourseID → CourseName

Here:

- StudentID determines CourseID (direct dependency).
- CourseID determines CourseName.
- CourseName depends on StudentID indirectly, creating a transitive dependency.

Example of a Table Not in 3NF

StudentID	StudentName	CourseID	CourseName	Instructor
1	Alice	C101	DBMS	Dr. Smith
2	Bob	C102	Networks	Dr. Johnson
3	Mark	C101	DBMS	Dr. Smith

Problems:

1. **Transitive Dependency:**
   - `CourseName` and `Instructor` depend on `CourseID`, not directly on `StudentID`.
   - Indirect dependency leads to redundancy.
2. **Redundancy:**
   - `CourseName` and `Instructor` repeat for every student enrolled in the same course.
3. **Anomalies:**
   - If "DBMS" is renamed, multiple rows must be updated.

Converting to 3NF

Table 1: Students Table

StudentID	StudentName	CourseID
1	Alice	C101
2	Bob	C102

Table 2: Courses Table

CourseID	CourseName	Instructor
C101	DBMS	Dr. Smith
C102	Networks	Dr. Johnson

Benefits of 3NF

1. **Reduced Redundancy:**
   - CourseName and Instructor are stored only once in the Courses table.
2. **Avoids Anomalies:**
   - Updating "DBMS" to "Advanced DBMS" is straightforward and consistent, requiring a single update in the Courses table.
3. **Better Data Integrity:**
   - Each table focuses on storing data for a single entity (e.g., students, courses).

Boyce-Codd Normal Form (BCNF)

Definition of BCNF:

A table is in Boyce-Codd Normal Form (BCNF) if:

- It is already in Third Normal Form (3NF).
- For every functional dependency (A → B) in the table, A must be a superkey.

Why BCNF Is Necessary

Even if a table is in 3NF, certain anomalies may still exist due to improper key usage. BCNF resolves these anomalies by ensuring every determinant in a functional dependency is a superkey.

Key Terminologies in BCNF

Functional Dependency (FD)

- Functional dependency represents a relationship where one attribute uniquely determines another.
  - **Notation**: A→B, meaning A uniquely determines B.
  - **Example**: In a "Students" table, StudentID → StudentName.

Superkey

- A superkey is a set of attributes that can uniquely identify each row in a table.
  - **Example**: In a "Students" table, both StudentID and StudentID + CourseID can be superkeys if each combination is unique.

Key Terminologies in BCNF

Candidate Key

- A candidate key is a minimal superkey, meaning it contains no unnecessary attributes.

Violation of BCNF

- BCNF is violated if a functional dependency exists where the determinant (A) is not a superkey.
    - **Example**: A→B but A is not a superkey.

Example of a Table Not in BCNF

EmployeeID	Project	Manager
E1	Alpha	M1
E2	Beta	M2
E3	Alpha	M1
E4	Gamma	M3

Functional Dependencies (FDs)

From the table, we can see:

- **EmployeeID → Project**
  (Each employee works on exactly one project).
- **Project → Manager**
  (Each project is managed by exactly one manager).

Example of a Table Not in BCNF

Candidate Key

- EmployeeID uniquely identifies each row → so it is the primary key.

Why Not in BCNF?

Check each FD:

- **EmployeeID → Project** ✅
  (EmployeeID is a key, so this is fine).

- **Project → Manager** ❌
  But Project is not a super key, and still determines Manager.
  This violates BCNF.

Decomposing to BCNF

1. Employee–Project

EmployeeID	Project
E1	Alpha
E2	Beta
E3	Alpha
E4	Gamma

2. Project–Manager

Project	Manager
Alpha	M1
Beta	M2
Gamma	M3

Benefits of BCNF

- **Removes Redundancy**
  - No unnecessary repetition of data.
  - Example: Manager’s name doesn’t repeat for every employee of the same project.

- **Prevents Anomalies**
  - **Insertion anomaly**: You can insert a project and its manager without needing an employee.
  - **Update anomaly**: If a project changes its manager, you update it only once.
  - **Deletion anomaly**: Deleting an employee won’t make you lose info about a project’s manager.

Benefits of BCNF

Ensures Data Consistency

- Same fact is stored only once → less chance of contradictions in data.

Better Integrity

- Data always follows the correct dependencies.
- Relationships are cleaner and more logical.

Efficient Storage

- By avoiding duplicate data, it reduces storage usage.

Easier Maintenance

- Changes in data are localized (done in one place instead of multiple rows).

4NF (Fourth Normal Form)

4NF (Fourth Normal Form) addresses issues related to multi-valued dependencies (MVDs). While BCNF eliminates anomalies caused by functional dependencies, it does not deal with scenarios where a single attribute can have multiple independent multi-valued attributes.

Key Rule for 4NF

A table is in 4NF if:

1. It is in BCNF.
2. It has no multi-valued dependencies unless they are trivial.

What is a Multi-Valued Dependency (MVD)?

A multi-valued dependency occurs when one attribute in a table determines multiple values of another attribute, independent of all other attributes.
For example:

- If a student can enroll in multiple courses and participate in multiple extracurricular activities, these two sets of information are independent.

Example of 4NF Violation

Consider the following table:

StudentID	Course	Activity
S001	Math	Football
S001	Science	Football
S001	Math	Basketball
S001	Science	Basketball

Functional and Multi-Valued Dependencies:

1. **StudentID** $\twoheadrightarrow$ **Course**: A student can enroll in multiple courses.
2. **StudentID** $\twoheadrightarrow$ **Activity**: A student can participate in multiple activities.

Here, the attributes `Course` and `Activity` are **independent of each other**, but their values are duplicated, leading to **redundancy**.

Problems with the Table

- **Redundancy**: Each combination of Course and Activity for the same StudentID is repeated unnecessarily.
- **Update Anomaly**: If a student stops participating in Basketball, you must update multiple rows.
- **Insertion Anomaly**: You cannot add a new course for a student without specifying their activities and vice versa.
- **Deletion Anomaly**: Deleting a course could unintentionally remove information about a student's activities.

Decomposing into 4NF

1. StudentCourse Table

Stores the relationship between students and courses.

StudentID	Course
S001	Math
S001	Science

2. StudentActivity Table

Stores the relationship between students and activities.

StudentID	Activity
S001	Football
S001	Basketball

Why This is in 4NF

No **Multi-Valued Dependencies**: Each table represents a single independent relationship:

- StudentID → Course
- StudentID → Activity

Eliminates Redundancy: Information is not duplicated across rows.

Prevents Anomalies:

- **Update Anomaly**: Changing a student's activity or course affects only one table.
- **Insertion Anomaly**: You can add a new course or activity independently.
- **Deletion Anomaly**: Deleting a course does not remove a student's activity information.

Thank You

UNIT - 5

Creating and Altering Database and Tables

Table of Contents

- Introduction to SQL
- Creating Database with Different Type of Arguments and Alter Database
- Creating Normal tables and Complex tables with different Type of Constraints (Key, Check, Default)
- Alter Tables: Adding and Dropping Attributes and Other Constraints
- Drop Statement: Table, Database.

Introduction to SQL

What is SQL?

SQL (Structured Query Language) is the standard language for interacting with relational databases.

Why SQL?

- To define database structure (DDL).
- To manipulate data (DML).
- To enforce data security and constraints.

SQL Operations:

- **Data Definition Language (DDL):** CREATE, ALTER, DROP
- **Data Manipulation Language (DML):** SELECT, INSERT, UPDATE, DELETE

Examples of SQL query

1. SHOW DATABASES;
2. CREATE DATABASE database_name;
3. USE database_name;
4. CREATE TABLE table_name (column_name data_type);
5. INSERT INTO table_name (column1, column2) VALUES (value1, value2);
6. SELECT * FROM table_name;
7. UPDATE table_name SET column1 = value1 WHERE condition;
8. DELETE FROM table_name WHERE condition;
9. ALTER TABLE table_name ADD column_name data_type;
10. DROP TABLE table_name;

Creating Database with Different Types of Arguments

When creating a database in MySQL, you can include various optional arguments to define its behavior. These arguments typically specify the character set (encoding) and collation (sorting rules).

Basic Syntax

CREATE DATABASE database_name
[CHARACTER SET character_set]
[COLLATE collation_name];

Different Types of Arguments

1. **CHARACTER SET (Optional):**
   - Defines the encoding used to store and retrieve text data in the database.
   - Common character sets in MySQL:
     - **utf8mb4** (Recommended for full Unicode support, including emojis).
     - **latin1** (Western European languages).
2. **COLLATE (Optional):**
   - Collation defines the rules for comparing and sorting characters in a character set. It determines how text is compared, ordered, and matched.
   - Example collations:
     - **utf8mb4_general_ci**: Case-insensitive sorting for Unicode.
     - **latin1_swedish_ci**: Default for latin1 character set.

Creating a Database

Basic Database:

```
CREATE DATABASE LibraryDB;
```

Explanation: This creates a database named LibraryDB with MySQL's default character set and collation.

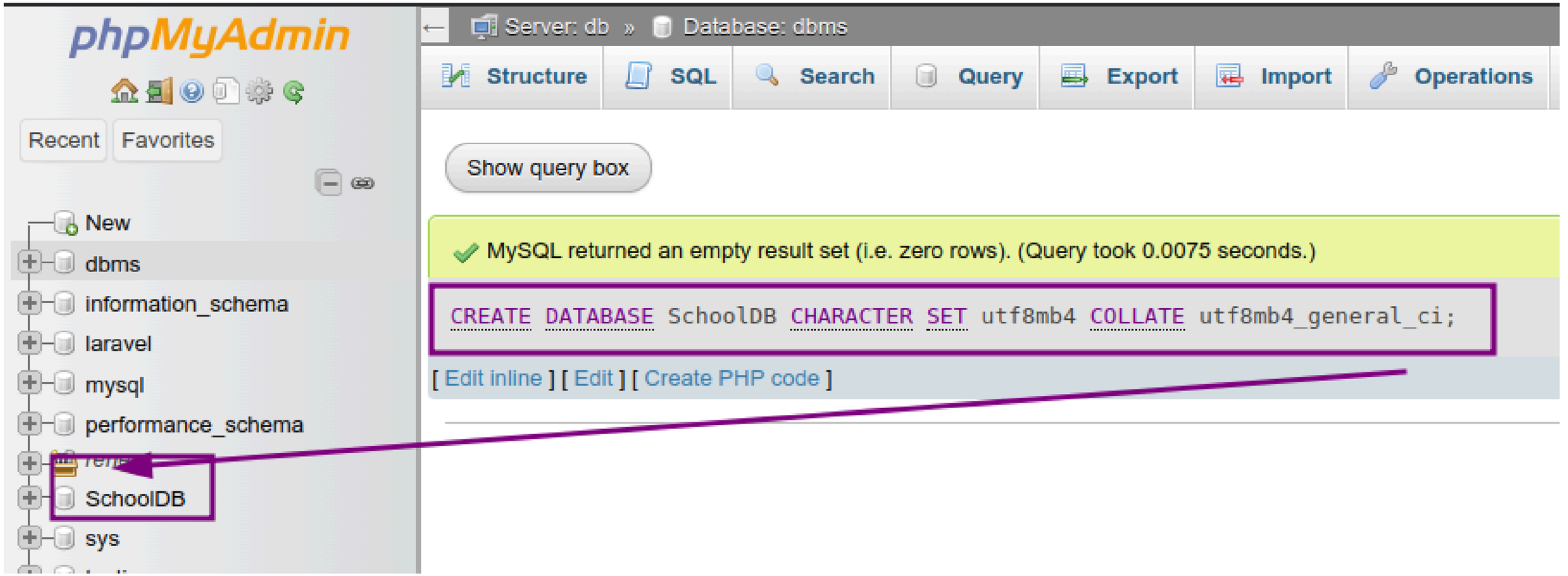
Database with CHARACTER SET and COLLATE:

```
CREATE DATABASE SchoolDB
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;
```

Explanation:

- *The database SchoolDB uses the utf8mb4 character set to support Unicode, including special characters like emojis.*
- *The utf8mb4_general_ci collation ensures case-insensitive comparisons for text data.*

Creating a Database



Creating a Database

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Table structure | Relation view

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**EmpID**	int			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**EmpName**	varchar(191)	utf8mb4_0900_ai_ci		No	*None*			Change Drop More
☐	3	**Department**	varchar(191)	utf8mb4_0900_ai_ci		Yes	*NULL*			Change Drop More
☐	4	**created_at**	timestamp			Yes	*NULL*			Change Drop More

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Table structure | Relation view

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**DepartmentID**	int			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**DepartmentName**	varchar(100)	latin1_swedish_ci		No	*None*			Change Drop More
☐	3	**Location**	varchar(100)	latin1_swedish_ci		Yes	*NULL*			Change Drop More

Altering Databases

You can change the properties of an existing database using the ALTER DATABASE statement. This is useful when you need to adjust the character set or collation.

Syntax:

```
ALTER DATABASE database_name
[CHARACTER SET character_set]
[COLLATE collation_name];
```

Example:

Changing character set

```
ALTER DATABASE SchoolDB CHARACTER SET latin1;
```

Rename Database (MySQL Workaround)

MySQL does not provide a direct SQL command or query to rename a database. If you need to rename a database, you must use workarounds like exporting and importing data or directly renaming the database files at the filesystem level (with caution).

Workaround:

- Create a new database.
- Export data from the old database using mysqldump.
- Import data into the new database.
- Drop the old database.

Why No RENAME DATABASE Command?

MySQL previously had a RENAME DATABASE command, but it was removed due to potential issues with data consistency and integrity.

For renaming:

- Use dump and restore methods (recommended).
- Avoid trying to manipulate database files directly unless absolutely necessary and you understand the risks.

If you’re looking for simplicity, tools like phpMyAdmin, MySQL Workbench, or database management scripts can automate the process.

Creating Normal Tables

Definition:

- A normal table is a basic table created to store structured data with fields (columns) and rows.
- It defines column names, data types, and optional constraints while creating the table.

Syntax:

```
CREATE TABLE table_name (
  column_name1 data_type constraint,
  column_name2 data_type constraint,
  ...
);
```

Creating Normal Tables

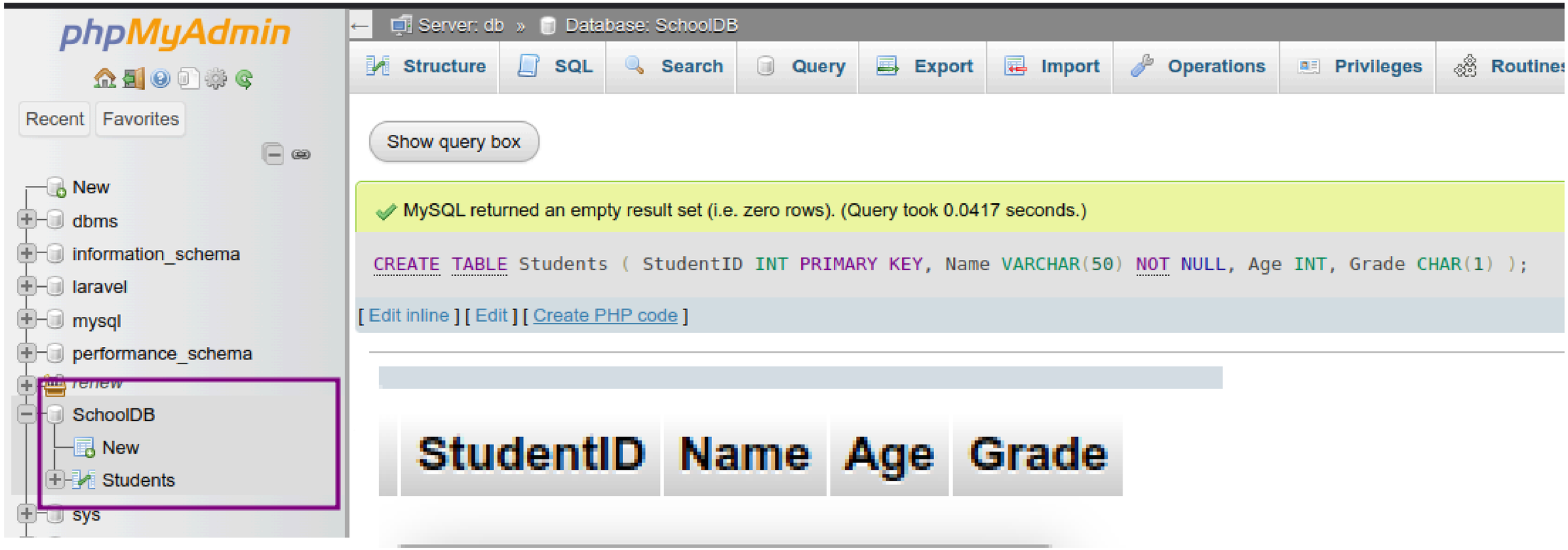
Example:

```
CREATE TABLE Students (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Age INT,
  Grade CHAR(1)
);
```

Explanation:

- StudentID: Primary Key ensures each value is unique.
- Name: Cannot be NULL, so a name must be entered for every student.
- Age: Integer value for age.
- Grade: A single-character grade field.

Creating Normal Tables



Creating Complex Tables

Definition:

- A complex table includes advanced constraints or relationships, like foreign keys, check conditions, and default values, to enforce data integrity.

Syntax:

```
CREATE TABLE table_name (
  column_name1 data_type constraint,
  column_name2 data_type constraint,
  ...
  FOREIGN KEY (column_name) REFERENCES other_table(column_name)
);
```

Creating Complex Tables

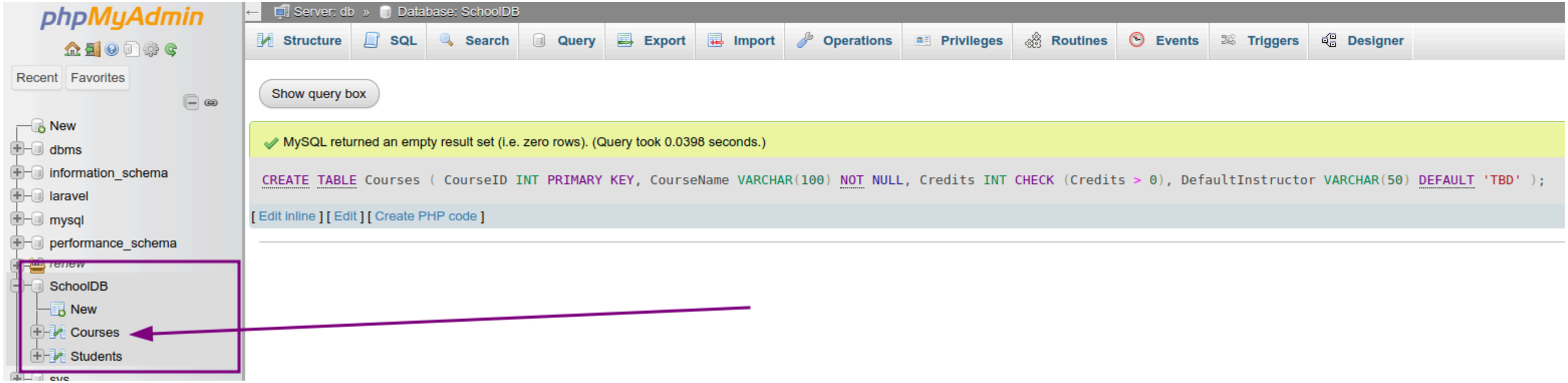
Example:

```
CREATE TABLE Courses (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(100) NOT NULL,
  Credits INT CHECK (Credits > 0),
  DefaultInstructor VARCHAR(50) DEFAULT 'TBD'
);
```

Explanation:

- CourseID: Primary Key ensures uniqueness.
- CourseName: Cannot be NULL, ensuring every course has a name.
- Credits: CHECK constraint ensures only positive values.
- DefaultInstructor: DEFAULT sets a value of 'TBD' if no instructor is specified.

Creating Complex Tables




```
SELECT * FROM `Courses`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

CourseID	CourseName	Credits	DefaultInstructor

Creating Complex Tables Examples

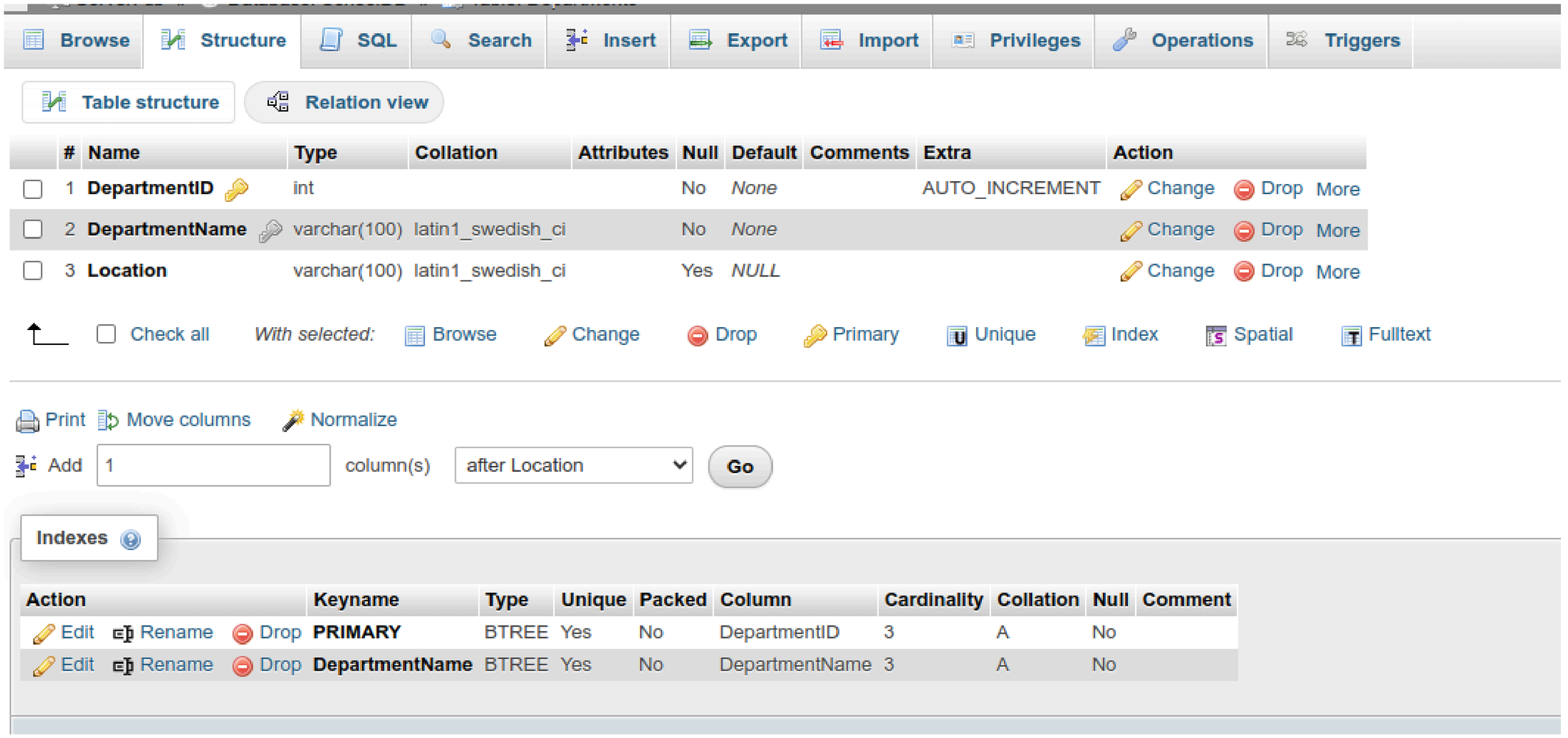
Creating Departments table:

```
CREATE TABLE Departments (
    DepartmentID INT AUTO_INCREMENT,
    DepartmentName VARCHAR(100) NOT NULL,
    Location VARCHAR(100),
    PRIMARY KEY (DepartmentID),
    UNIQUE (DepartmentName)  -- Ensures no two departments have the same name
);
```

Insert data into Departments table:

```
INSERT INTO Departments (DepartmentName, Location)
VALUES ('HR', 'New York'),
       ('Finance', 'London'),
       ('IT', 'San Francisco');
```

Creating Complex Tables Examples



Creating Complex Tables Examples

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds.)

```
SELECT * FROM `Departments`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	DepartmentID	DepartmentName	Location
Edit Copy Delete	1	HR	New York
Edit Copy Delete	2	Finance	London
Edit Copy Delete	3	IT	San Francisco

Creating Complex Tables Examples

Creating Employees table:

```
CREATE TABLE Employees (
    EmployeeID INT AUTO_INCREMENT,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    BirthDate DATE,
    Salary DECIMAL(10, 2) CHECK (Salary > 0),  -- Check constraint to ensure salary is positive
    DepartmentID INT,
    Email VARCHAR(100) UNIQUE,  -- Unique constraint for email
    HireDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  -- Default value for HireDate
    PRIMARY KEY (EmployeeID),  -- Primary Key constraint on EmployeeID
    INDEX idx_lastname (LastName),  -- Index on LastName column
    CONSTRAINT fk_department
        FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)  -- Foreign Key constraint referencing Departments table
);
```

Creating Complex Tables Examples

Insert data into Employees table:

```
INSERT INTO Employees (FirstName, LastName, BirthDate, Salary, DepartmentID, Email)
VALUES ('John', 'Doe', '1985-05-10', 60000, 1, 'john.doe@example.com'),
    ('Jane', 'Smith', '1990-07-22', 75000, 2, 'jane.smith@example.com'),
    ('Mike', 'Johnson', '1980-12-03', 80000, 3, 'mike.johnson@example.com');
```

Creating Complex Tables Examples

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Table structure | Relation view

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**EmployeeID**	int			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**FirstName**	varchar(50)	latin1_swedish_ci		No	*None*			Change Drop More
☐	3	**LastName**	varchar(50)	latin1_swedish_ci		No	*None*			Change Drop More
☐	4	**BirthDate**	date			Yes	*NULL*			Change Drop More
☐	5	**Salary**	decimal(10,2)			Yes	*NULL*			Change Drop More
☐	6	**DepartmentID**	int			Yes	*NULL*			Change Drop More
☐	7	**Email**	varchar(100)	latin1_swedish_ci		Yes	*NULL*			Change Drop More
☐	8	**HireDate**	timestamp			Yes	CURRENT_TIMESTAMP		DEFAULT_GENERATED	Change Drop More

Check all *With selected:* Browse | Change | Drop | Primary | Unique | Index | Spatial | Fulltext

Print | Move columns | Normalize

Add 1 column(s) after HireDate Go

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	**PRIMARY**	BTREE	Yes	No	EmployeeID	3	A	No	
Edit Rename Drop	**Email**	BTREE	Yes	No	Email	3	A	Yes	
Edit Rename Drop	**idx_lastname**	BTREE	No	No	LastName	3	A	No	
Edit Rename Drop	**fk_department**	BTREE	No	No	DepartmentID	3	A	Yes	

Creating Complex Tables Examples

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Triggers

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds.)

```
SELECT * FROM `Employees`
```

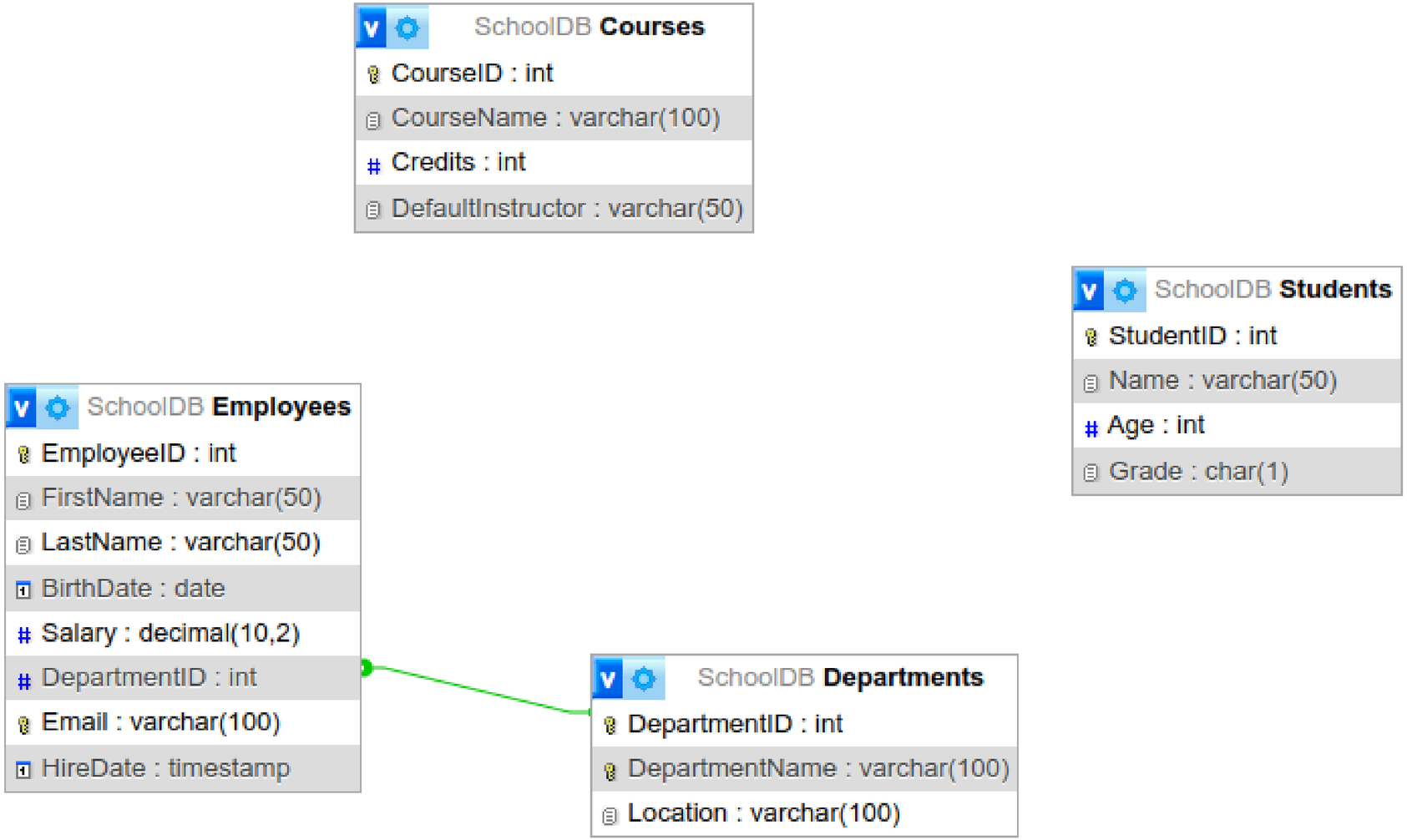
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	EmployeeID	FirstName	LastName	BirthDate	Salary	DepartmentID	Email	HireDate
Edit Copy Delete	1	John	Doe	1985-05-10	60000.00	1	john.doe@example.com	2024-12-26 17:46:09
Edit Copy Delete	2	Jane	Smith	1990-07-22	75000.00	2	jane.smith@example.com	2024-12-26 17:46:09
Edit Copy Delete	3	Mike	Johnson	1980-12-03	80000.00	3	mike.johnson@example.com	2024-12-26 17:46:09

Creating Complex Tables Examples



Types of Constraints

Key Constraints:

- **Primary Key:** Ensures uniqueness for each row.
  - StudentID INT PRIMARY KEY;

- **Foreign Key**: Links a column to another table.
  - CONSTRAINT fk_department FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)

Check Constraints:

- Ensures a column meets specific conditions.
  - Age INT CHECK (Age >= 18);
  - *SELECT CONSTRAINT_NAME, CHECK_CLAUSE FROM INFORMATION_SCHEMA.CHECK_CONSTRAINTS WHERE CONSTRAINT_SCHEMA = 'schoolDb1';* – ***view all check constraints***

Types of Constraints

Drop Check Constraints:

- Ensures a specified constraint is removed.
  - ALTER TABLE Courses DROP CONSTRAINT chk_credits;

Default Constraints:

- Sets a default value if none is provided.
  - Status VARCHAR(10) DEFAULT 'Active';

Altering Tables

Definition:

- ALTER TABLE modifies the structure of an existing table, allowing you to add, drop, or modify columns and constraints.

Syntax:

```
ALTER TABLE table_name
ADD column_name data_type constraint;

ALTER TABLE table_name
DROP COLUMN column_name;

ALTER TABLE table_name
ADD CONSTRAINT constraint_name constraint_definition;
```

Altering Tables

Adding a New Column:

- ALTER TABLE Employees ADD Country VARCHAR(100) AFTER Email;

Explanation: Adds an Country column to the Employees table

Dropping a Column:

- ALTER TABLE Employees DROP COLUMN Country;

Explanation: Removes the Country column from the Employees table.

Adding a Constraint:

- ALTER TABLE Employees ADD CONSTRAINT chk_Grade CHECK (gender IN ('M', 'F'));
- ALTER TABLE Courses ADD CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 6)

Explanation: Ensures Gender values are limited to 'M', or 'F'.

Dropping Tables and Databases

Definition:

- The DROP statement permanently deletes a table or database, removing all associated data.

Dropping a Table: Deletes the Students table and all its data.

Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Students;
```

Dropping a Database: Permanently deletes the SchoolDB database along with all its tables and data.

Syntax:

```
DROP DATABASE database_name;
```

Example:

```
DROP DATABASE SchoolDB;
```

Exercises

Create Database

- Create a database named UniversityDB with UTF8.

Create a Normal Table

- Create a table Students with these columns:
  - student_id → INT, Primary Key, Auto Increment
  - name → VARCHAR(50), NOT NULL
  - age → INT, with a CHECK constraint (age >= 16)
  - department → VARCHAR(50), with Default value 'General'

Exercises

Create Complex Tables with Constraints

- Create a table Instructors with these columns:
  - instructor_id → INT, Primary Key, Auto Increment
  - instructor_name → VARCHAR(100), NOT NULL
  - email → VARCHAR(100), UNIQUE
- Create a table Courses with these columns:
  - course_id → INT, Primary Key
  - course_name → VARCHAR(100), UNIQUE, NOT NULL
  - credits → INT, with a CHECK constraint (value must be between 1 and 6)
  - instructor_id → INT, Foreign Key referencing Instructors(instructor_id)

Exercises

Alter Table (Courses Table Only)

- Add a new column semester → VARCHAR(20), with Default value 'Fall'.
- Drop the CHECK constraint on the column credits.

Drop Statements

- Drop the table Courses.
- Drop the tables Students and Instructors.
- Finally, drop the database UniversityDB.

UNIT - 6

Manipulating and Querying Data

Table of Contents

- Adding Data with the INSERT Statement
- Retrieving Data with SELECT Statement and FROM Clause and Filter Data with WHERE Clause
- Order and Grouping Data with ORDER and GROUP by Clause and Summarizing the Select Statement
- Retrieving Data from Different Tables using: INNER JOINS, OUTER JOIN and CROSS JOIN
- Building Nested Queries
- Manipulate Data Using UPDATE Statement and Removing Rows Using DELETE Statement
- Creating and Altering View

Introduction to Data Manipulation and Querying

Definition: Data manipulation involves adding, retrieving, and managing data in a database, while querying focuses on extracting useful insights.

Key Objectives:

- Learn how to add data using the INSERT statement.
- Understand the SELECT statement to retrieve data.
- Use clauses like FROM, WHERE, ORDER BY, and GROUP BY effectively.
- Summarize data with aggregate functions.

Importance:

- Enables efficient data storage and retrieval.
- Essential for decision-making and reporting.

Adding Data with the INSERT Statement

Introduction: The INSERT statement is used to add new rows of data to a table. It is fundamental for populating databases with information.

Syntax:

- INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

Basic INSERT:

- INSERT INTO students (name, age, grade) VALUES ('John Doe', 20, 'A');

Multiple Rows INSERT: Adds multiple rows at once.

- INSERT INTO students (name, age, grade) VALUES ('Jane Smith', 22, 'B'), ('Tom Brown', 19, 'C');

Retrieving Data with SELECT Statement

Introduction: The SELECT statement retrieves data from one or more tables. It is the most widely used SQL command for accessing information.

Syntax:

- SELECT column1, column2, ...FROM table_name;

Retrieve All Columns: Fetches all columns from the table.

- SELECT * FROM students;

Retrieve Specific Columns: Fetches only selected columns.

- SELECT name, grade FROM students;

Filtering Data with WHERE Clause

Introduction: The WHERE clause is used to filter records based on specified conditions. It allows precise control over the data retrieved.

Syntax:

- SELECT column1, column2, ...FROM table_name WHERE condition;

Filter by Grade: Retrieves records where grade is 'A'

- SELECT name, grade FROM students WHERE grade = 'A';

Filter by Age Range: Fetches records within a specified age range

- SELECT name, age FROM students WHERE age BETWEEN 18 AND 22;

Filtering Data with WHERE Clause

Supported Operators:

- Comparison: =, >, <, >=, <=, !=
- Logical: AND, OR, NOT
- Pattern Matching: LIKE, %, _
- Null Checks: IS NULL, IS NOT NULL

Advanced Example:

- SELECT name FROM students WHERE name LIKE 's%';//name starts with s
- SELECT name FROM students WHERE name LIKE '%san%'; //name contains san
- SELECT name FROM students WHERE name LIKE '%sh';// name ends with sh

Filtering Data with WHERE Clause

Basic Comparison Operators:

Equality (=): SELECT * FROM employees WHERE department = 'HR';

Retrieves employees from the HR department.

Inequality (!= or <>): SELECT * FROM products WHERE price <> 100;

Retrieves products whose price is not 100.

Greater/Less Than (>, <, >=, <=): SELECT * FROM orders WHERE quantity > 10;

Retrieves orders where the quantity is greater than 10.

Filtering Data with WHERE Clause

Logical Operators:

AND: SELECT * FROM employees WHERE age > 30 AND department = 'Sales';

Retrieves employees older than 30 and in the Sales department.

OR: SELECT * FROM employees WHERE department = 'HR' OR department = 'Finance';

Retrieves employees in either HR or Finance.

NOT: SELECT * FROM employees WHERE NOT department = 'HR';

Retrieves employees not in the HR department.

Ordering Data with ORDER BY Clause

Definition:

The ORDER BY clause is used to arrange the results of a query in a specified sequence, which can be either ascending (ASC, default) or descending (DESC). Sorting helps in analyzing data by viewing the most relevant results first.

Syntax:

- SELECT column1, column2, ...FROM table_name ORDER BY column_name [ASC|DESC];

Example:

Sorting by one column

- SELECT name, age FROM students ORDER BY age ASC;

This returns a list of students ordered by their age from youngest to oldest.

Ordering Data with ORDER BY Clause

Sorting by multiple columns:

- SELECT * FROM students ORDER BY name ASC, salary DESC;

This first sorts employees by name in ascending order. Within each name, it sorts them by age in descending order.

Sorting by an expression:

SELECT name, age, 2024 - birth_year AS calculated_age FROM students ORDER BY calculated_age DESC;

Ordering Data with ORDER BY Clause

Assume the `employees` table has the following data:

id	name	department_id	salary	hire_date
1	Alice	1	50000.00	2020-03-15
2	Alice	2	60000.00	2021-07-01
3	Bob	3	70000.00	2022-01-10
4	Charlie	4	80000.00	2019-11-20

Query Result:

id	name	department_id	salary	hire_date
2	Alice	2	60000.00	2021-07-01
1	Alice	1	50000.00	2020-03-15
3	Bob	3	70000.00	2022-01-10
4	Charlie	4	80000.00	2019-11-20

Grouping Data with GROUP BY Clause

Definition:

The GROUP BY clause groups rows with the same value in a specified column, enabling aggregation for grouped data using functions like SUM, COUNT, etc.

Syntax:

- SELECT column1, aggregate_function(column2) FROM table_name GROUP BY column1;

column1: Specifies the column used for grouping.

Aggregate functions perform calculations on each group.

Example:

- SELECT age, COUNT(*) AS student_count FROM students GROUP BY age;

(Groups students by age and counts how many students belong to each age group)

- SELECT department, COUNT(*) AS total_department_employees FROM employees GROUP BY department;

(Retrieve total number of employees in each department)

Grouping Data with GROUP BY Clause

	id	name	department_id	salary	hire_date
Edit Copy Delete	1	Alice	1	55000.00	2020-03-15
Edit Copy Delete	2	Bob	2	70000.00	2021-07-01
Edit Copy Delete	3	Charlie	3	80000.00	2022-01-10
Edit Copy Delete	4	Diana	4	60000.00	2019-11-20
Edit Copy Delete	5	John Wick	3	80000.00	2022-01-10
Edit Copy Delete	6	Lara Smith	4	77000.00	2019-11-20

department_id	total_department_employees
1	1
2	1
3	2
4	2

Grouping Data with GROUP BY Clause

Combining GROUP BY with HAVING:

SELECT department_id, COUNT(*) AS employee_count FROM employees GROUP BY department_id HAVING employee_count > 1;

(*Filters out departments with fewer than 1 employees)*

Important Notes:

- Columns in SELECT must appear in GROUP BY unless used with aggregate functions.
- Avoid grouping on too many columns as it can lead to a high number of groups.

department_id	employee_count
3	2
4	2

Summarizing Data with Aggregate Functions

Aggregate functions perform calculations on a set of rows and return a single result. They are essential for summarizing large datasets.

Common Aggregate Functions:

- **SUM(column):** Computes the total of a numeric column.
- **AVG(column):** Calculates the average value of a numeric column.
- **COUNT(column)**: Counts rows in the result.
- **MAX(column)**: Returns the highest value in a column.
- **MIN(column)**: Returns the lowest value in a column.

Examples:

Total and average of a numeric column:

- SELECT SUM(salary) AS total_salary, AVG(salary) AS average_salary FROM employees;
- SELECT MAX(score) AS highest_score, MIN(score) AS lowest_score FROM exams;

Summarizing Data with Aggregate Functions

Calculating percentages using aggregate functions:

```
SELECT department, COUNT(*) * 100.0 / (SELECT COUNT(*) FROM employees) AS percentage
FROM employees
GROUP BY department;
```

Combining Clauses:

```
SELECT department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary FROM employees
WHERE salary > 60000 GROUP BY department HAVING AVG(salary) > 70000
ORDER BY avg_salary DESC;
```

- Filters employees earning more than 60,000 (WHERE).
- Groups the filtered results by department (GROUP BY).
- Calculates the count and average salary for each department.
- Filters departments with an average salary exceeding 70,000 (HAVING).
- Sorts the final results by average salary in descending order (ORDER BY).

Summarizing Data with Aggregate Functions

	id	name	department_id	salary	hire_date
Edit Copy Delete	1	Alice	1	55000.00	2020-03-15
Edit Copy Delete	2	Bob	2	70000.00	2021-07-01
Edit Copy Delete	3	Charlie	3	80000.00	2022-01-10
Edit Copy Delete	4	Diana	4	60000.00	2019-11-20
Edit Copy Delete	5	John Wick	3	80000.00	2022-01-10
Edit Copy Delete	6	Lara Smith	4	77000.00	2019-11-20

Initial table

After applying where salary > 60000

	id	name	department_id	salary	hire_date
Edit Copy Delete	2	Bob	2	70000.00	2021-07-01
Edit Copy Delete	3	Charlie	3	80000.00	2022-01-10
Edit Copy Delete	5	John Wick	3	80000.00	2022-01-10
Edit Copy Delete	6	Lara Smith	4	77000.00	2019-11-20

Summarizing Data with Aggregate Functions

Group by department_id

department_id	employee_count	avg_salary 1
3	2	80000.000000
4	1	77000.000000
2	1	70000.000000

After applying having avg_salary > 70000 in group by data

department_id	employee_count	avg_salary 1
3	2	80000.000000
4	1	77000.000000

INNER JOIN

Definition:

An INNER JOIN retrieves rows from both tables where there is a match in the specified column(s). Rows without matching data in both tables are excluded.

Basic Example: Retrieve employee names and their department names.

```
SELECT A.ID, A.Name, B.DeptName
FROM TableA A
INNER JOIN TableB B
ON A.DeptID = B.DeptID;
```

Output:

ID	Name	DeptName
1	Alice	HR
2	Bob	IT

INNER JOIN

Table A:

ID	Name	DeptID
1	Alice	101
2	Bob	102
3	Charlie	103
4	Diana	NULL

Table B:

DeptID	DeptName
101	HR
102	IT
104	Marketing
105	Sales

INNER JOIN

With WHERE Clause: Retrieve employees from the "HR" department.

```
SELECT A.ID, A.Name, B.DeptName
FROM TableA A
INNER JOIN TableB B
ON A.DeptID = B.DeptID;
WHERE B.DeptName = 'HR';
```

Output:

ID	Name	DeptName
1	Alice	HR

OUTER JOIN

Definition:

An OUTER JOIN retrieves all rows from one table and the matching rows from another table. Unmatched rows are filled with NULL.

Types:

- **LEFT JOIN:** Includes all rows from the left table and matched rows from the right table.
- **RIGHT JOIN:** Includes all rows from the right table and matched rows from the left table.

LEFT OUTER JOIN

```
SELECT A.ID, A.Name, B.DeptName
FROM TableA A
LEFT JOIN TableB B
ON A.DeptID = B.DeptID;
```

LEFT OUTER JOIN

Output:

ID	Name	DeptName
1	Alice	HR
2	Bob	IT
3	Charlie	NULL
4	Diana	NULL

All rows from Table A are included, with matching rows from Table B. If no match, NULL is shown.

RIGHT OUTER JOIN

SELECT A.ID, A.Name, B.DeptName FROM TableA A RIGHT JOIN TableB B ON A.DeptID = B.DeptID;

Output:

ID	Name	DeptName
1	Alice	HR
2	Bob	IT
NULL	NULL	Marketing
NULL	NULL	Sales

All rows from Table B are included, with matching rows from Table A. If no match, NULL is shown.

FULL OUTER JOIN

MySQL does not have a built-in FULL OUTER JOIN keyword. But you can simulate it using UNION:

```
SELECT * FROM Employees
LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID
UNION
SELECT * FROM Employees
RIGHT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

FULL OUTER JOIN

EmployeeID	FirstName	LastName	BirthDate	Salary	DepartmentID	Email	Country	HireDate	DepartmentID	DepartmentName	Location
1	Johne	Doe	1985-05-10	60000.00	1	john.doe@example.com	Nepal	2025-10-30 01:54:13	1	HR	New York
2	Janes	Smith	1990-07-22	75000.00	2	jane.smith@example.com	*NULL*	2025-10-30 01:54:13	2	Finance	London
3	Mikee	Johnson	1980-12-03	80000.00	3	mike.johnson@example.com	*NULL*	2025-10-30 01:54:13	3	IT	San Francisco
4	Alina	Johnson	1980-12-03	34000.00	3	alina@example.com	India	2025-10-30 01:54:13	3	IT	San Francisco

Result: Combines both left and right join results.

CROSS JOIN

A CROSS JOIN returns the Cartesian product of the two tables (all possible combinations of rows).

Example:

SELECT e.name AS EmployeeName, d.name AS DepartmentName

FROM employees e

CROSS JOIN departments d;

Output:

EmployeeName	DepartmentName
Alice	HR
Alice	Finance
Bob	HR
Bob	Finance

CROSS JOIN

Initial Tables :

`employees`

id	name	department_id	salary	hire_date
1	Alice	1	50000.00	2020-03-15
2	Bob	2	60000.00	2021-07-01

`departments`

id	name
1	HR
2	Finance

Nested Queries (Subqueries)

Definition:

A subquery is a SQL query nested within another query. It provides data to the outer query, allowing complex queries to be broken into simpler steps. Subqueries can appear in SELECT, FROM, or WHERE clauses.

Subquery in WHERE Clause: Find employees with a salary higher than the average salary.

SELECT Name, Salary FROM Employees WHERE Salary > (SELECT AVG(Salary) FROM Employees);

Name	Salary
Bob	6000
Diana	7000

Nested Queries (Subquery in WHERE Clause)

Definition:

A subquery is a SQL query nested within another query. It provides data to the outer query, allowing complex queries to be broken into simpler steps. Subqueries can appear in SELECT, FROM, or WHERE clauses.

Subquery in WHERE Clause: Find employees with a salary higher than the average salary.

SELECT Name, Salary FROM employees WHERE Salary > (SELECT AVG(Salary) FROM employees);

Name	Salary
Bob	6000
Diana	7000

Explanation:

The subquery (SELECT AVG(Salary) FROM Employees) calculates the average salary.

Outer query retrieves employees whose salary exceeds this value.

Subquery in SELECT Clause

Display employees along with their department names.

```
SELECT Name,
      (SELECT name
       FROM departments
       WHERE departments.id = employees.department_id) AS Department
FROM employees;
```

Name	Department
Alice	HR
Bob	IT
Charlie	Marketing
Diana	IT

Subquery in FROM Clause

Find the total salary by department.

```
SELECT departments.name as DeptName, TotalSalary
FROM (SELECT department_id, SUM(Salary) AS TotalSalary
      FROM employees
      GROUP BY department_id) AS SubQuery
JOIN departments
ON SubQuery.department_id = departments.id;
```

DeptName	TotalSalary
HR	5000
IT	13000
Marketing	5500

UPDATE Statement

The UPDATE statement modifies existing records in a table.

Increase salary of all employees in the HR department by 10%.

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id = (SELECT id FROM departments WHERE name = 'HR');
```

Before:

	id	name	salary	hire_date	department_id
Edit Copy Delete	9	Ian Clark	50000.00	2022-04-18	9

After:

	id	name	salary	hire_date	department_id
Edit Copy Delete	9	Ian Clark	55000.00	2022-04-18	9

DELETE Statement

The DELETE statement removes rows from a table.

Remove employees hired before 2021.

```
DELETE FROM employees WHERE hire_date < '2022-01-01';
```

Before:

	id	name	salary	hire_date	department_id
Edit Copy Delete	1	Alice Johnson	60000.00	2022-01-15	1
Edit Copy Delete	2	Bob Smith	60000.00	2023-03-22	2
Edit Copy Delete	3	Carol Davis	80000.00	2021-11-12	2
Edit Copy Delete	4	David Wilson	55000.00	2022-07-05	4
Edit Copy Delete	5	Eve Brown	65000.00	2023-06-01	2
Edit Copy Delete	6	Frank White	58000.00	2021-09-15	4
Edit Copy Delete	7	Grace Green	72000.00	2022-10-20	7
Edit Copy Delete	8	Hannah Hall	75000.00	2023-02-25	7
Edit Copy Delete	9	Ian Clark	55000.00	2022-04-18	9
Edit Copy Delete	10	Jack Lewis	67000.00	2021-08-30	10

DELETE Statement

Aefore:

	id	name	salary	hire_date	department_id
Edit Copy Delete	1	Alice Johnson	60000.00	2022-01-15	1
Edit Copy Delete	2	Bob Smith	60000.00	2023-03-22	2
Edit Copy Delete	4	David Wilson	55000.00	2022-07-05	4
Edit Copy Delete	5	Eve Brown	65000.00	2023-06-01	2
Edit Copy Delete	7	Grace Green	72000.00	2022-10-20	7
Edit Copy Delete	8	Hannah Hall	75000.00	2023-02-25	7
Edit Copy Delete	9	Ian Clark	55000.00	2022-04-18	9

Creating and Altering Views

Views are virtual tables created by a query. They do not store data but display data derived from the base tables.

Create View:

CREATE VIEW employee_details AS

SELECT e.name AS EmployeeName, d.name AS DepartmentName, e.salary FROM employees e

INNER JOIN departments d ON e.department_id = d.id;

	EmployeeName	DepartmentName	salary
Edit Copy Delete	Alice Johnson	Human Resources	60000.00
Edit Copy Delete	Bob Smith	Finance	60000.00
Edit Copy Delete	David Wilson	Sales	55000.00
Edit Copy Delete	Eve Brown	Finance	65000.00
Edit Copy Delete	Grace Green	Legal	72000.00
Edit Copy Delete	Hannah Hall	Legal	75000.00
Edit Copy Delete	Ian Clark	Operations	55000.00

Creating and Altering Views

Alter View:

CREATE OR REPLACE VIEW employee_details AS

SELECT e.name AS EmployeeName, d.name AS DepartmentName, e.salary, e.hire_date FROM employees e

INNER JOIN departments d ON e.department_id = d.id;

Use View:

*SELECT * FROM*

employee_details

WHERE salary > 60000;

Creating and Altering Views

Showing rows 0 - 2 (3 total, Query took 0.0009 seconds.)

```
SELECT * FROM employee_details WHERE salary > 60000;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

	EmployeeName	DepartmentName	salary
Edit Copy Delete	Eve Brown	Finance	65000.00
Edit Copy Delete	Grace Green	Legal	72000.00
Edit Copy Delete	Hannah Hall	Legal	75000.00

Thank You